

Towards effective and efficient decision-making in self-adaptive systems

Jeroen Van Der Donckt

Thesis voorgedragen tot het behalen
van de graad van Master of Science
in de ingenieurswetenschappen:
computerwetenschappen, hoofdoptie
Artificiële intelligentie

Promotoren:

Prof. dr. D. Weyns
Dr. S. Michiels

Assessoren:

Dr. ir. K. Yskout
H. Janjua

Begeleider:

Prof. dr. D. Weyns

© Copyright KU Leuven

Without written permission of the thesis supervisors and the author it is forbidden to reproduce or adapt in any form or by any means any part of this publication. Requests for obtaining the right to reproduce or utilize parts of this publication should be addressed to the Departement Computerwetenschappen, Celestijnenlaan 200A bus 2402, B-3001 Heverlee, +32-16-327700 or by email info@cs.kuleuven.be.

A written permission of the thesis supervisors is also required to use the methods, products, schematics and programmes described in this work for industrial or commercial use, and for submitting this publication in scientific contests.

Zonder voorafgaande schriftelijke toestemming van zowel de promotoren als de auteur is overnemen, kopiëren, gebruiken of realiseren van deze uitgave of gedeelten ervan verboden. Voor aanvragen tot of informatie i.v.m. het overnemen en/of gebruik en/of realisatie van gedeelten uit deze publicatie, wend u tot het Departement Computerwetenschappen, Celestijnenlaan 200A bus 2402, B-3001 Heverlee, +32-16-327700 of via e-mail info@cs.kuleuven.be.

Voorafgaande schriftelijke toestemming van de promotoren is eveneens vereist voor het aanwenden van de in deze masterproef beschreven (originele) methoden, producten, schakelingen en programma's voor industrieel of commercieel nut en voor de inzending van deze publicatie ter deelname aan wetenschappelijke prijzen of wedstrijden.

Preface

In front of you lies the final work of an important chapter in my life; studying computer science. Although I am sure that I will never stop learning, I will (probably) never be a student again. I feel grateful to have had the opportunity to study whatever I wanted without having to worry about anything else. I truly hope that I can transfer the passion and energy I put into my studies into the following chapters of my life.

I would like to thank the people without whom this would not have been possible. First, I must thank my adviser Danny Weyns who, through more than 5,000 Skype messages and many (online) meetings, molded and guided me from an eager but mostly curious student to a more focused and organized researcher. I am very grateful to Danny Weyns for teaching me how to structure proper research, push for goals, and enjoy the whole research process. Next, I want to thank the members of Danny his research group for the pleasant collaboration; Federico Quin and Omid Gheibi.

I also want to thank the people who helped colour this wonderful chapter in my life. Thank you Alexander for keeping up with me throughout these last 5 years. Also, thank you Michiel, I dearly enjoyed yours (and Alexander's) company over the many lunches we shared. A very big thanks to Thomas, who learned me to enjoy life more fully and still motivates me to pursue my passions. Thank you Emiel, for allowing me to rubber-duck about my thesis. Last but not least, I want to thank my family. Thank you to my parents for supporting me in my studies and particularly for the fact that during these five years I only had to worry about my studies and nothing else. I want to thank especially Jonas, for not only being an amazing twin brother and making the quarantine less bothersome, but also for proof-reading this work and providing me with helpful feedback.

I hope you enjoy the read, cheers!

Jeroen Van Der Donckt

Contents

Preface	i
Abstract	iv
Samenvatting	v
List of Figures and Tables	vi
List of Abbreviations and Symbols	xi
1 Introduction	1
1.1 The adaptation space problem	2
1.2 Research goal	2
1.3 Contribution and outline	5
2 Background	7
2.1 Self-adaptive systems	7
2.2 Adaptation goals	10
2.3 Machine learning	11
2.4 Evaluation case: DeltaIoT	19
3 Methodology	23
3.1 Metrics	24
3.2 DLASeR methodology	27
3.3 DLASeR+ methodology	27
4 Deep Learning For Adaptation Space Reduction	29
4.1 Tackling the two types of adaptation goals individually	29
4.2 DLASeR neural network architecture(s)	30
4.3 Combining the goals into the DLASeR framework	30
4.4 The learning pipeline	32
4.5 Conclusion	39
5 Deep Learning For Adaptation Space Reduction Plus	41
5.1 DLASeR+ Architecture	41
5.2 The learning pipeline	45
5.3 Conclusion	53
6 Evaluation	55
6.1 Evaluation setup	55
6.2 DLASeR	56

6.3	DLASer+	62
6.4	Threats to validity	71
7	Related work	73
7.1	Learning used for adaptation space reduction	73
7.2	Learning used for the reduction of features	74
7.3	Learning used for efficient decision-making	74
7.4	Search-based techniques to explore large adaptation spaces	75
7.5	Approaches for efficient verification	76
8	Conclusion	77
8.1	Future work	77
A	Additional results	81
B	Dutch summary scientific paper	85
	Bibliography	91

Abstract

Many software systems today have to operate under uncertain conditions, such as sudden changes in the availability of resources. Self-adaptation is a common approach to tackle such uncertainties. When the system goals may be compromised, the self-adaptive system has to select the best adaptation option to reconfigure itself. A common approach to select an adaptation option is to analyze the possible adaptation options, i.e., the adaptation space. When the adaptation space is large, analyzing all adaptation options using rigorous methods can be very resource- and time-consuming, or even be infeasible.

Classical machine learning has proven to be a useful manner to reduce the adaptation space. In this thesis we research the effectiveness of deep learning for the purpose of efficient decision-making. As initial work we developed a novel approach, called DLASeR (Deep Learning for Adaptation Space Reduction), aiming to indicate the applicability of deep learning in self-adaptive systems. Given the observed benefits of this initial solution, we developed a more general and extendable framework, named DLASeR+. Concretely, we support the three possible types of adaptation goals; threshold, set-point, and optimization goals. These goals are leveraged to make decision-making of the adaptation options more efficient.

We evaluate both our novel approaches on two instances of an IoT network. Results show that by only considering threshold goals, our solution outperforms the state-of-the art. Incorporating our framework for optimization and set-point goals, leads to even larger reductions of the adaptation space. We observe that applying deep learning to analyze a small subsection of the whole adaptation space leads to a very low loss in quality of the selected adaptation options. As for scalability, we notice that there is little to no significant loss in the effectiveness when we increase the size of the considered adaptation space.

List of Figures and Tables

List of Figures

2.1	Conceptual representation of a self-adaptive system (based on [67]). . .	8
2.2	Integration of MAPE-K in our conceptual representation.	9
2.3	The three types of adaptation goals.	12
2.4	An overview of a neuron in an ANN. Left: the neuron corresponds to the green circle. Right: the green rectangle represents (again) a neuron. The blue and red rectangle correspond respectively to the weighted sum of inputs and the activation function.	15
2.5	DeltaIoT deployment at the Arenberg III KU Leuven campus (borrowed from [64]). The gateway is indicated by the blue icon in the center. The variety of information collected by the different sensors is sent over multiple wireless links to this gateway.	19
2.6	Dynamic behavior of an adaptation option over 50 cycles reflected in changes of its quality properties.	20
2.7	Adaptation space of DeltaIoTv2 at a certian cycle.	21
3.1	Overview of the research methodology of this work.	23
3.2	Detailed research methodology for DLASeR. The '...' indicates that the steps are sequential to the initial steps in Figure 3.1 (up to the <i>Define Metrics</i> step).	27
3.3	Detailed research methodology for DLASeR+. The '...' indicates that the steps are sequential to the initial steps in Figure 3.1 (up to the <i>Define Metrics</i> step).	28
4.1	Overview of the deep learning architecture of DLASeR. Top: classification deep neural networks with sigmoid activation are employed for each threshold goal. Bottom: a regression deep neural networks with linear activation is employed for an optimization goal.	31

4.2	Illustration of both stages of DLASeR. Each dot represents an adaptation option. In the first stage, for each threshold goal the corresponding deep learning model predicts the relevant adaptation options sequentially (1.1 and 1.2): green dots satisfy the threshold goal, red dots do not satisfy the threshold goal. In the second stage, the relevant subset obtained from stage 1 is regressed by the deep neural network corresponding to the optimization goal, then the adaptation options are ranked (2.1). . .	33
4.3	The offline part of the DLASeR learning pipeline. In the standalone model selection process the optimal hyperparameters are determined for all models; for each goal, i.e., threshold or optimization, there is a standalone deep learning model with an accompanying scaler. The output of this process is a completely configured deep learning model and a scaler for each adaptation goal.	34
4.4	The online part of the DLASeR learning pipeline. In the training cycle, the deep learning models are trained to adapt to the current state of the system at hand. In the succeeding online cycles, adaptation space reduction is applied by means of the activities described in section 4.3. The deep learning models are maintained in these online cycles by employing online learning.	36
4.5	Runtime architecture of DLASeR. The threshold deep learner and optimization deep learner components together with their models are determined during the offline activities as shown in Figure 4.3. The runtime activities within the two learners follows the workflow of online activities shown in Figure 4.4.	40
5.1	Unified architecture of DLASeR+ for a mix of threshold and set-point goals and one optimization goal.	43
5.2	Overview of the deep neural network architecture of DLASeR+. Top: the adaptation goal specific heads correspond to the ones displayed in the legend at the bottom. Bottom: legend of the two types of output heads. Both neural network heads have an output of dimensionality 1, with sigmoid and linear activation for respectively classification and regression.	44
5.3	Illustration of the two stages of DLASeR+. Each dot represents an adaptation option. In the first (classification) stage the initial adaptation options are classified for threshold and set-point goals (1.1): blue dots satisfy the threshold goals, pink dots satisfy the set-point goals, and green dots satisfy both kind of goals. Then the intersection of adaptation options that satisfy all threshold and set-point goals are kept, i.e., the green dots (1.2). In the second (regression) stage the relevant subset of adaptation options obtained in stage 1 are ranked (2.1).	46

5.4	The offline part of the DLASeR+ learning pipeline. During model selection the optimal hyper-parameters are determined for the shared core layers (<i>Core layers selection</i>) and for each adaptation type its goal-specific heads (<i>Goal-specific head selection</i>). The output of this process is a completely configured deep learning model (consisting of the core layers with on top the various goal-specific heads) and a scaler. . .	47
5.5	The online part of the DLASeR+ learning pipeline. In the training cycle, the deep neural network model is initialized for the problem at hand. In the succeeding online cycle, adaptation space reduction is applied by means of the activities described in subsection 5.1.2. Each online learning cycle also maintains the deep learning model by employing online learning.	49
5.6	Runtime integration of DLASeR+ with MAPE-K.	54
6.1	The predicted adaptation space reduction by DLASeR for some cycle in DeltaIoTv1. The orange and blue dots represent respectively the adaption options that are predicted to meet and not meet the threshold goals T1 & T2 (represented by the two red lines) of the TTO configuration.	59
6.2	The predicted adaptation space reduction by DLASeR for two sequential adaptation cycles in DeltaIoTv2. The orange and blue dots represent respectively the adaption options that are predicted to meet and not meet the threshold goals T1 & T2 of the TTO configuration. Left: the threshold predictions of the adaptation space for some cycle x . Right: the threshold predictions of the adaptation space for the succeeding adaptation cycle, i.e., cycle $x + 1$	59
6.3	The ranking of the adaptation space by DLASeR for some cycle in DeltaIoTv1. The dots represent the adaptation options, their color corresponds to the predicted quality of the optimization goal O1 (minimize energy consumption) of the TTO configuration. Figure A.1 in the appendix is a 2D version of this figure.	60
6.4	Comparison of DLASeR's quality of the adaptation decisions for threshold goals T1 & T2 of the TTO configuration.	61
6.5	Quality of the adaptation decisions for both types of goals (T1 & T2 & O1) of the TTO configuration.	62
6.6	The energy consumption (S1) of the selected adaptation option over 274 sequential cycles for the TTS setting on DeltaIoTv2. The gray dotted lines indicate the bounds of the set-point goal.	67
6.7	The predicted ranking of the packet loss (O1) for the TSO setting on DeltaIoTv2 for a certain adaptation cycle. On the y-axis we present the verified packet loss. On the x-axis, the adaptation options are ordered according to their predicted ranking (low = left, high = right).	67
6.8	Effect on the realization of the adaptation goals for the TTO setting. . .	68
6.9	Effect on the realization of the adaptation goals for the TTS setting. . .	68
6.10	Effect on the realization of the adaptation goals for the TSO setting. . .	69

A.1	2D visualization of the predictions of the optimization goal O1 by DLASer for DeltaIoTv1. The red lines correspond to the threshold goals T1 (top) and T2 (bottom) of the TTO configuration (see Table 6.1).	82
A.2	Comparison of DLASer+'s quality of the adaptation decisions for threshold goals T1 & T2 of the TTO configuration (see Table 6.1).	83
A.3	Comparison of DLASer+'s quality of the adaptation decisions for both types of goals (T1 & T2 & O1) of the TTO configuration (see Table 6.1).	84
B.1	Geïntegreerde architectuur van DLASer+ voor een mix van threshold- en set-point doelen en één optimalisatiedoel.	88

List of Tables

2.1	Comparison of (classical) machine learning and deep learning.	13
2.2	A confusion matrix.	18
3.1	Alignment of the different aspects of the research question to evaluation metrics. Left: different aspects of research question to be evaluated. Right: metrics for each aspect.	24
6.1	Overview of the different evaluation settings for DLASer and DLASer+. The considered qualities are packet loss (PL), latency (LA), and energy consumption (EC). Each configuration is evaluated on DeltaIoTv1 and DeltaIoTv2.	56
6.2	Best grid search results for DLASer on the TTO configuration.	57
6.3	Results for DLASer and comparison with Quin F. et al. [53]. Table A.1 in the appendix presents the detailed results for the DLASer's reduction of the adaptation space on TTO configuration.	58
6.4	Best grid search results for DLASer+ on the TTO, TTS, and TSO configuration.	63
6.5	Results for the coverage of relevant adaptation options for the different evaluation settings.	64
6.6	Results for the reduction of the adaptation space for DLASer+ on the three configurations.	65
6.7	Analysis and learning time for DLASer+. Time reduction compares the total time used by DLASer+ with the time used by the reference approach.	69
A.1	Adaptation space reduction for DLASer on the TTO configuration (see Table 6.1).	81
A.2	Analysis and learning time for DLASer. Time reduction compares the total time used by DLASer with the time used by the reference approach.	81
A.3	Comparisons of the reduction for DLASer+, DLASer, and Quin F. et al. [53] for the TTO configuration (see Table 6.1).	83

B.1	Totale reductie van de adaptatieruimte en tijd reductie van DLASer+ voor de drie configuraties (zie Tabel 6.1).	89
-----	--	----

List of Abbreviations and Symbols

Abbreviations

AANR	Average ANalysis Reduction
AASR	Average Adaptation Space Reduction
AI	Artificial Intelligence
ANN	Artificial Neural Network
CPU	Central Processing Unit
DL	Deep Learning
DLAsER	Deep Learning for Adaptation Space Reduction
DLAsER+	Deep Learning for Adaptation Space Reduction Plus
DNN	Deep Neural Network
FN	False Negative
FP	False Positive
GPU	Graphical Processing Unit
IoT	Internet-of-Things
MAPE-K	Monitor-Analyze-Plan-Execute & Knowledge
MSE	Mean Square Error
NN	Neural Network
RAM	Random Access Memory
RFID	Radio-Frequency IDentification
ReLU	Rectified Linear Unit
SAS	Self-Adaptive System
tanh	hyperbolic tangent
TN	True Negative
TP	True Positive

Symbols

C	coulomb, unit of electric charge
---	----------------------------------

Chapter 1

Introduction

Many software systems operate under uncertain environments. For such systems, employing a stationary approach, i.e., an approach that does not adapt to changes in its environment, results in sub-optimal performance. For example, suppose that some web-based system runs a fixed amount of servers independent of the current load. This will result in a waste of resources when the load is very low, but when the load is high, e.g., request bursts, the amount of servers can be insufficient [36].

Self-adaptation is one prominent approach to tackle such problems [12, 67]. A self-adaptive system reasons about itself and its environment, based on its observations it determines whether adaptation is required. In the case that adaptation is required, the system adapts itself in order to meet its adaptation goals, or gracefully degrade if the goals may temporary not be achievable. In the example of the web-based system above, enhancing the system with self-adaptation enables to increase and decrease the number of servers based on the monitored environment, e.g., the current load. This results in higher user satisfaction as well as improved economical and ecological use of resources.

In this research, we apply architecture-based adaptation [21, 38, 74] using the MAPE-K feedback loop (Monitor - Analyzer - Planner - Executor - Knowledge) [33]. The feedback loop forms a managing system on top of the existing software system. In this managing system, the monitor monitors the system and its environment. The analyzer determines whether adaptation is required and if so it analyzes the options for re-configuring the system. The planner then selects the best adaptation option according to the system its adaptation goals and composes a plan for adapting the managed system. Finally, the plan is executed by the executor. The four components share common knowledge, e.g., models of the system, the adaptation goals, the set of adaptation options.

In the MAPE-K feedback loop, the analyzer and planner their decisions are guided by adaptation goals. Adaptation goals are quality constraints that should be maintained by the system. Thus, a violation of these goals will result in an adaptation action. One can depict three different types of adaptation goals; threshold goals, set-point goals, and optimization goals [59].

1.1 The adaptation space problem

Our focus is on the analysis of the adaptation options (a task of the analyzer), and ranking the adaptation options enabling to select the best option (a task of the planner). Both tasks are essential for the decision-making in self-adaptive systems. The collection of these adaptation options is called the *adaptation space*. This adaptation space consists of all the possible configurations that can be reached by applying adaptation actions to the current configuration of the system.

During the analysis and planner tasks, the MAPE-K feedback loop estimates a set of quality properties for each adaptation option of the adaptation space. Each estimated quality property corresponds with an adaptation goal. We consider threshold goals that require a system parameter to stay above/below a given threshold, optimization goals that require to minimize or maximize a system parameter, and set-point goals that require a system parameter as close as possible at a give value.

Selecting an adaptation option from a large adaptation space is often computationally expensive [12, 15, 69], in particular when rigorous analysis techniques are used, see for example [7, 45].

To determine the best adaptation option, many existing adaptation approaches apply exhaustive verification techniques at runtime. These approaches are restricted to small-scale settings, which often limits their applicability in practice [71]. Certain verification techniques provide an interesting trade-off between the analysis time and the accuracy of the verification results (e.g., by controlling the number of sample system executions generated) [77]. This makes it appropriate for systems with strict time constraints in which accuracy is not a primary concern. Furthermore, efficient analysis and planning is of utmost importance for self-adaptive systems to react quickly to changes at runtime. Finally, the adaptation space exhibits dynamic behavior that is difficult to predict upfront. This dynamic behavior is observed in terms of the qualities of the adaptation options. These qualities vary over time, as the uncertainties and the current system configuration changes.

The various factors presented above indicate that handling large adaptation spaces is challenging. It is this challenge that we call *the adaptation space problem*.

1.2 Research goal

Different techniques have been proposed to deal with the adaptation space problem. One line of research tackles the problem from the point of view of verification, see for instance [19]. Another line of research tackles the problem from the point of view of the adaptation space. *Adaptation space reduction* aims at retrieving a subset of adaptation options from an adaptation space that contains only relevant system configurations for analysis. An adaptation option is labeled relevant when it is predicted to satisfy the adaptation goals.

Existing work shows that different techniques have been applied in this approach to reduce the adaptation space, including search-based techniques [48] and feature-based techniques [43]. Recently, different machine learning techniques have been

investigated to reduce the adaptation space, see for instance [18, 53, 32, 63]. Among the applied learning techniques are decision trees, classification, regression, and online learning. However, most of these techniques rely on domain expertise to perform feature engineering, which hampers the applicability in practice. Furthermore, most existing approaches are limited to threshold goals, i.e., goals where a system quality needs to stay below or above a given value. For instance, the throughput of a web-based system should not drop below a required level. This is not sufficient for many practical systems today as they also need to address optimization goals and set-point goals. For instance, minimizing the operational cost of a web-based system and maintaining a constant energy consumption (to avoid large load changes on the electricity network) are respectively an optimization goal and a set-point goal. This leads to the following research goal¹:

Enable effective and efficient decision-making in self-adaptive systems based on threshold, optimization, and set-point goals.

With *effective*, we mean the solution should ensure that: (1) the reduced adaptation space is significantly smaller, (2) the relevant adaptation options should be covered well, (3) the effect of the adaptation space reduction on the realization of the adaptation goals is negligible. With *efficient* we mean the solution should ensure that: (4) the learning time is small compared to the time needed for analysis, (5) there is no notable effect on 1, 2, 3, and 4 for larger sizes of the adaptation space.

Note that we explicitly use decision-making instead of adaptation space reduction. With decision-making, we want to indicate that we focus not only on analysis (a task of the analyzer), but reduce the adaptation space even further up to a decision, i.e., a ranking among the proposed adaptation options (a task of the planner).

To answer the research goal, we present two novel approaches for adaptation space reduction. Both approaches leverage deep learning to enable effective and efficient decision-making in self-adaptive systems. The first approach, called *DLASeR* (Deep Learning for Adaptation Space Reduction)², can be considered as initial work indicating the applicability of deep learning in self-adaptive systems. The second approach, called *DLASeR+* (Deep Learning for Adaptation Space Reduction Plus), is the more general and extendable version of our exploratory work (*DLASeR*), aiming to answer the proposed research question. In order to keep the deep learning approaches, i.e., *DLASeR* and *DLASeR+*, up to date with the changing environment of the system, we employ (incremental) online learning.

While *DLASeR* and *DLASeR+* can handle an arbitrary mix of adaptation goals, the concrete architecture we present in this work is tailored to a set of threshold goals and one optimization goal for *DLASeR* and a set of threshold and set-point goals and one optimization goal for *DLASeR+*.

We explore deep learning for five important reasons. First, besides some exceptions, classic machine learning models are usually linear in nature, whereas deep learning can work with non-linear models. Second, considering that the (linear)

¹Research goal and research question will be used interchangeably in this text.

²We first introduced *DLASeR* in [65]

machine learning models in previous work [53] add little additional training and inference time, compared to the verification time, argues that deep learning might not introduce a large overhead either. Third, classic learning techniques usually require some form of human input for feature engineering, whereas deep learning can handle raw data, i.e., meaningful features can be automatically learned and extracted, thus making feature engineering unnecessary. Fourth, learned features and even entire models can be reused across similar tasks, known as transfer learning. This is a consequence of representation learning, which is the basic core concept that drives deep learning. We exploit representation learning in the DLASeR+ neural network architecture. Fifth, given the success of deep learning in various other domain, e.g., computer vision [57] and natural language processing [54], we were curious to explore how well deep learning can perform for an important problem in self-adaptive-systems.

1.2.1 DLASeR

In DLASeR we apply deep learning to perform effective and efficient decision-making in self-adaptive systems. We support multiple threshold goals and up to one optimization goal. The work we produce in DLASeR can be seen as *proof of concept* indicating whether deep learning is useful for adaptation space reduction.

DLASeR encompasses two approaches, tackling either threshold goals or optimization goals. For each adaptation goal, DLASeR maintains a deep neural network trained to predict the corresponding goal. In the threshold approach, classification deep neural networks are employed to reduce the adaptation space for the threshold goals. In the optimization approach, a regression deep neural network is applied to rank the adaptation options. These two approaches can be applied sequentially, i.e., apply the optimization approach after the threshold approach to further reduce the adaptation space. Hence, DLASeR does not require both kinds of goals to be present.

1.2.2 DLASeR+

DLASeR+ takes our initial work (DLASeR) to the next level by supporting not only threshold and optimization goals, but also set-point goals. Whereas our exploratory work used a model per goal, DLASeR+ works with an integrated learning architecture that uses a single model, where layers are shared and reused across different adaptation goals. Furthermore, DLASeR+ requires a single grid search process and a single prediction step in each adaptation cycle, whereas our initial approach required a grid search process for each goal and prediction for each goal in each cycle.

DLASeR+ reduces the adaptation spaces in two stages. To that end, the predicted values of its deep neural network are utilized. In the first stage, classification heads are applied to reduce the adaptation space for the threshold and set-point goals. In the second stage, a regression head is applied to rank the options of the reduced adaptation space for the optimization goal. Just as for DLASeR, depending on the

types of adaptation goals at hand, one of the stages may be omitted. Therefore, this framework easily allows combinations of the three various types of adaptation goals.

1.2.3 Evaluation

The effectiveness of DLASeR and DLASeR+ is evaluated on two instances of a self-adaptive internet-of-things (IoT) system, i.e., DeltaIoT [28]. The IoT is a particularly interesting domain to apply self-adaptation, given its complexity and high degrees of uncertainties [75]. The two instances of DeltaIoT differ in size of their adaptation space, enabling us to evaluate the different aspects of effectiveness and efficiency. To that end, we define appropriate metrics to evaluate our approaches and compare the results with other work. The results of DLASeR will be interpreted as an indication for the applicability of deep learning to enhance the decision-making in self-adaptive systems. Whereas, the results of DLASeR+ will show whether we achieved the research goal, i.e., enable efficient and effective decision-making in self-adaptive systems for the various types of adaptation goals.

1.3 Contribution and outline

In this thesis we explore the applicability of deep learning in self-adaptive systems. To that end, we developed two frameworks to perform effective adaptation space reduction. The first, DLASeR, can be considered as exploratory work. The second, DLASeR+, takes the DLASeR solution to the next level by being more general and extendable. In particular, both approaches combine deep neural network solutions into a coherent framework that is capable to tackle the different kinds of adaptation goals.

First, in [chapter 2](#) we provide relevant background on self-adaptive systems, the different kinds of adaptation goals, the essential concepts of deep learning, and the evaluation case. Then, [chapter 3](#) gives a detailed overview of the research methodology. We also present the alignment of various metrics to capture the different aspects of the research question. In [chapter 4](#) we introduce our initial work; the DLASeR framework. We develop this framework with the aim of indicating whether deep learning is applicable for decision-making in self-adaptive systems. Next, [chapter 5](#) presents our main contribution; the DLASeR+ framework. With DLASeR+ we want to achieve the research goal (see [section 1.2](#)). In [chapter 6](#) we evaluate DLASeR and DLASeR+ and use the obtained results to answer the research question. The threats to validity are also discussed. Then, [chapter 7](#) provides a structured overview of related work that focuses on adaptation space reduction. Finally, in [chapter 8](#) we conclude our observations made in this research towards effective and efficient decision-making in self-adaptive systems. We end with indicating future work.

In [Appendix B](#) a Dutch (summary) paper of this work can be found.

The code and the results of this work are publicly available at <https://github.com/jvdd/DLASeR>.

Chapter 2

Background

In this chapter the necessary background information is introduced. First, we provide essential information on (architecture-based) self-adaptive systems. Then, we explain the different kinds of adaptation goals we study in this research, i.e., threshold, optimization, and set-point goals. Next, we introduce the relevant concepts of deep learning. Finally, we present the evaluation case.

2.1 Self-adaptive systems

A self-adaptive system (SAS) is a computing system that can adapt itself autonomously to maintain a certain quality of service or business continuity. This software principle is mainly driven by uncertainties, e.g., changes in the operational environment, dynamics in the availability of resources, and variations of user goals [67].

In a high-level manner, the workings of a self-adaptive system can be described as follows. Self-adaptive systems collect additional data about itself and the uncertainties during operation. Based on this collected data, the system determines whether adaptation is required. This process of deciding whether adaption is necessary, is guided by the system its goals, e.g., maintain some certain quality, or minimize some cost. If adaptation is needed, the system reconfigures or adjusts itself to cope with the changing conditions, or if necessary to degrade gracefully. In most cases, this process is repeated until external intervention, e.g., a human halts the system. Common application domains of self-adaptation are cyber-physical systems, personalized web tasking, internet of things networks, and robotics [47, 9, 5, 17].

2.1.1 Conceptual representation

A common view for self adaptive systems, called the *internal principle*, states that these systems compromise two distinct parts [67]. The first part, called the *managed system*, is the system that is subject to adaptation. This system interacts with the environment and handles the domain concerns, i.e., concerns for which the system is built. The second part, called the *managing system*, manages the managed system.

This system is responsible for the adaptation concerns, i.e., concerns about the domain concerns.

Based on these concepts, we can introduce in Figure 2.1 a conceptual representation of a self-adaptive system [67]. As visible in the figure, this overview compromises the aforementioned managed and managing system. Furthermore, we formally add the environment to this view. The environment corresponds to the external world with which the self-adaptive system interacts and in which the effects of the system will be observed and evaluated. It is mainly this environment that introduces the uncertainties, to which the self-adaptive system is subject. The adaptation goals capture the qualities or constraints that the self-adaptive system should preserve under varying uncertainties. If one or more of these goals are violated, or expected to be violated, the managing system has to decide how to adapt the managed system. To that end, the information from both the environment and the managed system is used to make an informed decision. There are various ways to realize this decision-making process. In Section 2.1.2 we elaborate on the specific architecture that we applied in this thesis.

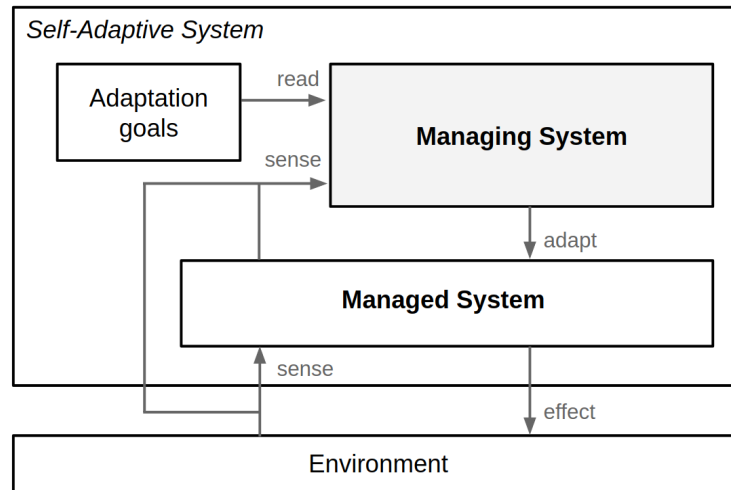


Figure 2.1: Conceptual representation of a self-adaptive system (based on [67]).

2.1.2 MAPE-K

Architecture-based adaptation is a common technique to implement self-adaptation in a software system [68, 21, 38, 74]. Concretely, the system is equipped with a feedback loop that enables the system to adapt itself at runtime to changes, maintaining its required quality goals, i.e., the adaptation goals [71]. The feedback loop uses architectural models as first-class citizens to reason about adaptation. A common way to describe the workings of the managing system is by means of the Monitor-Analyze-Plan-Execute & Knowledge loop (MAPE-K loop) [33]. MAPE corresponds to the different components that make up the feedback loop. The monitor monitors both the managed system and the environment. Based on the collected information,

the analyzer decides whether adaptation is required and if so it analyzes the options for re-configuring the system. The planner then selects the best adaptation option according to the adaptation goals and composes a plan for adapting the managed system. Finally, the executor applies the adaptation actions prescribed by the plan on the managed system. The four components share common knowledge, e.g., models of the system, the adaptation goals, the set of adaptation options, an adaptation plan, among others.

Figure 2.2 shows how we can integrate MAPE-K in our conceptual representation. The managing system of Figure 2.1 is now replaced by the MAPE-K feedback loop. Thus, the components of MAPE-K make up the basic functionality of a self-adaptive system. In the figure, we see how each MAPE component interacts with the shared knowledge. This interaction is in two directions. On the one hand the components can store models or information in the knowledge. On the other hand, each component can retrieve information from the knowledge. Note that the adaptation goals are also part of the common knowledge, since this information is important for both the analyzer and the planner.

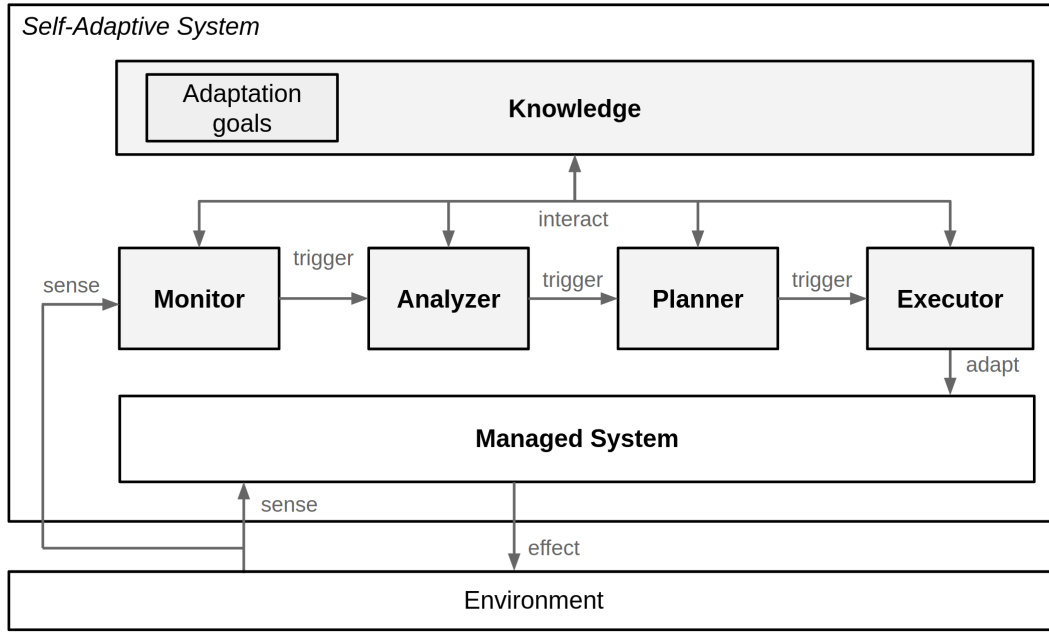


Figure 2.2: Integration of MAPE-K in our conceptual representation.

2.1.3 Adaptation space

The adaptation space is the collection of adaptation options. An adaptation option is a possible parameter setting or configuration to which the managed system can adapt to. Often, this space of adaptation options, i.e., the adaptation space, tends to be discrete.

If adaptation is required, it is the task of the analyzer and the planner of the MAPE-K feedback loop to find the best adaptation option. This task involves estimating the expected quality properties related to the adaption goals for either all or a relevant subset of the adaption options. Different techniques can be used to make these predictions. We will refer to these techniques as *verification* techniques. One approach represents the quality of the system as a parameterized Discrete Time Markov Model [7]. In this solution, a model is configured based on the current knowledge of the uncertainties and the quality properties of interest. By utilizing probabilistic model checking, one can determine the expected quality of an adaptation option. Other approaches tend to provide assurances by applying statistical model checking [69, 72, 29]. Statistical model checking is a kind of simulation approach that is quite similar to probabilistic model checking [42, 76]. Concretely, this approach involves simulating the system for finitely many executions to apply statistical hypothesis testing. This enables to make statistical statements on the evaluated qualities, and thus provide assurances.

We remark that these (predicted or verified) qualities are domain dependent, e.g., in the case of an IoT, one may be interested in the energy consumption and the latency, whereas for a service based system the qualities may include the failure rate.

Dynamic behaviour

An important feature of the adaptation space is its dynamic behavior. This behavior is observed in terms of the qualities of the adaptation options, i.e., the qualities of each adaptation option may change over time. The main cause for this dynamic behavior are the uncertainties of the system itself and the environment. The uncertainties cause for changes in two different parts; (1) the uncertainties have a considerable effect on the environment, and (2) due to the uncertainties the system may adapt itself to another state. Therefore, it is expected that the effects of adaptation options (measured in qualities) will be different depending on the state of the environment (1), and the state of the managed system (2). Hence, it is due to this dynamic property that frequent verification is required, i.e., analysis and verification at runtime, by one of the approaches mentioned in the previous section.

2.2 Adaptation goals

An adaptation goal refers to a quality requirement of the managed system that the managing system should preserve. A violation (or an expected violation) of one or more of the adaptation goals triggers an adaptation, aiming to satisfy all the adaptation goals again.

Shevstov S. et al. [59] state, in the context of control-theoretic adaptation, that one can depict three different types of adaptation goals; (1) threshold goals, (2) optimization goals, and (3) set-point goals. Intuitively, a *threshold goal* states that the value of some quality property of the system should be below (or above) a certain threshold value. A *set-point goal* states that some value of a quality property of the system should be kept at a certain value, i.e., the set-point, with an margin

of at most ϵ . Finally, an *optimization goal* states that some value of a quality property of the system should be minimized (or maximized). Formally, we define the satisfaction of goals by the adaptation options as follows (for the explanation, we consider only threshold goals below a value and optimization goals that minimize a value; the other variants are defined similarly). Consider C the set of possible configurations, each representing an adaptation option of the adaptation space. We refer to a particular quality property q_x of an adaptation option $c_i \in C$ as $c_i[q_x]$, with $x \in \{t, s, o\}$ referring to quality properties related to threshold, set-point, and optimization goals respectively. Further, consider a threshold goal g_t , a set-point goal g_s , and an optimization goal g_o . The set of adaptation options T that satisfy g_t , the set S that satisfy g_s , and the set O that satisfy g_o are then defined as follows:

$$T = \{c_i \in C \mid c_i[q_t] < g_t\} \quad (2.1)$$

$$S = \{c_i \in C \mid g_s - \epsilon \leq c_i[q_s] \leq g_s + \epsilon\} \quad (2.2)$$

$$O = \{c_i \in C \mid c_i[q_o] < c_j[q_o], \forall c_j \in C \setminus c_i\} \quad (2.3)$$

As an illustration Figure 2.3 shows the three types of adaptation goals for a toy example. Suppose that our self-adaptive system is defined by two qualities; Quality 1 and Quality 2, we visualize each adaptation option in terms of these qualities. Hence, each dot in these figures represents an adaptation option based on the values of Quality 1 and Quality 2 (we are here mainly interested in Quality 1). Figure 2.3a displays the threshold goal **Quality 1** < 6.5. All the dots below the red dotted line represent adaptation options that satisfy the threshold goal. The adaptation options above the red dotted line do not meet this goal. In Figure 2.3b we show the set-point goal **Quality 1** = 6 +/- 0.2. The two blue dots are the only adaptation options that meet this set-point goal, since both are in the range defined by the set-point value and the margin ϵ . The other adaptation options have values out of the range [5.8%, 6.2%] and do not satisfy the goal. Finally, Figure 2.3c presents an optimization goal, concretely we want to minimize Quality 1. The darker the dot, the lower Quality 1, hence, the dot at the bottom has the lowest value for Quality 1.

2.3 Machine learning

Artificial intelligence (AI) is a domain of computer science that focuses on the development of computing techniques or algorithms that enable machines to perform certain cognitive functions. Such cognitive functions can range from solving specific problems to observe trends in data. It is important to see AI as a very broad domain, with many waves throughout its history. In the last decade machine learning became very popular, mainly but not exclusively due to the rise of deep learning [39]. We want to stress that machine learning is a lot more than just deep learning, and that depending on the case, deep learning might not be the best choice.

Machine learning is defined as a subset of artificial intelligence, where algorithms learn to perform certain tasks by leveraging large amounts of structured data. The main classical machine learning tasks are; classification, regression, clustering, and dimensionality reduction.

2. BACKGROUND

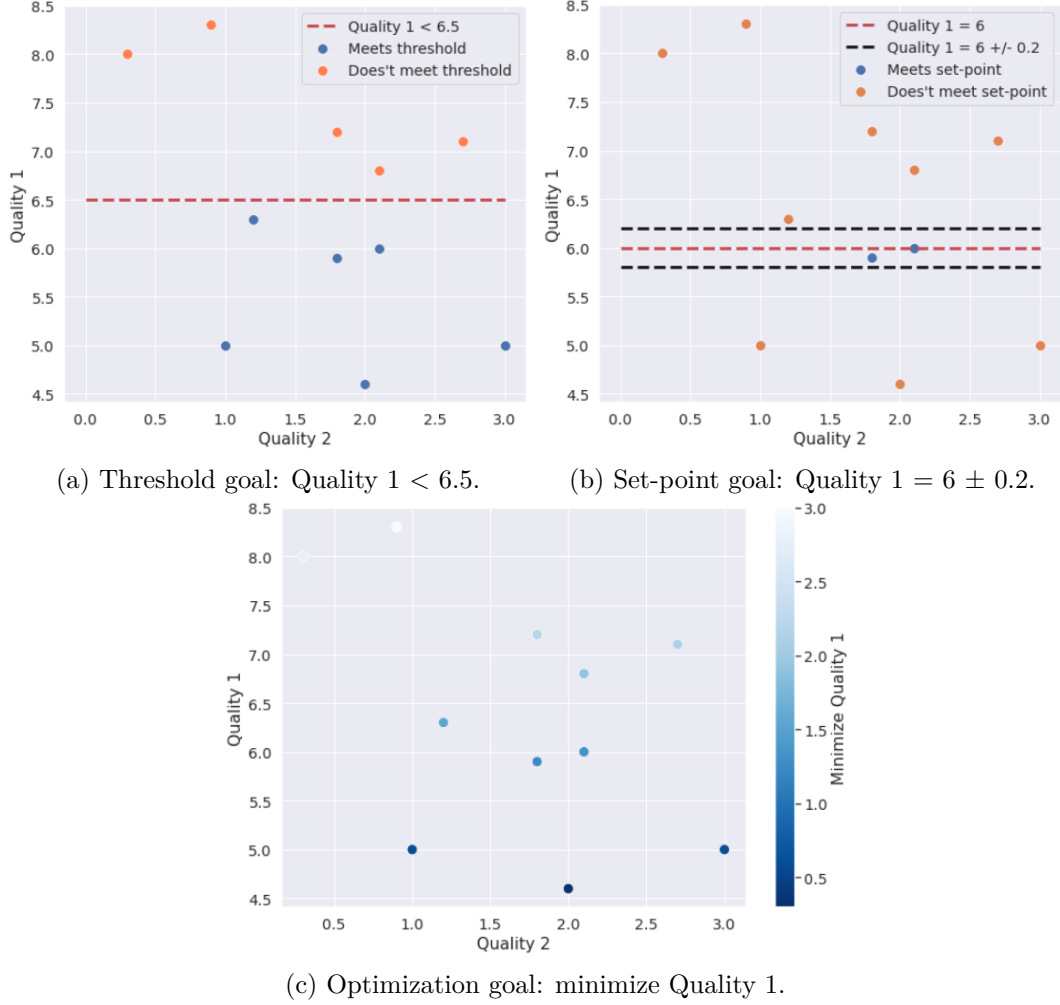


Figure 2.3: The three types of adaptation goals.

2.3.1 Deep learning

Deep learning (DL) refers to a subset of machine learning mechanisms in which the learning methods are based on *deep artificial neural networks* (ANNs) leveraging *representation learning* [25]. The two key terms in this definition are artificial neural networks and representation learning. We elaborate on both of these concepts in the following sections. First, we outline the principal differences between machine learning and deep learning. Then, we introduce the fundamental concepts of deep learning.

Deep learning vs classical machine learning

Table 2.1 presents the main discrepancies between (classical) machine learning and deep learning.

Table 2.1: Comparison of (classical) machine learning and deep learning.

(classical) ML	DL
Structured data	Raw data
Computational lightweight	Computational expensive
Isolated, single task	Transfer learning
Requires generally less data	Requires generally more data
Easier to interpret	Harder to interpret

The first key difference is in terms of the input data. Machine learning requires structured data, i.e., tabular data or data with typed fields defined in a fixed format. In order to obtain such data formats, several preprocessing steps might be necessary. Furthermore, classical ML algorithms often rely on complex feature engineering. This process usually consists of an exploratory data analysis step, followed by a dimensionality reduction technique for easier processing, to finally select the best features. Quite often, this process requires the knowledge of a domain expert to construct meaningful features for the data. In the case of deep learning, there is no need to (extensively) preprocess the input data. The input data can directly be passed, in a raw format, to the neural network. The deep neural network (DNN)¹ itself can learn to perform such feature transformations and selections [41, 25]. This omits the time-consuming and expensive feature preprocessing step.

The second difference is related to computational cost. Classical machine learning tends to be generally more computational lightweight than deep learning. Deep learning algorithms are optimized to run in a highly parallelized manner on high-end Graphical Processing Units (GPUs). Such GPUs are deployed in a dedicated setup with fast Central Processing Units (CPUs) and Random Access Memory (RAM) to deliver large amounts of data as quickly as possible. Such setups are very expensive and consume enormous amounts of energy [61], hindering on the edge deployment on everyday devices. As an extreme example, consider OpenAI Five. This DL system has been trained for over 10 months on thousands of GPUs to demonstrate that self-play reinforcement learning can achieve superhuman performance on a difficult task [49]. Note that this project was one of the largest deep learning projects up-to-date. This is largely in contrast with classical ML algorithms, as these can often be trained on just a decent CPU. Hence, there is no need for dedicated hardware. This enables to iterate faster and thus try out many techniques in a shorter period of time.

As third difference, we consider the reusability of trained models. In the case of traditional machine learning, models are isolated and purely trained on specific tasks. If one wants to tackle multiple tasks in the same domain, separate isolated models have to be trained without considering past learned knowledge in other (related) tasks. Thus, no knowledge is retained which can be transferred from one

¹Artificial neural network, deep neural network, and neural network will be used interchangeably in this text.

model to another. In deep learning, this situation is completely different. Transfer learning allows leveraging knowledge from previously trained models (in the same domain) [50]. This corresponds to reusing a large part of a trained ANN and only fine-tune, i.e., retrain, a small part of it. This makes the learning process faster, more accurate and fewer data is needed.

The fourth difference is in terms of the amount of available data. On small amounts of data, classical ML algorithms tend to perform significantly better than deep learning solutions. But, in the case that a lot of data is available, DNNs will scale much better with more data than classical ML algorithms, i.e., more data allows training larger models resulting in better performance [25]. This is also one of the main drivers behind the principle of data-augmentation, i.e., augment your available data by applying clever changes to it.

The last key difference has to do with interpretability. As stated before, many deep learning models are considered as black boxes [3]. The lack of interpretable DL solutions thus hinders acceptance in several application domains, e.g., physicians do not accept DL health solutions due to their limited interpretability [44]. This problem does not apply that much in classical ML, where most algorithms are more mature making them easier to interpret and understand.

Fundamentals of deep learning

As stated in the definition at the start of this section, deep learning is driven by two main concepts; (1) artificial neural networks, and (2) representation learning. We elaborate on both in the following paragraphs.

Artificial neural networks are the core of deep learning. These networks are commonly described in terms of their input and output, which are the only externally observable parts of an ANN. Both the input and the output are represented as a vector of numbers. A neural network applies a chain of mathematical operations aiming to transform the input to the desired output. In the case that the data is categorical, one needs to encode the input (and possibly output) data to a vector, e.g., with 1-hot-encoding.

The basic building block of a neural network is a neuron, also called a perceptron. Figure 2.4 presents an overview of the functionality of a neuron. In the case of a fully connected neural network, a neuron is connected to all the inputs and produces a single output.² Figure 2.4a displays the interface of a neuron. We observe that the neuron is connected to all the inputs and produces a single output. Figure 2.4b shows in detail how the neuron produces this output. To obtain the output, the neuron applies two operations sequentially:

$$z = \vec{w} \cdot \vec{x} + b \tag{2.4}$$

$$y = f(z) \tag{2.5}$$

with $\vec{x}$ the current input vector, $\vec{w}$ the weights associated with the inputs, b a constant, z an intermediate computed value, $f(\cdot)$ an activation function, and y the output. First,

²We limit this introduction to only fully connected neural networks (in the context of supervised machine learning), other types of ANNs are out of scope for this thesis.

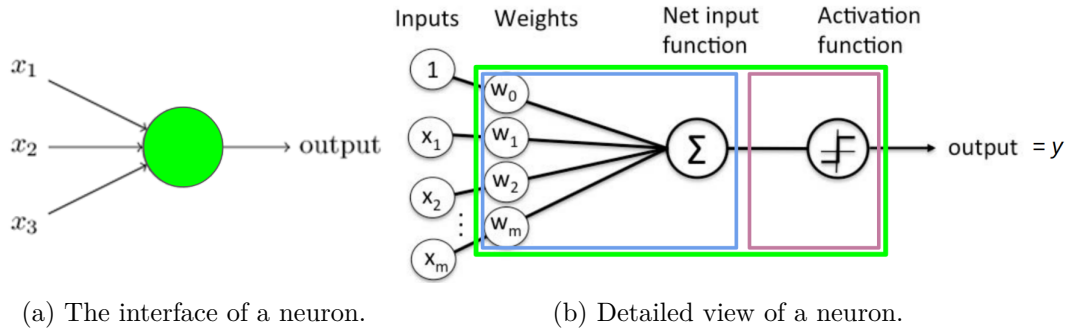


Figure 2.4: An overview of a neuron in an ANN. **Left:** the neuron corresponds to the green circle. **Right:** the green rectangle represents (again) a neuron. The blue and red rectangle correspond respectively to the weighted sum of inputs and the activation function.

the weighted sum (z) of all the inputs is calculated, see Equation 2.4 (indicated by the blue rectangle in Figure 2.4b). The bias b in this equation corresponds to weight w_0 in the figure. The weights associated with the inputs are learnable. Intuitively, these weights are reflecting how important the input is. When the weighted sum is computed, the neuron applies an activation function f , see Equation 2.5 (indicated by the red rectangle in Figure 2.4b). This function introduces a non-linearity in the neural network. Since a weighted sum is a linear function, and a linear combination of linear functions is still a linear combination, a neural network without a non-linear activation function is only able to learn linear combinations of input values. Non-linearity in learning is very important to learn more complex concepts. Examples of common non-linear activation functions are hyperbolic tangent ($\tanh$), sigmoid, and rectified linear unit (ReLU).

A deep neural network is structured in two dimensions; width and depth. The depth of a network corresponds to the number of layers, whereas each layer is defined by its width, i.e., the number of neurons present in that layer. As rule of thumb, the depth is calculated as the number of hidden layers + 1 (for the output layer). Hidden layers are the layers of neurons that are positioned strictly between input and output layers. These layers are called hidden since they aren't visible to the external world. Each neuron in some layer x is connected to all the neurons in the preceding layer $x - 1$. Hence, the name given to this type of neural networks; *fully connected neural networks*.

In the next section we elaborate on how deep neural networks are trained, but first we shall clarify what is actually learned. The key concept behind ANNs is representation learning [25]. Representation learning learns representations of input data, typically through transformations, which makes it easier to perform certain tasks [6]. In deep learning, representations are learned through multiple non-linear transformations on the input data. These learned representations provide abstract and useful representations for the learning task, e.g., classification, regression, or prediction. Representation learning is important because of various reasons. First, the

performance of any DL model is critically dependent on the learned representations. Next to this, representation learning explains why in many solutions a simple linear layer is stacked on top of a complex block with many non-linear layers, e.g., state-of-the-art BERT for NLP-tasks [16]. A simple linear layer is thus sufficient to perform the (hard) task described by the output on the rich learned representations of the input. Finally, the most important property of representation learning, which is a direct consequence of the previous characteristic, is that it enables transfer learning, i.e., reuse of the learned representations, complemented with fine-tuning for task specific learning [50].

Training a deep neural network

To give a high-level overview of how ANNs are trained, we can distinguish three different steps; (1) forward propagation, (2) loss calculation, and (3) backpropagation.

In the *forward propagation* step, the input data is passed through the network, i.e., from input (layer) to output layer. The neurons apply their transformations to the data they receive from all the connected neurons of the previous layer and pass the result on to the connected neurons of the next layer. In the final layer, the neurons apply their transformations to obtain the (predicted) output.

In the second step, a loss function estimates the loss (or error). The loss captures how good or bad the predicted result is (i.e., the output predicted by the network) compared to the correct result (i.e., the expected output). To obtain a loss as close as possible to zero, the parameters of the neurons are gradually adjusted in the next step.

As final step, the loss value is propagated backwards through the network, hence *backpropagation*. You can think of this step as recursively applying the chain rule to compute the gradients all the way from the output to the input of the network. These gradients tell in which direction the loss is increasing and indicate the influence of the neuron's computations on the loss. Based on this information, the neurons update their weights using an optimizer that exploits these gradients. Examples of commonly used optimizers are Adam, RMSprop, and Nadam [56].

hyperparameters of a neural network

When training a neural network, there are several non-learnable parameters that control the learning process. We refer to these non-learnable parameters as *hyperparameters*. The most relevant hyperparameters are the number of layers of the deep learning model (i.e., network depth), the number of neurons in each layer, the batch size, the learning rate, and the optimizer.

The depth of the network and the number of neurons in each layer influence the complexity and also the total number of learnable parameters, which affects the learning time. The batch size defines how much of the data samples shall be looked at before doing a single update of the model (based on the collective outputs of the model). A large batch size results in fewer updates, where a small one results in many updates. This affects the granularity of the learning. Batch size is highly

correlated with the learning rate. The learning rate determines the degree of the update impact that are performed during the learning process. This will affect the learning speed. Finally, the model optimizer influences the parameter updates. To that end it utilizes the learning rate and the local gradient at the neurons. Setting all these hyperparameters properly increases a deep neural network its generalization performance.

In addition to the aforementioned hyperparameters, there is also the scaler algorithm. This hyperparameter is not part of the neural network, but transforms the input-data before it is passed through the neural network. Normalizing the data of the features is a common preprocessing step in machine learning. Learning algorithms tend to perform better when scaling is applied to the data [31]. Shanker M. et al. [58] demonstrated these effects also for neural networks. Therefore, selecting the right scaler algorithm is essential to maximize the performance of your DL model.

2.3.2 Classification and regression

Classification and regression are two important learning tasks that require different predictions. The difference between both lies in what should be predicted. Suppose that we have some machine learning model M that is described by the following equation;

$$\vec{y} = M(\vec{x}) \quad (2.6)$$

with $\vec{x}$ the input and $\vec{y}$ the output of the learner, and M a function that maps input to output. In the case of classification, M needs to map the input to a set of classes that are represented by different labels with different encodings (e.g., rainy = 0 and sunny = 1). A classification with only two labels is called binary classification, while multi-class classification has more than two labels. Regression on the other hand, maps the input data to continuous real values. Whereas for classification the predicted output is not ordered, for regression the output is ordered. In sum, the main difference between both tasks is that the output variable $\vec{y}$ for classification is categorical (or discrete), while for regression it is numerical (or continuous).

2.3.3 Evaluation metrics

Due to the fundamental discrepancy between classification and regression ML models, different metrics are required to capture their quality. First, we present the F1 metric that is applicable to classification algorithms. Then, we elaborate on the Spearman's rank correlation coefficient, which is (somewhat) relevant for regression.³

F1 score: precision and recall

The quality of a classification algorithm is often expressed in terms of precision and recall. We introduce their definitions intuitively in the case of binary prediction.

³A common metric to evaluate regression models is mean squared error (MSE). However, we are actually more interested in the ranking of the regressed output, which is not captured by MSE. The Spearman correlation coefficient does capture this ranking.

Suppose that an algorithm predicts whether a certain adaptation option meets a threshold goal. In this binary classification setting we mean with relevant that an adaptation option meets its threshold goal, with not relevant vice versa. Then, precision is the fraction of selected adaptation options that are relevant, while recall is the fraction of the relevant options that are selected from the total number of relevant options. Hence, precision and recall are a measure of relevance.

Table 2.2: A confusion matrix.

		Predicted	
		True	False
Actual	True	TP	FN
	False	FP	TN

Precision and recall can be derived from a confusion matrix. Table 2.2 displays such a matrix. In order to define precision and recall, we introduce the terms true positive (TP), false positive (FP), false negative (FN), and true negative (TN). In the confusion matrix we observe that TP corresponds to the number of correct positively classified items. FP indicates the number of incorrect positively predicted items. FN corresponds to the number of positive items that were predicted as negative. Finally, TN expresses the amount of correctly negatively classified items. Now we can introduce the formulas for precision and recall:

$$Precision = \frac{TP}{TP + FP} \quad (2.7)$$

$$Recall = \frac{TP}{TP + FN} \quad (2.8)$$

In both equations 2.7 and 2.8 the numerator is equal to the number of correct positively predicted values. For precision, we take the total number of positive predicted values as denominator. Whereas for recall, we consider the number of actual positive values as denominator.

The F1 score combines both metrics with equal importance into a single metric. Concretely, the F1 score is defined the harmonic mean of precision and recall [23]:

$$F1 = 2 * \frac{precision * recall}{precision + recall} \quad (2.9)$$

Spearman correlation

Spearman correlation, or Spearman's rho, is a non-parametric metric that measures rank correlation. We use this as a metric to capture the ranking of the predicted values of quality properties of regression models. The Spearman correlation essentially converts the predicted values to numeric ranks and then assesses how well a monotonic function describes the relationship between the predicted ranks and the true ranks. Spearman's rho is defined as [37]:

$$\rho_{xy} = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{\sqrt{n \sum x_i^2 - (\sum x_i)^2} \sqrt{n \sum y_i^2 - (\sum y_i)^2}} \quad (2.10)$$

with x the predicted values, y the actual values, and n the number of observations. The result is a value between 0 and 1 (for an increasing monotonic trend, or between 0 and -1 for a decreasing monotonic trend). Large errors are penalized harder. For example, a swapping of the first and third rank in the prediction results is worse (Spearman's rho will be closer to 0) compared to a swapping of the first and second rank.

2.4 Evaluation case: DeltaIoT

DeltaIoT is a reference Internet-of-Things (IoT) application that has been deployed at the Arenberg III Campus of KU Leuven [28]. This IoT network has been developed to support research on self-adaptation, i.e., evaluate new self-adaptive approaches and compare their effectiveness with others, e.g. cost-benefit-analysis at runtime [64]. Next to the real physical setup deployed by VersaSense⁴, DeltaIoT also comprises a simulator for offline experimentation.



Figure 2.5: DeltaIoT deployment at the Arenberg III KU Leuven campus (borrowed from [64]). The gateway is indicated by the blue icon in the center. The variety of information collected by the different sensors is sent over multiple wireless links to this gateway.

Figure 2.5 shows the physical setup of DeltaIoT. We observe that three different sensors are utilized. radio-frequency identification (RFID) sensors are used to provide access control to labs, passive infrared sensors monitor the occupancy of several buildings, and heat sensors are employed to sense the temperature. The data of these sensors is relayed to the gateway by means of wireless multi-hop communication.

⁴VersaSense website: www.versasense.com

2. BACKGROUND

Each sensor is plugged into a battery-powered mote, i.e., a networked tiny embedded computer. The motes take care of the routing of sensor data to the gateway. The communication in DeltaIoT is time-synchronized and organized in cycles with a fixed number of slots. Neighbouring motes are assigned such slots during which they can exchange packets. Motes collect data (locally generated or received from other motes) in a buffer. When a mote gets a turn to communicate with another mote, it forwards the packets to the other mote. Packets that cannot be sent remain in the buffer until the mote is assigned a next slot.

DeltaIoT has three main qualities in which we are interested; packet loss, latency, and energy consumption. For these qualities we define corresponding adaptation goals, e.g, average latency of packages delivery should not exceed a predefined percentage and energy consumption of the network should be minimized. Ensuring such goals is challenging since the IoT network is subject to various types of uncertainty. The main uncertainties are interference along network links and changing load in the network, that is, motes only send packets when there is useful data, which may depend on conditions in the environment that are difficult to predict. Figure 2.6 illustrates how the three quality properties of one adaptation option change over a series of cycles. Thus, the figure highlights the dynamic behavior (see section 2.1.3) for an instance of DeltaIoT.

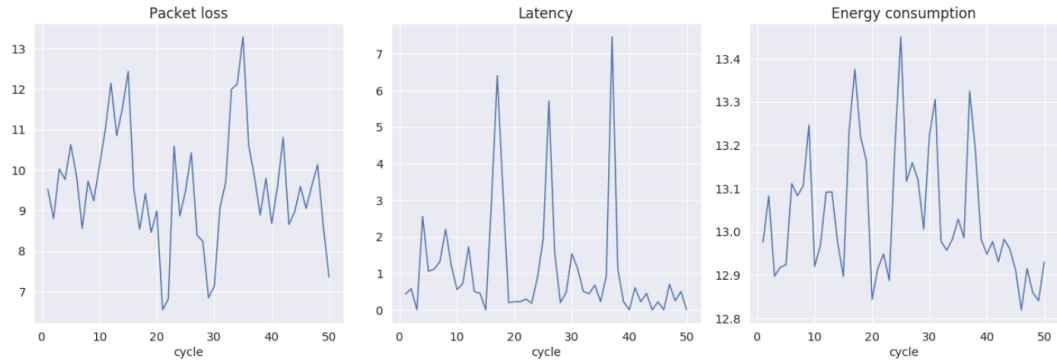


Figure 2.6: Dynamic behavior of an adaptation option over 50 cycles reflected in changes of its quality properties.

The IoT network can be adapted using two parameters: power setting and link distribution. The power setting parameter refers to the setting of the transmission power of each mote. The options are discretized as an integer in the range of 1 to 15. This parameter introduces trade-offs, e.g., using more energy to transmit packets will reduce packet loss but also decrease the lifetime of the battery. The link distribution parameter refers to the way data packets are relayed from the motes to the gateways. Observe in Figure 2.5 that several motes have multiple links over which data can be sent. We refer to the distribution of packets sent by a mote to its parents as the link distribution. If a mote has only one parent, it is obvious that it relays 100% of its packets through that single parent. But when a mote has multiple parents, the distribution of packets over the different links to the parents can be changed.

Note that the sum of the distributions should remain 100%. By changing the link distribution, paths with more interference can be avoided. However, this may cause delays at the buffers of the nodes along these paths.

DeltaIoT comprises a real physical setup and a realistic simulator for offline experimentation. For practical reasons, we use the simulator for the evaluation of our approaches, since extensive experimentation on a physical IoT deployment is particularly time-consuming. This simulator also enables to easily alter the layout of the IoT network. We consider two instances of DeltaIoT. The first one, referred to as *DeltaIoTv1*, consists of 15 nodes (shown in Figure 2.5); the second instance, referred to as *DeltaIoTv2*, consists of 37 nodes. The larger IoT network is more challenging in terms of the number of possible configurations to adapt the system, i.e., the adaptation space. The adaptation space of DeltaIoTv1 contains 256 possible configurations, while DeltaIoTv2 has 4096 configurations. Figure 2.7 illustrates the adaptation space of DeltaIoTv2 at a certain cycle, displayed in terms of the latency and packet loss qualities.

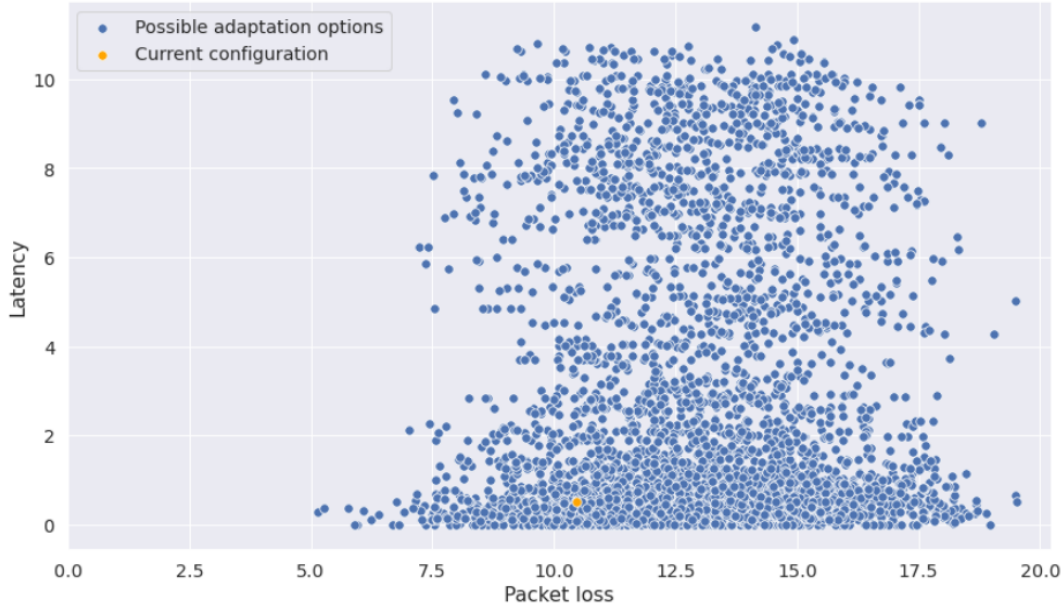


Figure 2.7: Adaptation space of DeltaIoTv2 at a certain cycle.

Chapter 3

Methodology

To realize the research goal, we followed a systematic methodology as shown in Figure 3.1.

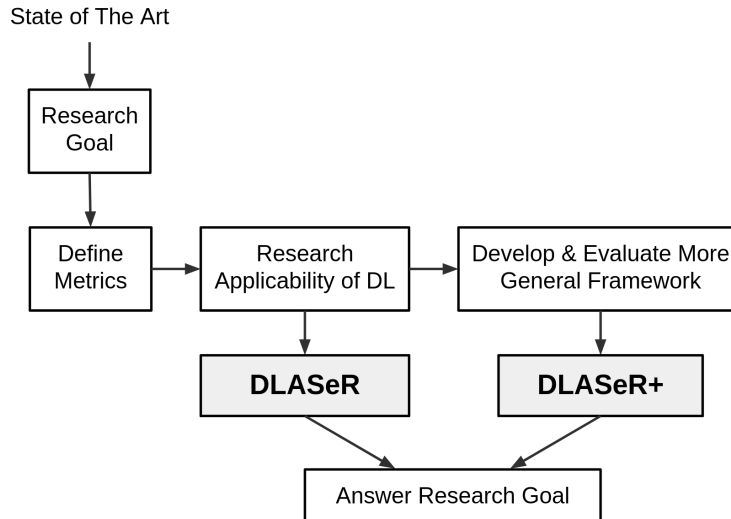


Figure 3.1: Overview of the research methodology of this work.

Based on the state-of-the-art, we defined in [section 1.2](#) the research goal. In the following section, we specify the metrics that enable us to measure the effectiveness and efficiency, allowing to compare our work with other approaches.

Earlier work applied classical machine learning techniques to reduce large adaptation spaces based on (only) threshold goals [53]. Inspired by recent progress in the area of deep learning, we want to research its applicability to enhance the decision-making process in self-adaptive systems. To that end, we propose **DLASeR**; an initial approach for effective and efficient decision-making in self-adaptive systems, based on multiple threshold goals and an optimization goal. The main purpose of this work is to obtain an indication whether deep learning can effectively reduce large adaptation spaces, without compromising the system goals. DLASeR can thus be considered as an initial step towards effective and efficient decision-making.

Once we have shown the applicability of deep learning, we developed **DLASer+**; a more general and extendable framework that enhances decision-making in self-adaptive systems, based on multiple threshold and set-point goals and an optimization goal. We evaluate its performance for various combinations of these three types of goals on the two instances of the DeltaIoT installation. Based on the obtained results, we answer the research goal by means of the metrics specified in the next section.

While DLASer and DLASer+ can handle an arbitrary mix of adaptation goals, we focus in this work on systems that can have multiple threshold (and set-point goals), but only one optimization goal. In particular, the DLASer and DLASer+ architecture maps each adaptation goal to a rule. Decisions are then made by first applying the rules of the threshold (and set-point) goals, and then applying the rule of the optimisation goal. Handling multiple optimization goals using different decision-making techniques, for instance based on Pareto optimality [27] are outside the scope of this work.

3.1 Metrics

To answer the research question, we determined metrics that allow us to evaluate the different aspects of effectiveness and efficiency of DLASer+ (and DLASer). Table 3.1 summarizes the metrics for each aspect.

Table 3.1: Alignment of the different aspects of the research question to evaluation metrics. **Left:** different aspects of research question to be evaluated. **Right:** metrics for each aspect.

Effectiveness	
Coverage relevant adaptation options	F1 score (for threshold and set-point goals) Spearman’s rho (for optimization goal)
Reduction adaptation space	Average adaptation space reduction (AASR) Average analysis reduction (AANR) Total reduction
Effect on realization adaptation goals	Differences in mean values of quality properties of the goals with and without learning
Efficiency	
Adaptation time	Time used for analysis (time in s) Time used for learning (time in s) Time reduction for analysis with learning (%)
Scalability	Metrics above for increasing size of adaptation space

We use different metrics to capture the coverage of relevant adaptation options for classification and regression. In particular, we use the F1 score for classification and Spearman’s rank correlation coefficient (Spearman’s rho) for regression, see [subsection 2.3.3](#). We define average adaptation space reduction (AASR), average analysis reduction (AANR), and total reduction to capture and compare the reduction of

adaptation spaces. To capture the effect on the realization of the adaptation goals, we consider the distribution differences, observed with box plots. We measure the time used for analysis and learning and compare this with the time that would be necessary to analyze the complete adaptation space. Finally, for scalability, we apply the different metrics for scenarios with large adaptation spaces.

We further elaborate on AASR, AANR, and total reduction. The other basic metrics were introduced in the background, see [subsection 2.3.3](#).

3.1.1 Average Adaptation Space Reduction

To capture how well the adaptation space is reduced, we define a new metric called average adaptation space reduction (*AASR*). *ASR* (adaptation space reduction) is defined as:

$$ASR = 1 - \frac{selected}{total} \quad (3.1)$$

with *selected* the number of adaptation options selected by learning and *total* the total number of adaptation options. For instance, an adaptation space reduction of 70% means that after learning only 30% of the original adaptation space is considered for analysis. We obtain the average adaptation space reduction (*AASR*) by taking the micro-average of the adaptation space reduction (*ASR*) over multiple adaptation cycles. *AASR* is a particularly suitable metric for stakeholders as it covers the high-end goal of adaptation space reduction.

Remark that the average adaptation space reduction is determined by the system's adaptation goals. In particular, for the three types of goals considered in this work, the *AASR* is determined by the threshold and set-point goals, corresponding to the percentage of adaptation options that are predicted to be conform with these goals. For systems with only optimization goals, the *AASR* is zero (since the selected and total number of adaptation options are the same).

It is also interesting to note that the *AASR* depends on the restrictiveness of both threshold and set-point goals. Suppose that the threshold and set-point goals are not very restrictive, thus many adaptation options will comply with the adaptation goals. In this case, the reduction will be rather small. In the other case, for very restrictive threshold and set-point goals the opposite is true. A larger reduction for these goals can be expected.

3.1.2 Average Analysis Reduction

To capture the effect of the adaptation space reduction on the effort required for analysis, we define a new metric called average analysis reduction (*AANR*). *ANR* (analysis reduction) is defined as:

$$ANR = 1 - \frac{analyzed}{selected} \quad (3.2)$$

with *analyzed* the number of adaptation options that have been analyzed and *selected* the number of adaptation options selected by learning. For instance, an

analysis reduction of 90% means that only 10% of the adaptation options selected by learning were analyzed to find an option to adapt the system. We obtain the average analysis reduction (*AANR*) by taking the micro-average of the analysis reduction (*ANR*) over multiple adaptation cycles. Similarly to *AASR*, *AANR* also covers a high-end goal of adaptation space reduction.

Note that the *AANR* depends on the analysis approach that is used to find an adaptation option from the reduced adaptation space (and not on the constraints imposed by the adaptation goals as for *AASR*). In particular, the *analysis reduction* corresponds to 100% minus the percentage of selected adaptation options that have to be analyzed until an adaptation option is found that meets all the threshold and set-point goals. For systems that include threshold, set-point and an optimization goal, the selected adaptation options are analyzed in the order predicted for the optimization goal. For systems with only threshold and set-point goals, the most basic method to analyze the selected adaptation options is by randomly picking (without replacement) one. For this specific case, we randomly shuffle the selected options in each cycle, resulting in a more representative *AANR* score. Remark that for systems with a large adaptation space and only an optimization goal, $AANR \approx 100\%$ (since *analyzed* is 1 and *selected* is a large number equal to the size of the adaptation space).

3.1.3 Total reduction

Besides *AASR* and *AANR*, we also measure the fraction of the total adaptation space that was analyzed. We define this *total reduction* as:

$$Total\ reduction = 1 - \frac{analyzed}{total} \quad (3.3)$$

with *analyzed* the number of adaptation options that have been analyzed and *total* the total number of adaptation options. For instance, a total reduction of 95% means that only 5% of all the adaptation options were analyzed to find an option to adapt the system.

Note that this metric combines *AASR* and *AANR* as follows:

$$Total\ reduction = 1 - (1 - AASR) \times (1 - AANR) \quad (3.4)$$

In this equation $(1 - AASR)$ corresponds to the fraction of selected adaptation options and $(1 - AANR)$ corresponds to the fraction of analyzed adaptation options among the selected ones. As this metric combines both *AASR* and *AANR*, the impact of the restrictiveness of the threshold and set-point goals is mitigated (since it captures a combination of the adaptation space reduction and the analysis approach).

The total reduction metric is particularly interesting when threshold and set-point goals are present. In that case it summarizes both *AASR* and *AANR*. When there is just a single adaptation goal and that goal is an optimization goal, then the total reduction is equivalent to *AANR* (as the *AASR* will be 0%).

3.2 DLASeR methodology

In Figure 3.2 we show an elaborate overview of the research methodology of the DLASeR framework. In this initial work, we continue from the research goal and defined metrics of Figure 3.1. First, we define how to tackle the two types of adaptation goals individually, i.e., either threshold goals or an optimization goal. In this step we define the neural network architecture, which is a standalone neural network for each adaptation goal. Then, we decide how to combine these two types of adaptation goals into the framework. Next, we devise DLASeR’s learning pipeline that applies deep learning for the two types of adaptation goals. The pipeline is then instantiated for each of the threshold goals and the optimization goal. We evaluate the effectiveness of DLASeR on the two instances of the DeltaIoT setup. To that end we apply the metrics that were defined earlier in the methodology, enabling us to compare with other related approaches and to answer the research goal. In the case of DLASeR, the research goal is to validate whether deep learning can enhance decision-making in self-adaptive systems.

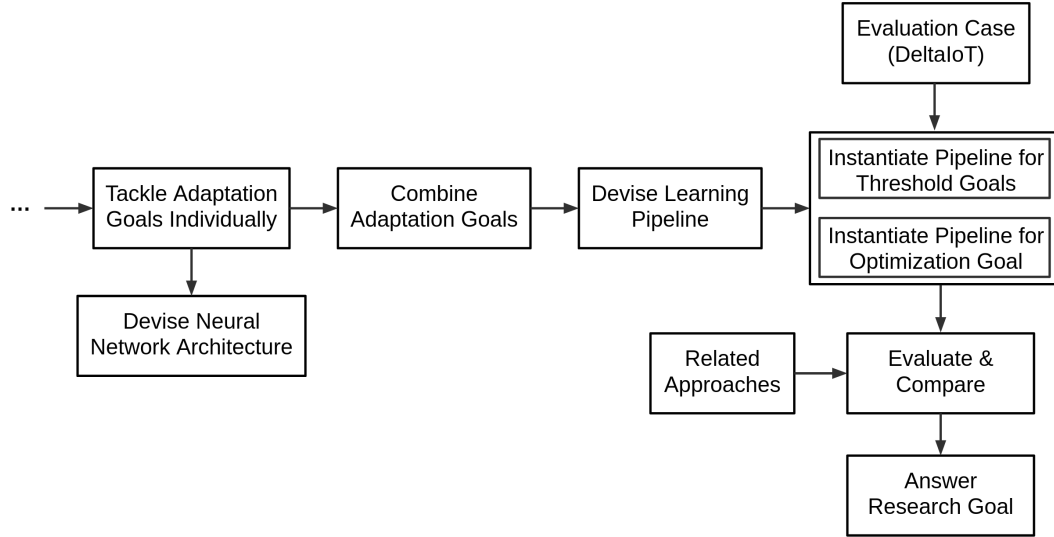


Figure 3.2: Detailed research methodology for DLASeR. The ‘...’ indicates that the steps are sequential to the initial steps in Figure 3.1 (up to the *Define Metrics* step).

3.3 DLASeR+ methodology

In Figure 3.3 we display the detailed research methodology of the DLASeR+ framework. Again, we continue from the research goal and the defined metrics of Figure 3.1. First, we define the DLASeR+ architecture that is able to deal with threshold goals, set-point goals, and an optimization goal. We then devise DLASeR+’s learning pipeline that applies deep learning for the three types of adaptation goals. Next, we instantiate this pipeline for various setups for the two evaluation cases of DeltaIoT

3. METHODOLOGY

applying different combinations of adaptation goals. We then evaluate DLASer+ and compare with representative related approaches. Finally, we answer the research question, and discuss the extent to which we meet our objectives.

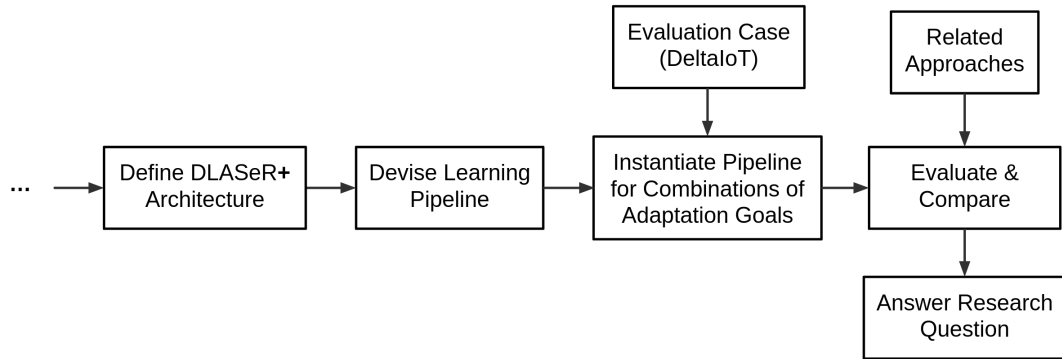


Figure 3.3: Detailed research methodology for DLASer+. The '...' indicates that the steps are sequential to the initial steps in Figure 3.1 (up to the *Define Metrics* step).

Chapter 4

Deep Learning For Adaptation Space Reduction

In this chapter we present our novel (initial) work that aims to indicate the applicability of deep learning in self-adaptive systems. This initial approach is called "Deep Learning for Adaptation Space Reduction" (DLASeR). We outline how this approach enhances decision-making based on two types of adaptation goals.

First, we focus on how each of the two types of goals, i.e., threshold and optimization goals, is tackled individually. Then, we describe how each adaptation goal is tackled by its standalone neural network. Next, we present how the adaptation space is reduced if there is a combination of these two types of adaptation goals. Then, the learning pipeline is introduced, defining how the configuration of the learning models is determined, how they are initialized, and how they are maintained when they are deployed. Finally, we conclude by integrating the previous aspects in the MAPE-K architecture.

4.1 Tackling the two types of adaptation goals individually

In this section we explain the specific learning technique that is applied by DLASeR to handle threshold and optimization goals. We focus on two techniques; classification and regression.

4.1.1 Threshold goals

For handling threshold goals we use classification techniques. This is also the same technique that has been applied (for threshold goals) in previous work with classical machine learning [53]. Concretely we apply binary classification; 1 if the adaptation option meets the threshold goal, 0 in the other case.¹

¹When we say that an adaptation option meets or optimizes a goal, we mean that the quality (for which the goal was introduced) of that adaptation option satisfies or optimizes the goal.

Thus, based on threshold goals, we reduce the adaptation space to only the valid classified adaptation options. We also support multiple threshold goals, in that case the adaptation space reduction corresponds to the intersection of the predicted reduction for each threshold goal.

4.1.2 Optimization goal

We handle an optimization goal by regressing the quality that has to be optimized. Based on the regressed values, the adaptation options are ranked. From this ranking we infer which adaptation option maximizes or minimizes the adaptation goal.¹

The adaptation space reduction occurs by selecting the adaptation that is predicted to optimize the goal. Thus, in the case of a single optimization goal, the adaptation space is reduced to only one adaptation option.

4.2 DLASeR neural network architecture(s)

The core of the DLASeR approach is a collection of standalone neural networks. With standalone we want to indicate that each deep neural network corresponds to exactly one adaptation goal. Thus, each neural network aids, in a standalone manner, adaptation space reduction (based on its corresponding goal). This also means that each neural network predicts (and is trained) for exactly the same input data.

As visible in Figure 4.1, we use two different types of neural network architectures (corresponding to the techniques presented in the previous section); classification and regression neural networks. Both types are fully connected deep neural networks that produce a single output, i.e., the output layer has a dimensionality of 1. They differ in terms of their expected output. The former architecture is utilized for classification of adaptation options based on threshold goals. In this case we apply a sigmoid activation in the last layer (see Figure 4.1a), since an output value between 0 and 1 is expected, based on which (by means of some threshold, e.g., 0.5) we perform classification. The latter architecture is utilized for regressing the quality of an optimization goal. Based on these regressed values, we rank the adaptation options. Here, we apply a linear activation in the final layer (see Figure 4.1b), since we want to regress a quality.

4.3 Combining the goals into the DLASeR framework

The sections above described how both types of adaptation goals are handled separately, i.e., by either classification or regression techniques. Now, we explain how the two kinds of adaptation goals can be combined. To that end, we consider the adaptation goals as rules. Note that adaptation space reduction corresponds to the number of verified adaptation options. Thus, the task is to select adaptation options for verification in a clever manner.

In order to combine the goals, we apply the two neural network architectures sequentially. To that end, the DLASeR solution works in two stages. In the first stage,

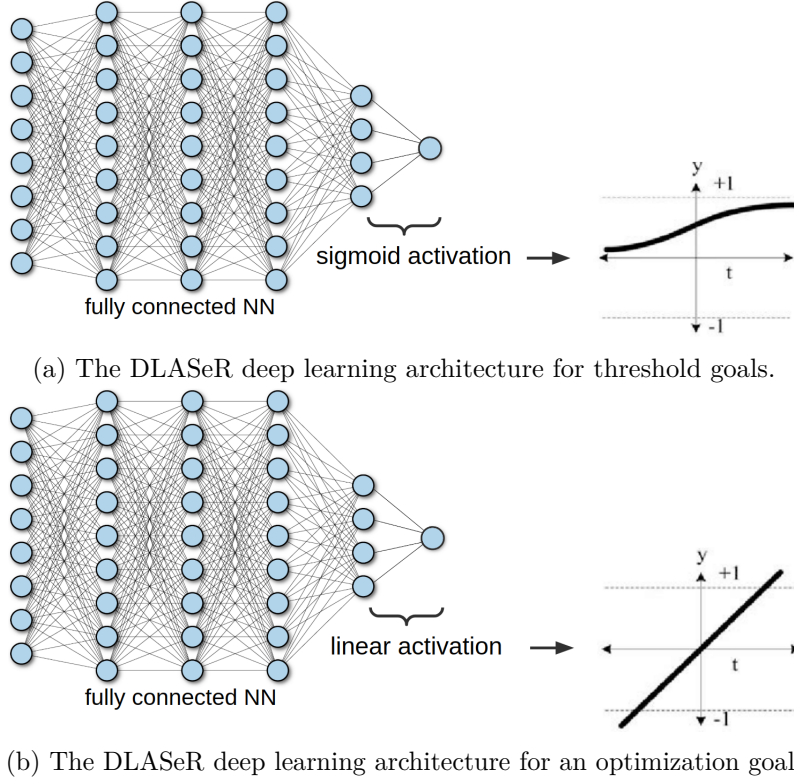


Figure 4.1: Overview of the deep learning architecture of DLASeR. **Top:** classification deep neural networks with sigmoid activation are employed for each threshold goal. **Bottom:** a regression deep neural networks with linear activation is employed for an optimization goal.

classification deep neural networks are employed to reduce the adaptation space for the threshold goals. In the second stage, a regression deep neural network is applied to rank the options, further reducing the adaptation space for the optimization goal. Note that, as these two stages are sequential, either of them can be omitted when the corresponding type of goal is not present.

4.3.1 Stage 1: threshold goals

In the first stage we tackle threshold goals. These goals are handled by classification. We will further refer to this stage as the *threshold stage*. Adaptation space reduction is performed by reducing the total collection of adaptation options to the ones that are classified as valid according to the threshold goals.

In this stage, we have for each threshold goal a classification deep neural network. These neural networks reduce the adaptation space sequentially.² The first neural

²Actually, the deep neural networks can predict in parallel as the predictions for each goal are independent. Based on the predicted results, we can calculate the reduced adaptation space as the intersection of the as valid classified adaptation options for all threshold goals.

network (corresponding to the first³ threshold goal) classifies all the adaptation options, i.e., the complete adaptation space. Then, the second network (corresponding to the second³ threshold goal) classifies the as relevant predicted adaptation options (obtained by the first classification NN). This process is repeated until all threshold goals are covered. At the end, we obtain the reduced adaptation space according to the threshold goals.

4.3.2 Stage 2: optimization goal

In the second stage we handle the optimization goal (as stated above, our architecture supports only up to one optimization goal). We will further refer to this stage as the *optimization stage*. We continue on the reduced adaptation space obtained from stage 1, i.e., the regression neural network only regresses the optimization quality of the as relevant predicted subset.

In this stage, the adaptation space reduction occurs by verifying the adaptation options in the ranked order, i.e., from low to high in the case of minimizing, and vice versa in the case of maximizing. We keep verifying adaptation options until one is found that satisfies (determined by verification) all the other goals.

4.3.3 Illustration of the DLASeR stages

In Figure 4.2 we illustrate how these two stages perform adaptation space reduction.

Stage 1: When threshold goals are present, we can perform adaptation space reduction by means of the sequential steps of stage 1. In the illustration we consider two threshold goals, resulting in two sequential steps. Since for each threshold goal there is a corresponding deep neural network, we chain these networks in order to perform adaptation space reduction. To that end, each network classifies the subset of relevant adaptation options predicted by the preceding network, where the first network classifies the whole adaptation space. The classified output of the final deep neural network corresponds to the reduced adaptation space, i.e., the adaptation options that were predicted relevant for all the threshold goals.

Stage 2: Then, if there is an optimization goal present, we perform the second stage. A regression neural network predicts the quality (for which the optimization goal was defined) on the subset of adaptation options obtained in stage 1. We rank the adaptation options based on the regressed values. It is in this ranked order that the adaptation options are verified until one is found that satisfies all the adaptation goals.

4.4 The learning pipeline

We introduce now the complete DLASeR approach. DLASeR spans two parts of a learning pipeline: an offline and online part. We defined a generic learning pipeline that can be used for multiple threshold goals and up to one optimization goal. For

³Chosen in arbitrary order (without replacement).

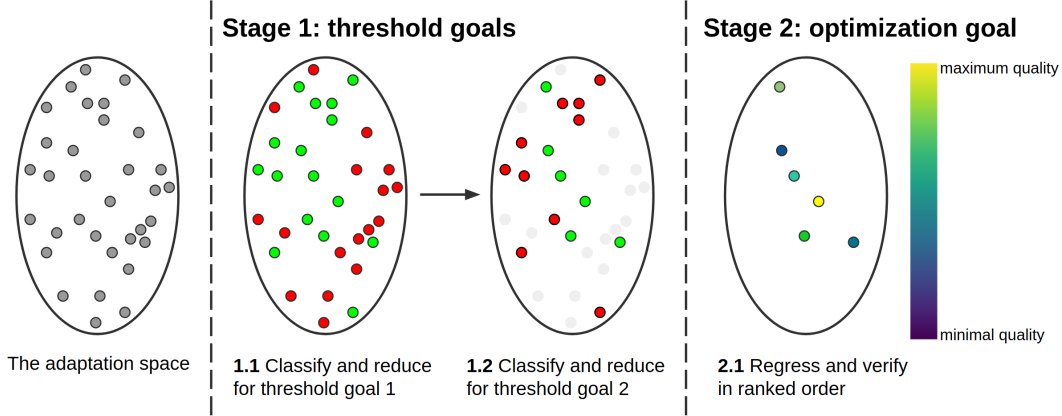


Figure 4.2: Illustration of both stages of DLASeR. Each dot represents an adaptation option. In the first stage, for each threshold goal the corresponding deep learning model predicts the relevant adaptation options sequentially (1.1 and 1.2): green dots satisfy the threshold goal, red dots do not satisfy the threshold goal. In the second stage, the relevant subset obtained from stage 1 is regressed by the deep neural network corresponding to the optimization goal, then the adaptation options are ranked (2.1).

each goal, we select a distinct deep learning model and a scaler that are maintained throughout the pipeline. In the offline part, the features are aggregated and the models with their accompanying scalers are fine-tuned. The second part defines how the models are maintained and utilized in an online setting. We use the DeltaIoT example to illustrate DLASeR and its learning pipeline.

4.4.1 Offline part of DLASeR pipeline

In the offline part of the pipeline, shown in Figure 4.3, we aggregate data of a series of adaptation cycles (via observation or simulation of the system). In order to successfully train a (deep) neural network, it is important that all relevant data is collected. We refer to this data as *features*.⁴ These features are a combination of both adaptation (or configuration) info and context/uncertainty info.

A feature vector for DeltaIoT contains: the distribution factor per link that determines the percentage of messages sent over the link and the setting of the transmission power for each link (the configuration parameters), the traffic load and the signal-to-noise ratio over the links (the uncertainties), and the current qualities and configuration of the system (the context).

The aggregated data allows us to perform back-testing of candidate learning solutions on the application and obtain initial insights in their performance. To that end, we have split the aggregated data in a train set and a validation set. It is

⁴A feature refers to measurable property or characteristic of the system that affects the effectiveness of the learning algorithms. These features should not be confused with features that define the adaptation space, such as for instance authentication or caching techniques as in [18].

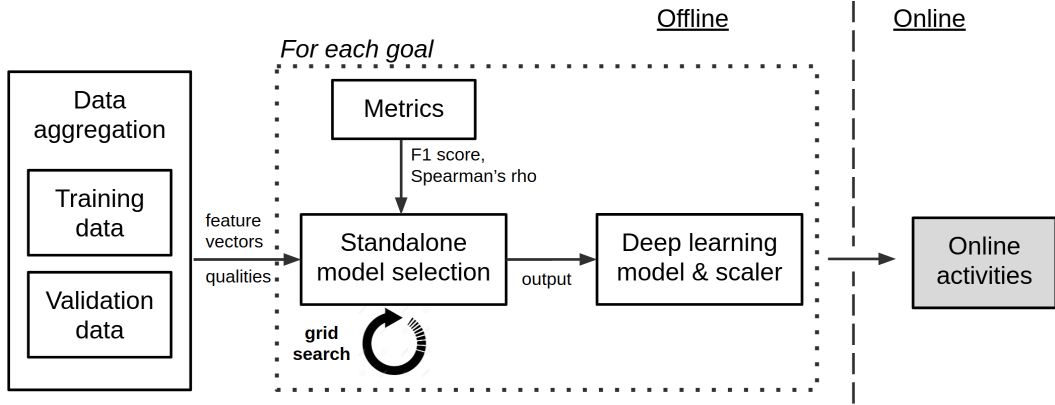


Figure 4.3: The offline part of the DLASeR learning pipeline. In the standalone model selection process the optimal hyperparameters are determined for all models; for each goal, i.e., threshold or optimization, there is a standalone deep learning model with an accompanying scaler. The output of this process is a completely configured deep learning model and a scaler for each adaptation goal.

important that both sets contain data of consecutive cycles without any overlap. If there would be an overlap, then we would have a data-leakage, making the results less representative. As illustration, if the train data contains all data from cycle 1 up to cycle n , then the validation data should contain all the remaining data starting from cycle $n + 1$.

The main activity in the offline part of the pipeline is *Standalone model selection*. Here, we want to indicate with standalone that the model selection process is performed separately for every adaptation goal. This activity aims at finding optimal hyperparameter settings for each deep neural network (and its scaler). During model selection, we applied a process called grid search [23]. This process uses back-testing to determine the optimal values of the hyperparameters. The hyperparameters encompass the number of layers of the deep learning model, the number of neurons, the batch size, the learning rate, the optimizer, and the scalar algorithm. Note that these hyperparameters are unique for each goal. Thus, for each goal we perform a model selection process.

Concretely, a model is trained and evaluated for every combination of hyperparameters. To that end, the data of the train set is used as input to learn the parameters of the deep learning models. The training process ends when the model is overfitting, i.e., the model learns the details and noise in the training data which reduces the performance of the model on new data. Once the models are trained, they are evaluated on the validation data. The effectiveness of models for threshold goals is measured by the F1 score, whereas we use Spearman's rho to determine the effectiveness of models for an optimization goal. Based on these two metrics, we select the best combination of hyperparameters for the corresponding goal.

Model selection results (for each adaptation goal) in a deep learning model and a scaler algorithm. We explain these now more in detail.

Scaler

During grid search, we determined the optimal scaler for the different deep learning models. The considered options for the scaler algorithm are standard scaling, min-max scaling, max-abs scaling, and no scaling.

Deep learning model

We started with studying classification deep neural networks. These networks are trained to predict whether an adaptation option meets a threshold goal. Then we studied regression deep neural networks. These networks are trained to predict a continuous value, based on which we infer a ranking for the adaptation options.

As we explained above, the hyperparameters determined in the model selection process during grid search (besides the scalar algorithm) are: the number of layers of the deep learning model (i.e., network depth), the number of neurons in each layer, batch size, learning rate, and the optimizer. The last three of these hyperparameters influence largely how the neural network adapts to changes. Ideally, we want to find a configuration that adapts not too slow nor too fast to changes. The first two of the above hyperparameters determine the complexity of the representations that can be learned by the neural network. Optimizing these two enables the deep neural network to construct meaningful representations and thus perform properly its goal.

When the deep learning models are fine-tuned and proper scalers are determined, these elements are integrated into a coherent solution; the DLASeR framework. The solution can then be deployed which brings us to the second part of the pipeline.

4.4.2 Online part of DLASeR pipeline

The online part, shown in Figure 4.4, consists of two cycles. In the training cycle, the deep learning models are initialized to the current state of the problem at hand. In the succeeding online cycles, the deep learning models performs adaptation space reduction and are updated by employing online learning. It is in this part of the pipeline that the deep learning models are deployed to aid decision-making in the self-adaptive system.

Training Cycle

In the *Training cycle*, the learning models are trained. The goal of this cycle is to properly initialize the learnable parameters of the models to the current setting of the problem at hand. To that end, relevant runtime data is collected to construct feature vectors. This data is then used to initialize the scaler (e.g., setting min/max values of parameters, the means, etc.) and the feature vectors are adjusted accordingly. The deep learning models (one for each goal) are then trained using the transformed features. The training cycle ends when the validation accuracy stagnates, i.e., when the difference between the predicted values with learning and the actual verification results are getting small. Since in this cycle the models are only initialized, there is no reduction of the adaptation space yet.

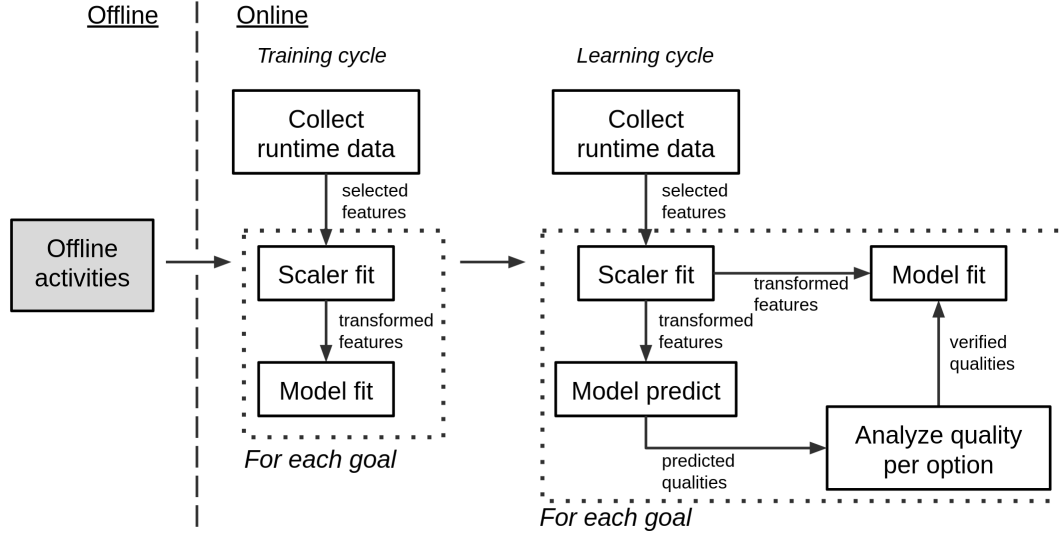


Figure 4.4: The online part of the DLASeR learning pipeline. In the training cycle, the deep learning models are trained to adapt to the current state of the system at hand. In the succeeding online cycles, adaptation space reduction is applied by means of the activities described in [section 4.3](#). The deep learning models are maintained in these online cycles by employing online learning.

Learning Cycle(s)

In the *Learning cycle*, the learning models are actively used to make predictions about the qualities of adaptation options to reduce the adaptation space (as explained in [section 4.3](#)). Furthermore, the analysis (verification) results are used to incrementally update the learning models. It is this interaction between the verifier and the DL models (by means of online learning) that enables adapting to changing environment and configurations.

Concretely, the features of the feature vectors are transformed similar as for the training cycle. The deep learning model then makes predictions for the feature vector of each adaptation option. The adaptation options are verified based on these predictions. In our work, we use statistical model checking [\[14, 2, 70, 73\]](#) at runtime to verify the adaptation options during analysis, however other analysis techniques can be applied.

Analyzing the adaptation options differs for the threshold and the optimization approach.

In the **classification stage** the adaptation space is reduced to the relevant subset according to the threshold goals (see [subsection 4.3.1](#)). To that end, the adaptation options that are predicted as relevant, i.e., compliant with the threshold goals, are selected. Algorithm 1 shows how the adaptation space is reduced in the classification stage.

If there is no optimization goal (thus no optimization stage), then these selected adaptation options are used for verification. In addition, we verify a fixed percentage

of the other (unselected) adaptation options, i.e., the *exploration rate*. This leads to a more representative distribution of the various adaptation options. As a final step, the verification results are used to further improve the learning model.

In the other case, i.e., there is an optimization goal (thus an optimization stage), then we perform no verification in this stage and continue on the reduced subset in the optimization stage. Note that if there are no threshold goals, we pass the complete set of adaptation options, i.e., the full adaptation space, to the optimization stage.

Algorithm 1 Stage 1: classification stage

```

1:  $pred\_subspace \leftarrow K.adaptation\_options$ 
2: for each  $DL\_thresh\_model$  in  $K.threshold\_models$  do
3:    $preds \leftarrow DL\_thresh\_model.predict(features)$ 
4:    $pred\_subspace \leftarrow pred\_subspace \cap preds$ 
5: end for
6: if  $K.optimization\_goal \neq \text{None}$  then
7:   Stop stage 1 and proceed to the optimization stage: Algorithm 2
8: end if
9:  $unselected \leftarrow K.adaptation\_options \setminus pred\_subspace$ 
10:  $explore \leftarrow unselected.randomSelect(exploration\_rate)$ 
11:  $Analyzer.verifyAdaptOpt(pred\_subspace \cup explore)$  ▷ Knowledge
12: Select valid adaptation option  $adapt\_opt$ , if none valid use fall back option
13:  $features, qualities \leftarrow K.verification\_results$ 
14: for each  $DL\_thresh\_model$  in  $K.threshold\_models$  do
15:    $DL\_thresh\_model.update(features, qualities)$ 
16: end for

```

In lines 1 to 5 the relevant subspace is predicted based on threshold goals. For each threshold goal, the corresponding deep learning model, stored in the Knowledge (K), predicts the relevant subspace sequentially. Note that the *predict()* function on line 3 first transforms the features by applying the scaler before making predictions. The intersection of the predicted subspaces that are relevant for each goal represents the relevant subspace. This process corresponds to the actual adaptation space reduction for all threshold goals.

In line 6 we check whether there is an optimization goal. If so, the classification stage ends there and verification is performed in the optimization stage to which the reduced subset is passed (see line 7). In the other case, when there is no optimization goal, lines 9 to 11 select the adaptation options for verification. We start from the relevant subspace and add a randomly selected set of adaptation options that are not in the predicted subspace based on the *exploration_rate*. Then, the combined set is sent to the verifier for analysis. The verification results, i.e., the predicted qualities of the verified adaptation options are stored in the knowledge. Line 12 selects a valid option to adapt the system, i.e., an adaptation option that satisfies all the threshold goals. If no valid adaptation option is found according to the goals, a fall-back option is used that implements a graceful degradation strategy.

In lines 13 to 16 the verification results are exploited to update the learning models for the threshold goals. First, we retrieve the features and the analysis results (i.e., the qualities for each threshold goal) of the verified adaptation options from the knowledge. Then, for each threshold goal, the corresponding model is updated. To that end, the relevant quality for the given model is selected, e.g. packet loss for the packet loss threshold model. This information is then used to update the parameters of the deep learning model using the same learning mechanism as we used in the training cycle.

The **optimization approach** continues on the adaptation options selected in the classification stage. In the case of absence of threshold goals, this stage starts from the complete adaptation space. In this stage a task of the planner (of MAPE-K) is performed; select the adaptation option (see subsection 4.3.2). Therefore, this stage leans more towards decision-making than adaptation space reduction. To perform this task, the regression neural network predicts the quality of the optimization goal for the retrieved subspace of adaptation options. Then, these options are ranked according to the optimization goal. Based on this ranking the adaptation options are verified until one is found that satisfies all other goals, i.e., the threshold goals. Algorithm 2 shows how the decision-making is made more efficient in the optimization stage.

This algorithm continues on the reduced adaptation space of Algorithm 1 (based on the threshold goals). In lines 2 to line 4 the predictions of the optimization deep learning model are used to rank the relevant adaptation options.

In lines 5 to 16 we iterate over the ranked adaptation options in the order of the predicted value for the quality of the optimization goal. In this iteration, we verify the chosen adaptation option and check whether it complies with the threshold goals. If this is the case we select this option for adaptation. If not, we continue verifying adaptation options until one is found that satisfies the threshold goals.

After iterating through the ranked adaptation (sub)space, line 17 selects a valid option to adapt the system, i.e., an adaptation option that satisfies all the threshold. If no valid adaptation option is found according to the goals, a fall-back option is used to ensure graceful degradation of the system. Then, lines 18 and 19 randomly select a sample of adaptation options from the options that were not verified based on the *exploration_rate*. In line 20, we verify these unexplored options. All the verification results are stored in the knowledge.

Finally, in lines 21 to 23 the deep learning models are updated, exploiting the verification results. This online learning should enable the models to copy with the dynamic behavior of the adaptation space.

Note that in the case where we only consider threshold goals, the adaptation space reduction occurs prior to verification. If we consider both threshold goals and an optimization goal, the adaptation space reduction is conducted by cleverly verifying adaptation options, i.e., in the order of their predicted ranking with respect to the optimization goal until an option is found that satisfies all the threshold goals.

Algorithm 2 Stage 2: optimization stage

```

1: Relevant subspace (pred_subspace) predicted in lines 1 to 5 from Algorithm 1
2: DL_opt_model  $\leftarrow$  K.optimization_model
3: ranking  $\leftarrow$  DL_opt_model.predict(features_pred_subspace)
4: ranked_subspace  $\leftarrow$  pred_subspace.sort(ranking)
5: valid_found  $\leftarrow$  False, idx  $\leftarrow$  0
6: verified_subspace  $\leftarrow$   $\emptyset$ 
7: while not valid_found do
8:   adapt_opt  $\leftarrow$  ranking[idx]
9:   Analyzer.verifyAdaptOpt(adapt_opt) ▷ Knowledge
10:  _, qualities  $\leftarrow$  K.verification_results[idx]
11:  if qualities meet all threshold goals then
12:    valid_found  $\leftarrow$  True
13:  end if
14:  verified_subspace.add(adapt_opt)
15:  idx  $\leftarrow$  idx + 1
16: end while
17: Select valid adaptation option adapt_opt, if none valid use fall back option
18: unselected  $\leftarrow$  K.adaptation_options \ verified_subspace
19: explore  $\leftarrow$  unselected.randomSelect(exploration_rate)
20: Analyzer.verifyAdaptOpt(explore) ▷ Knowledge
21: features, qualities  $\leftarrow$  K.verification_results
22: Update threshold models (line 14 to 16 from Algorithm 1)
23: DL_opt_model.update(features, qualities)

```

4.5 Conclusion

We conclude this chapter by indicating how the aspects, presented in the previous sections, are combined into the MAPE-K architecture.

4.5.1 Runtime integration with MAPE-K

Figure 4.5 shows how the deep neural network learners of DLASer are integrated in the MAPE-K architecture.

We focus here only on the analyzer, planner, and the relevant models of the knowledge. When the analyzer is triggered (1), it collects the possible adaptation options (and other relevant information) (2), i.e., all possible configurations of the managed system that are reachable through adaptation from the current configuration. Then the threshold deep learner determines the relevant set of adaptation options (3). To that end, the classification stage, as explained intuitively in subsection 4.3.1 and presented formally in Algorithm 1, is applied. The selected adaptation options are then written to the knowledge repository (4) and the planner is triggered (5). The planner invokes the optimization learner to deduce a ranking for the reduced adaptation space (6-7). The options are then verified one by one for the all the

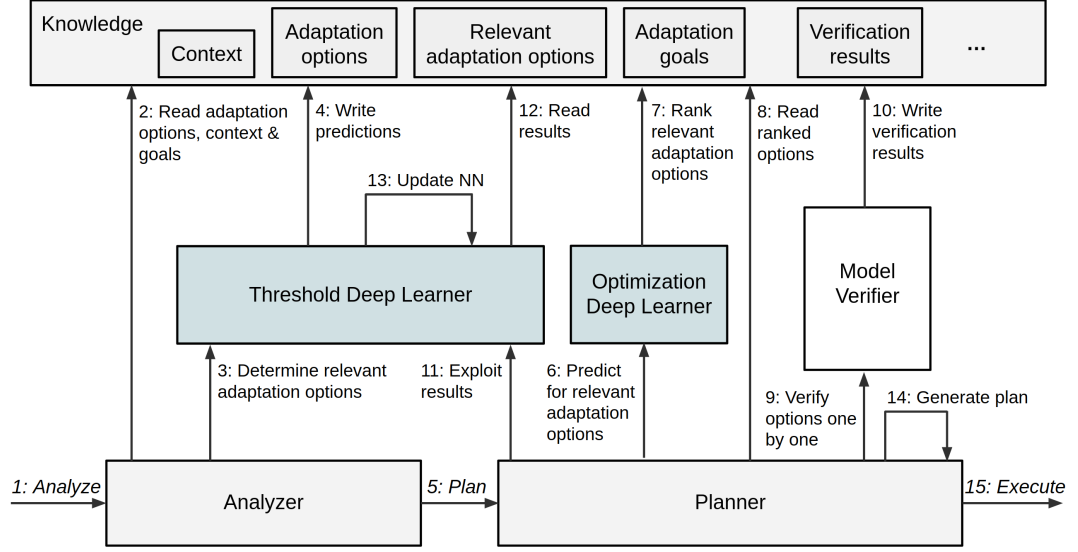


Figure 4.5: Runtime architecture of DLASeR. The threshold deep learner and optimization deep learner components together with their models are determined during the offline activities as shown in Figure 4.3. The runtime activities within the two learners follows the workflow of online activities shown in Figure 4.4.

goals in order of ranking (8–10). As soon as an option is found that complies with the threshold goals, the option will be selected for adaptation. Finally, in addition to the obtained verification results, we verify other unexplored options with an exploration sample. These verification results are exploited to update the deep learning models (11–13). After that, a plan is generated for the selected adaptation option and the executor is invoked to apply the adaptation (14–15).

Chapter 5

Deep Learning For Adaptation Space Reduction Plus

In this chapter we present the novel adaptation space reduction approach called “Deep Learning for Adaptation Space Reduction Plus” (DLASeR+). We outline how this approach enables effective and efficient decision-making for each type of adaptation goal, and how these adaptation goals are combined into the general and extendable DLASeR+ framework.

First, we present the architecture of DLASeR+. Then, we introduce the learning pipeline, defining how the configuration of the neural network architecture is determined, how it is initialized, and how it is maintained at runtime. Finally, we conclude by integrating DLASeR+ in the MAPE-K architecture.

5.1 DLASeR+ Architecture

In this section, we introduce the architecture of DLASeR+. We start with outlining how this approach deals with different types of adaptation goals, and how these adaptation goals are combined into a unified architecture. Finally, we zoom in on the neural network architecture of DLASeR+.

5.1.1 Adaptation space reduction for different adaptation goals

DLASeR+ can reduce the adaptation space for adaptation goals based on thresholds, set-points, and optimization criteria. We explain the specific learning technique that is applied to handle each type of goal. We will focus on two techniques; classification and regression.

Threshold goals

To deal with threshold goals, DLASeR+ relies on classification deep learning. Concretely we apply binary classification, using class 1 if an adaptation option meets the threshold goal and class 0 otherwise. We say that an adaptation option meets a threshold goal when the associated quality property of that adaptation option

is below (or above) the given threshold value g_t of that goal. Hence, DLASer+ reduces the adaptation space to the adaptation options that are valid, i.e., that are classified as 1. In case of multiple threshold goals, the adaptation space is reduced to the intersection of the subsets of adaptation options with predicted values that are classified as 1 for each of the different threshold goals.

Set-point goals

For set-points goals, DLASer+ also relies on classification deep learning. When a user defines a set-point goal, he has to specify a set-point value g_s and a margin ϵ . This margin defines the range $[g_s - \epsilon, g_s + \epsilon]$ in which adaptation options are considered valid.¹ Hence, DLASer+ again applies binary classification; class 1 for adaptation options that are predicted within the interval, and class 0 otherwise. In case of multiple set-point goals, the reduced adaptation space is the intersection of the subsets of adaptation options with predicted values that are classified as 1 for each of the different set-point goals.

Optimization goals

DLASer+ handles optimization goals by using regression deep learning. Based on the regressed values of a quality property, the adaptation options are ranked. From this ranking the adaptation option that maximizes or minimizes the adaptation goal can be derived. The adaptation space reduction is then determined by the number of adaptation options that need to be analyzed to make an adaptation decision. In the case of a single optimization goal (as we consider in this work), the adaptation space reduction is determined by the number of ranked adaptation options that need to be analyzed before one is found that complies with the other goals.

5.1.2 Unified DLASer+ architecture

The section above explains how DLASer+ handles the three types of adaptation goals separately. However, practical systems usually combine different types of adaptation goals. Over the years, different techniques have been developed to combine adaptation goals for the decision-making of self-adaptation. Classic examples are goals, utility functions, and rules [13, 66, 24]. In this paper, we present a unified DLASer+ architecture for a class of systems that combine multiple threshold and set-point goals with a single optimization goal. Our focus is on rule-based goals that are representative for a large number of practical self-adaptive systems.

Figure 5.1 shows the unified DLASer+ architecture. The deep neural network takes as input the initial adaptation space that consists of all the adaptation options, together with context data, such as the actual values of uncertainties in the environment and the current values of the relevant qualities. Central to the DLASer+ architecture is the use of a single deep neural network that covers all the adaptation

¹In control theory, this margin corresponds to the steady state error.

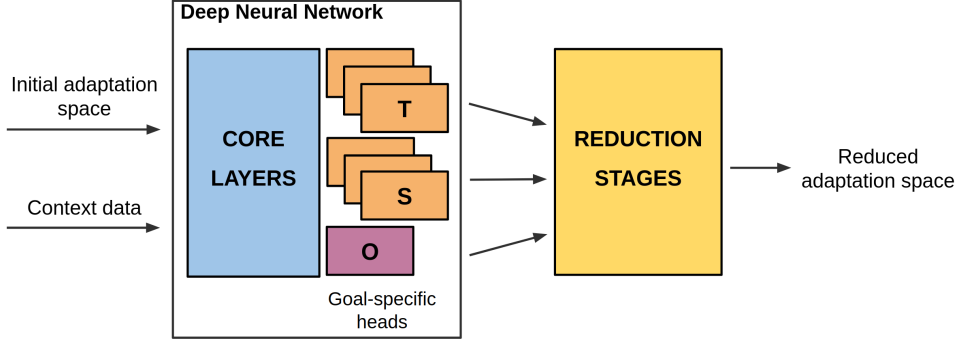


Figure 5.1: Unified architecture of DLASer+ for a mix of threshold and set-point goals and one optimization goal.

goals. Internally, the deep neural network consists of a number of core layers, complemented with goal specific heads. The core layers are shared among the different goals. Each head deals with a specific goal. The output produced by the deep neural network is then combined to produce the reduced adaptation space.

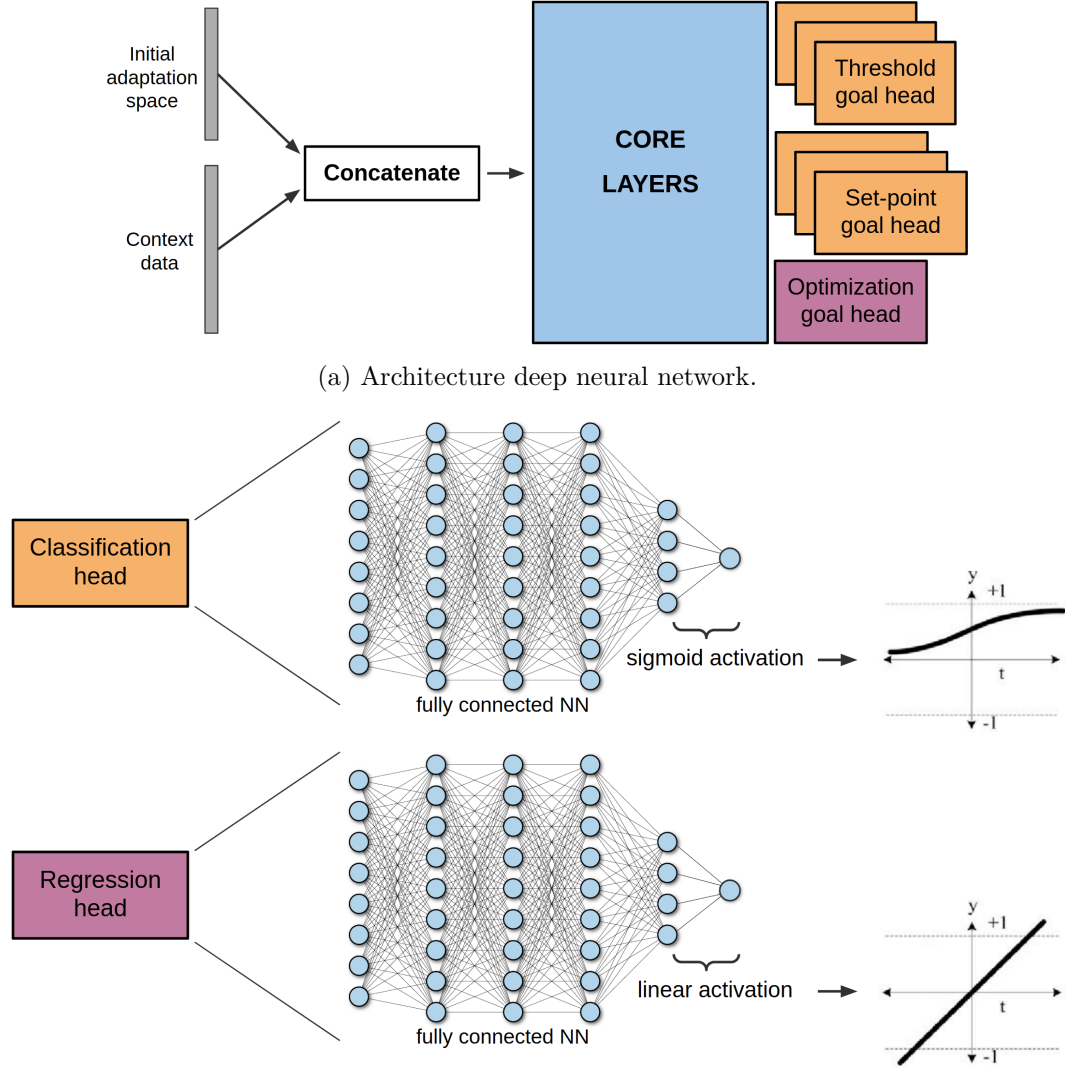
Neural network architecture

The core of the DLASer+ approach is a deep neural network that is kept up-to-date at runtime. In contrast with DLASer (see [section 4.2](#)), we utilize a single deep learning architecture for all the goals.

Figure 5.2 shows the internal architecture of the neural network. The figure at the top shows the structure of the network and the flow of the data through the network. The network starts with concatenating the input into feature vectors, one vector per adaptation option. A feature vector contains both the adaptation info and the context info.

The feature vectors are then fed to the core layers of the network. The core layers correspond to a fully connected neural network. Each fully connected layer is followed by a batch normalization layer [31]. The output of the last layer of these core layers can be considered as an encoding of the input data, since the output of this last core layer is passed to each of the goal-specific heads.

DLASer+ supports two kinds of goal-specific heads that can be added on top of the core layers: classification heads for threshold and set-point goals, and a regression head for an optimization goal. The figure at the bottom illustrates the difference between the two types. Both kinds of heads are fully connected neural networks, with a batch normalization layer after each fully connected layer. Each head produces a single output. However, the heads differ in the output they produce. The classification heads use a sigmoid activation function, producing values between 0 and 1. These values are classified based on predefined thresholds; for instance, all values below 0.5 are classified as class 0 and all values above (and including) 0.5 as class 1. The regression head uses a linear activation function which produces values that predict the quality property that needs to be optimized.



(b) Internal structure of the classification and regression heads in the DLASer+ architecture.

Figure 5.2: Overview of the deep neural network architecture of DLASer+. **Top:** the adaptation goal specific heads correspond to the ones displayed in the legend at the bottom. **Bottom:** legend of the two types of output heads. Both neural network heads have an output of dimensionality 1, with sigmoid and linear activation for respectively classification and regression.

Adaptation space reduction stages

By using a single deep neural network, the input data passes only once through the neural network to produce the output for the different adaptation goals. This output is then combined to reduce the adaptation in two stages. The first stage deals with adaptation goals that use classification deep learning, i.e., threshold and set-point goals. The first stage reduces the adaptation space to the adaptation options that are classified as being valid according to the threshold and set-point goals (similar to DLASer’s threshold stage, see [subsection 4.3.1](#)). We refer to this stage as the *classification stage*. The second stage deals with adaptation goals that use regression deep learning, i.e., the optimization goal. The second stage further reduces the adaptation space obtained from stage 1 by ranking the regressed values of the adaptations options (similar to DLASer’s optimization stage, see [subsection 4.3.2](#)). The adaptation options are ranked from low to high in the case of a minimization goal, and vice versa for a maximization objective. The ranked adaptation options can then be analyzed one by one until an option is found that satisfies (e.g., determined by verification) all the other goals. We refer to this stage as the *regression stage*.

It is important to note that either of these stages can be omitted when the corresponding type(s) of goals are not present for a problem at hand. For instance, for a system with only threshold goals (for which DLASer+ uses classification), the second stage can be omitted. In another example, suppose that there is only an optimization goal, the first stage is omitted and the second stage continues on the complete (i.e., nonreduced) adaptation space.

We explain now the two stages and illustrate the combination with a simple example shown in Figure 5.3. Suppose that the DLASer+ neural network predicted (once) for all the adaptation options. The illustration presents how the predictions are used to perform the adaptation space reduction.

In stage 1; the classification stage, all the adaptation options for all the threshold and set-point goals are classified relevant or not relevant. Then, the results are aggregated, reducing the initial adaptation space to the intersection of the adaption options that were classified relevant for all the aforementioned goals.

In stage 2; the regression stage, DLASer+ ranks the subset adaptation options obtained from stage 1. This ranking is based on the regressed quality for which the optimization goal was defined and depends on the optimization goal, i.e., ascending and descending order for respectively minimization and maximization. The adaptation options of the reduced adaptation space can then be analyzed in the order of the ranking until one is found that satisfies all the adaption goals.

5.2 The learning pipeline

We present now the DLASer+ learning pipeline that consists of an offline and online part. During the offline part, we fine-tune the deep neural network model, i.e, the core layers and for each type of goal its corresponding classification or regression heads. Next to this, we also select a scaler to transform the concatenated input

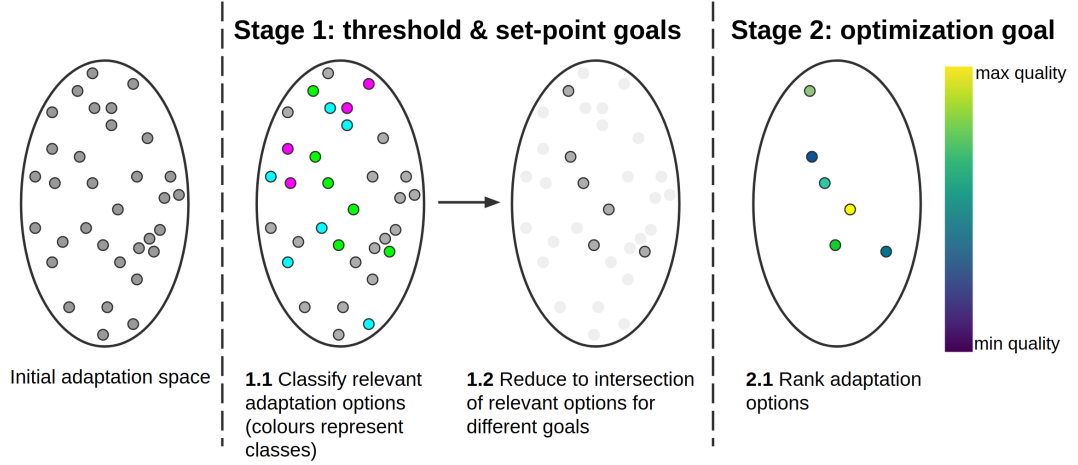


Figure 5.3: Illustration of the two stages of DLASer+. Each dot represents an adaptation option. In the first (classification) stage the initial adaptation options are classified for threshold and set-point goals (1.1): blue dots satisfy the threshold goals, pink dots satisfy the set-point goals, and green dots satisfy both kind of goals. Then the intersection of adaptation options that satisfy all threshold and set-point goals are kept, i.e., the green dots (1.2). In the second (regression) stage the relevant subset of adaptation options obtained in stage 1 are ranked (2.1).

features. During the online part, the deep learning model and the scaler are used by the running system to reduce adaptation spaces. Furthermore, in this online part, the model and the scaler are updated by using newly obtained data (from the verifier).

5.2.1 Offline part of the DLASer+ learning pipeline

During the offline part of the pipeline, shown in Figure 5.4, we collect data of a series of adaptation cycles (via observation or simulation of the system). This data comprises the adaptation options, context info, and also the associated qualities of the adaptation options. Once the data is collected, we concatenate the adaptation and context info in *feature vectors*, one per adaptation option.² The qualities associated with the adaptation options are used to validate the output of the generated goal-specific heads. As an example, the feature vector for DeltaIoT contains (1) data of adaptation options, i.e., the settings of the transmission power for each link and the distribution factors per link that determine the percentage of messages sent over the links, and (2) context data, i.e., the traffic load generated per mote and the signal-to-noise ratio over the links (uncertainties), and the current system configuration. The current qualities (one per adaptation goal) are for instance

²A feature refers to a measurable property or characteristic of the system that affects the effectiveness of the learning algorithms. These features should not be confused with features that define the adaptation space, such as for instance authentication or caching techniques as in [18].

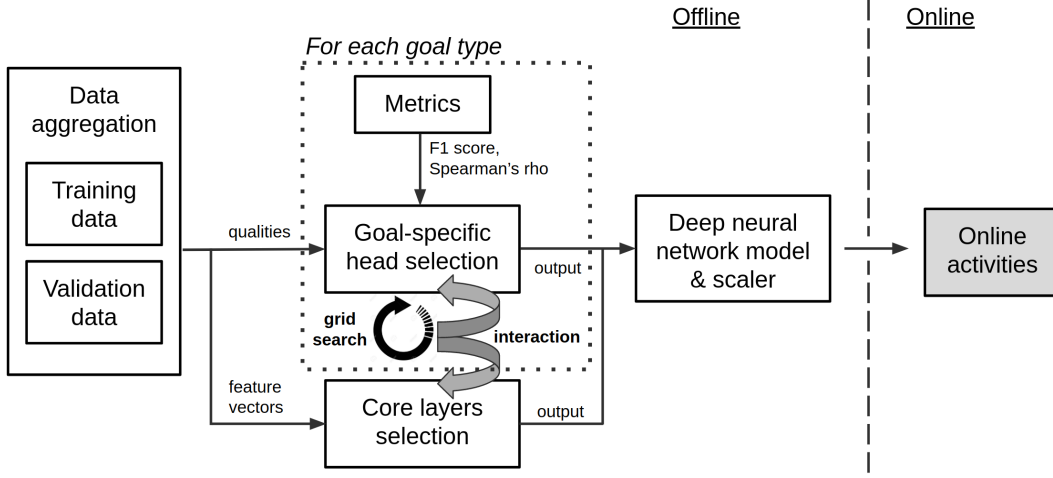


Figure 5.4: The offline part of the DLASer+ learning pipeline. During model selection the optimal hyper-parameters are determined for the shared core layers (*Core layers selection*) and for each adaptation type its goal-specific heads (*Goal-specific head selection*). The output of this process is a completely configured deep learning model (consisting of the core layers with on top the various goal-specific heads) and a scaler.

the packet loss and energy consumption of the adaptation options. To successfully train a deep neural network, it is important that all relevant data is collected.

The aggregated data is then used to perform a back-testing on the application to obtain insights in the performance of candidate learning solutions. To that end, the aggregated data is split in two sets: training data and validation data. It is important that both sets contain data of consecutive cycles without overlap. As illustration, if the training set contains all data from cycle 1 up to cycle n , then the validation set should contain all the remaining data starting from cycle $n + 1$. An overlap of the two sets would introduce data-leakage, which would jeopardize the validity of the results by introducing temporal correlations.

The main activity of the offline part of the pipeline is *model selection* for which we use back-testing, i.e., we utilize the aggregated data. Model selection aims at finding optimal hyperparameter settings. We make a distinction in the type of hyperparameters. On the one hand we have the hyperparameters of the shared core layers (this includes the scaler). On the other hand we have hyperparameters for each goal-specific head. In Figure 5.4 the former is tackled by *Core layers selection*, the latter is tackled by *Goal-specific head selection*. The hyperparameters that need to be optimized for the core layers and each of the goal-specific heads are the number of layers of the deep learning model and the number of neurons. The other hyperparameters, that are general for the full deep learning architecture, are the batch size, the learning rate, the optimizer, and the scaler algorithm.

To determine the optimal values of the hyper-parameters for the DLASer+ neural network model we applied grid search [23]. In contrast to DLASer where a grid

search process is needed for every adaptation goal, DLASer+ requires only one grid search for all the adaption goals. With grid search, the neural network model is trained (on the training data) and evaluated (on the validation data) for every combination of hyper-parameters. The training process ends when the model starts overfitting, i.e., the model starts learning details and noise in the training data to the extent that it negatively impacts the performance of the model on new data. Once the models are trained, they are evaluated on the validation data set, i.e., checking the quality of the predictions it made for the validation data. The effectiveness of models is measured by the F1 score for the classification heads and Spearman’s rho for the regression head. Since we have no direct means to combine these two metrics, the best model is selected based on the lowest validation loss. The loss function is the sum of the losses for each head, and thus captures the quality of the classification and regression heads. In this sum the regression head and the classifications head have equal importance.

The output of this model selection process is a completely configured deep learning model (consisting of the core layers with on top the various goal-specific heads) and a scaler. We explain these now more in detail.

Scaler

During grid search we determine the optimal scaler for the deep learning model. The considered options for the scaler algorithm are standard scaling, min-max scaling, max-abs scaling, and no scaling.

Core layers

As we explained above, the hyperparameters determined during grid search in the *Core layers selection* process (besides the scalar algorithm) are: the number of core layers, the number of neurons in each layer, the batch size, the learning rate, and the optimizer. The last three of these hyperparameters influence largely how the neural network adapts to changes. Ideally, we want to find a configuration that adapts not too slow nor too fast to changes. The first two of the above hyperparameters determine the *encoding layer*, i.e., the last layer of the core layers. The output of this encoding layer is passed to each of the goal-specific heads. This encoding layer can be considered as an encoded representation of the input; the adaptation info, the system (with its uncertainties), and the environment. Optimizing the first two hyperparameters enables the core layers to construct meaningful representations of the input data.

Goal-specific heads

For each of the two possible heads (i.e., classification or regression) we determine the following hyperparameters: the number of layers of the head, and the number of neurons in each layer. Optimizing these hyperparameters in the *Goal-specific head selection* process enables the goal-specific heads to predict their goal properly.

When the core layers and goal-specific heads are fine-tuned and a proper scaler is found, the integrated solution can be deployed, which brings us to the second part of the learning pipeline.

5.2.2 Online part of DLASer+ learning pipeline

During the online part of the learning pipeline, the deep neural network supports the decision-making process of a self-adaptive system. The online part, shown in Figure 5.5, consists of two cycles. In the training cycle the deep learning model is initialized based on the current state of the system. In the learning cycle the DL model performs adaptation space reduction and is updated by using online learning.

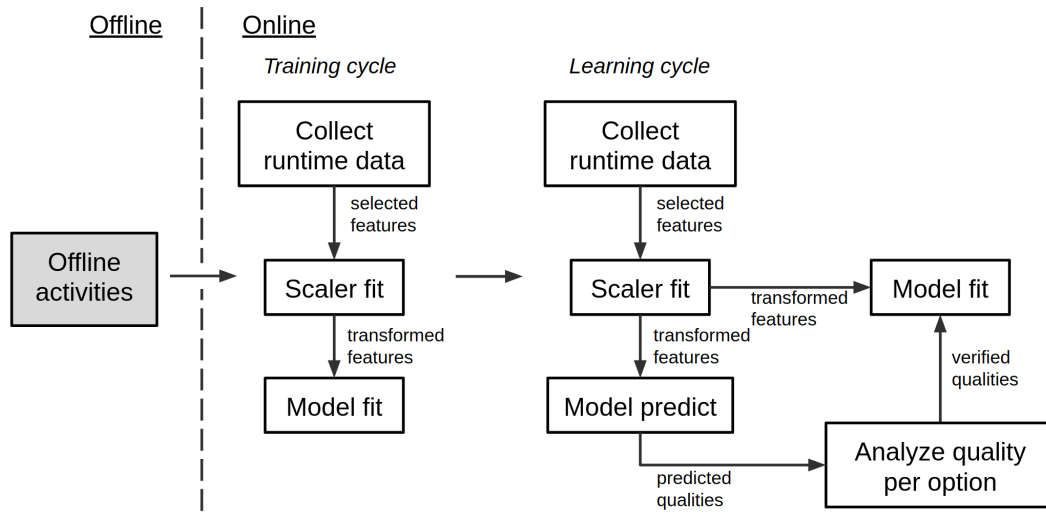


Figure 5.5: The online part of the DLASer+ learning pipeline. In the training cycle, the deep neural network model is initialized for the problem at hand. In the succeeding online cycle, adaptation space reduction is applied by means of the activities described in subsection 5.1.2. Each online learning cycle also maintains the deep learning model by employing online learning.

Training cycle

The goal of the training cycle is to initialize the learnable parameters of the model for the problem at hand. To that end, relevant runtime data is collected to construct feature vectors. This data is then used to update the scaler (e.g., updating min/max values of parameters, the means, etc.) adjusting the feature vectors accordingly. The deep neural network model is then trained using the transformed features. The training stage ends when the validation accuracy stagnates, i.e., when the difference between the predicted values and the actual verification results are getting small. In this cycle the maximum available time is used for analyzing adaptation options to generate data in order to initialize the deep neural network model; hence there is no reduction of the adaptation space yet.

Learning cycle(s)

In the learning cycle, the deep neural network model is actively used to make predictions about the adaptation options for the various adaptation goals aiming to reduce the adaptation space. Furthermore, the results of the analysis of the reduced adaptation space are then used to incrementally update the learning model.

Concretely, a feature vector is composed for each adaptation option that is scaled using the scaler. The deep neural network model then makes predictions for the feature vectors. Based on these predictions, the adaptation space is reduced and the adaptation options of the reduced adaptation space are then analyzed. In the realisation of DLASer+, we use runtime statistical model checking [14, 2, 70, 73] to analyze the adaptation options, however other analysis techniques can be applied. The interaction of the deep learning model with the verifier is two-way. On the one hand the learner aids in selecting the adaptation options for verification. On the other hand, the learner uses the verification results to incrementally update itself.

The analysis of the reduced adaptation space depends on the types of adaption goals at hand. For a self-adaptive system with a mix of threshold and set-point goals and an optimization goal, analysis consists of two stages: the classification stage and the regression stage, see [section 5.1.2](#). For systems without optimization goal, only the classification stage applies. Similarly, for systems with only an optimization goal, only the regression stage applies.

In the **classification stage** the adaptation space is reduced to the relevant subset of adaptation options that comply with the threshold and set-point goals. [Algorithm 3](#) shows how the adaptation space reduction is applied and how analysis is performed in the classification stage.

In [lines 1 and 2](#), we initialize the variable *pred_subspace* to the complete adaptation space and store the predicted output of the deep neural network in the *DNN_output* variable. Both these variables are used in [Algorithm 3](#) and [Algorithm 4](#). *K.DNN_model* refers to the deep neural network model that is stored in the Knowledge module (*K*). Since DLASer+ uses a single deep learning model for all the goals, prediction can be done in a single step. Note that the *predict()* function in [line 2](#) scales the elements of the feature vectors before making predictions. [Lines 3 to 5](#) check whether there are any threshold or set-point goals. If there are no threshold nor set-point goals, we proceed to the regression stage, i.e., [Algorithm 4](#). In the other case, i.e., there are threshold and/or optimization goals, the adaptation space (*pred_subspace*) is reduced in [lines 6 to 8](#). Concretely, we reduce the adaptation space to the intersection of the predicted subspaces that are relevant for each of these two types of classification goals. This step represents the actual adaptation space reduction for all threshold and set-point goals. The adaption option is analyzed to check whether it complies with the threshold and set-point goals. If this is the case this option is selected for adaptation. If not, analysis is continued until an adaptation option is found that satisfies the threshold and set-point goals.

[Line 9](#) checks whether there is an optimization goal. If there is such a goal, the classification stage ends and the system continues the regression stage using the reduced subset, i.e., *pred_subspace* (see [line 10](#)). If there is no optimization goal,

Algorithm 3 Stage 1: classification stage

```

1:  $pred\_subspace \leftarrow K.adaptation\_options$ 
2:  $DNN\_output \leftarrow K.DNN\_model.predict(feature\_vectors)$ 
3: if  $K.threshold\_goals == \text{None}$  &  $K.set\_point\_goals == \text{None}$  then
4:   Stop stage 1 and proceed to the regression stage: Algorithm 4
5: end if
6: for each  $class\_goal \leftarrow K.classification\_goals$  do
7:    $pred\_subspace \leftarrow pred\_subspace \cap DNN\_output.class\_goal$ 
8: end for
9: if  $K.optimization\_goal \neq \text{None}$  then
10:  Stop stage 1 and proceed to the regression stage: Algorithm 4
11: end if
12:  $pred\_subspace.shuffle()$ 
13:  $valid\_found \leftarrow \text{False}$ ,  $idx \leftarrow 0$ 
14:  $verified\_subspace \leftarrow \emptyset$ 
15: while not  $valid\_found$  &  $idx < pred\_subspace.size$  do
16:    $adapt\_opt \leftarrow pred\_subspace[idx]$ 
17:    $Analyzer.analyzeAdaptationOptions(adapt\_opt)$  ▷ Knowledge
18:    $\_, qualities \leftarrow K.verification\_results[idx]$ 
19:   if  $qualities$  meet all threshold and set-point goals then
20:      $valid\_found \leftarrow \text{True}$ 
21:   end if
22:    $verified\_subspace.add(adapt\_opt)$ 
23:    $idx \leftarrow idx + 1$ 
24: end while
25: Select valid adaptation option  $adapt\_opt$ , if none valid use fall back option
26:  $unselected \leftarrow K.adaptation\_options \setminus pred\_subspace$ 
27:  $explore \leftarrow unselected.randomSelect(exploration\_rate)$ 
28:  $Analyzer.analyzeAdaptationOptions(explore)$  ▷ Knowledge
29:  $features, qualities \leftarrow K.analysis\_results$ 
30:  $K.DNN\_model.update(features, qualities)$ 

```

the adaptation options for analysis are selected in lines 12 to 24. We first shuffle the relevant subspace (see line 12) and then iterate over the adaptation options in this newly obtained order (see lines 15 to 24). In this iteration, an adaptation option is analyzed to check whether it complies with the threshold and set-point goals. If this is the case this option is selected for adaptation and the iteration halts. If not, analysis is continued until an adaptation option is found that satisfies the threshold and set-point goals.

After iterating through the shuffled adaptation (sub)space, line 25 selects a valid option to adapt the system, i.e., an adaptation option that satisfies all the threshold and set-point goals. If no valid adaptation option is found according to the goals, a fall-back option is used that implements a graceful degradation strategy. Then, lines 26 and 27 use the *exploration_rate* to select a random sample from the adaptation options that were not analyzed. These options are then also analyzed in line 28 and the analysis results are stored in the knowledge.

Finally, in lines 29 and 30 the analysis results are exploited to update the deep neural network model. To that end, the features and the analysis results (i.e., the qualities per goal obtained by verification) of the analyzed adaptation options are retrieved from the knowledge. Based on this data, the neural network model is updated using the same learning mechanism as used in the training cycle.

The **regression stage** starts either from the adaptation options selected in the classification stage or from the complete adaptation space in case there is only an optimization goal. The set of adaptation options are then ranked according to the predicted quality of the optimization goal. Algorithm 4 shows how the adaptation options are ranked and how one of the options is selected based on the result of the analysis and its compliance with the threshold and set-point goals.

In lines 2 to 4, the predictions of the previous stage are reused to obtain a ranking of the relevant adaptation options. Since we have a single deep neural network model, we require only a single prediction for all the features (see algorithm 3, line 2).

Lines 5 to 16 iterate over the ranked adaptation options in descending order of the predicted value for the quality of the maximization goal (the opposite order is used for a minimization goal). The adaptation option is analyzed to check whether it complies with the threshold and set-point goals. If this is the case, this option is selected for adaptation. If not, analysis is continued until an adaptation option is found that satisfies the threshold and set-point goals.

After iterating through the ranked adaptation (sub)space, line 17 selects a valid option to adapt the system, i.e., an adaptation option that satisfies all the threshold and set-point goals. If no valid adaptation option is found according to the goals, a fall-back option is used to ensure graceful degradation of the system. Then, lines 18 and 19 use the *exploration_rate* to select a random sample of adaptation options from the options that were not analyzed. These options are then also analyzed in line 20 and the analysis results are stored in the knowledge.

Finally, lines 21 and 22 exploit the analysis results to update the deep neural network model. This enables the model to cope with the dynamic behavior of the self-adaptive system.

Algorithm 4 Stage 2: regression stage

```

1: pred_subspace retrieved from Algorithm 3
2: opt_goal  $\leftarrow$  K.optimization_goal
3: ranking  $\leftarrow$  DNN_output.opt_goal  $\triangleright$  Use prediction of Algorithm 3, line 2
4: ranked_subspace  $\leftarrow$  pred_subspace.sort(ranking)
5: valid_found  $\leftarrow$  False, idx  $\leftarrow$  0
6: verified_subspace  $\leftarrow$   $\emptyset$ 
7: while not valid_found & idx < ranked_subspace.size do
8:   adapt_opt  $\leftarrow$  ranked_subspace[idx]
9:   Analyzer.analyzeAdaptationOptions(adapt_opt)  $\triangleright$  Knowledge
10:  _, qualities  $\leftarrow$  K.verification_results[idx]
11:  if qualities meet all threshold and set-point goals then
12:    valid_found  $\leftarrow$  True
13:  end if
14:  verified_subspace.add(adapt_opt)
15:  idx  $\leftarrow$  idx + 1
16: end while
17: Select valid adaptation option adapt_opt, if none valid use fall back option
18: unselected  $\leftarrow$  K.adaptation_options \setminus verified_subspace
19: explore  $\leftarrow$  unselected.randomSelect(exploration_rate)
20: Analyzer.analyzeAdaptationOptions(explore)  $\triangleright$  Knowledge
21: features, qualities  $\leftarrow$  K.verification_results
22: K.DNN_model.update(features, qualities)

```

Note that in the case where we only consider threshold and/or set-point goals, the adaptation space reduction occurs prior to analysis. In case there is also an optimization goal, the adaptation space reduction is conducted by cleverly and efficiently analyzing adaptation options, i.e., in the order of their predicted ranking with respect to the optimization goal until an option is found that satisfies all the threshold and/or set-point goals.

5.3 Conclusion

We conclude this chapter by indicating how the DLASER+ approach is integrated into the MAPE-K architecture. To that end, we combine the aspects, presented in the previous sections.

5.3.1 Runtime integration with MAPE-K

Figure 5.6 illustrates how DLASER+ is integrated with a MAPE-K feedback loop.

The figure zooms in on the analyzer and planner components of the MAPE-K architecture, together with relevant models of the knowledge. When the monitor finishes an update of the knowledge with new runtime data, it triggers the analyzer (1). The analyzer reads the data that is necessary for the deep neural network

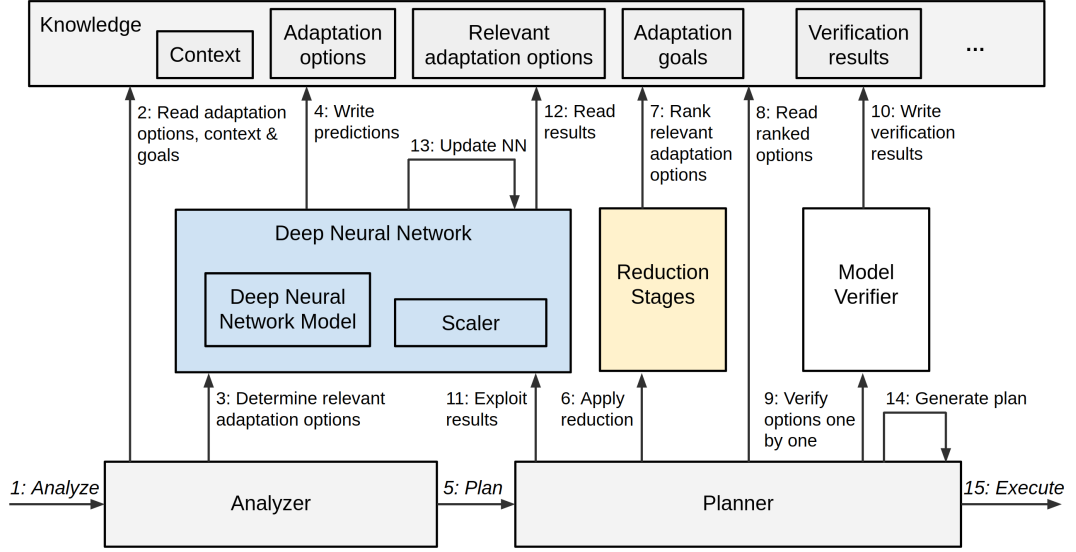


Figure 5.6: Runtime integration of DLASER+ with MAPE-K.

to determine the relevant adaptation options (2). The deep neural network then produces predictions that are written to the knowledge (3-4). Next, the analyzer triggers the planner (5). The planner then initiates adaptation space reduction; the ranked adaptation options are then written to the knowledge (6-7). Next, the planner reads the ranked options and invokes the model verifier to analyze the options one by one and write the results to the knowledge (8-10). When a suitable adaptation is found (after verifying some exploration samples), the planner invokes the deep neural network to read the verification results and update the network accordingly (11-13). Next, the planner generates a plan for the adaptation option that is selected to adapt the system (14). Finally, the planner triggers the executor to enact the actions of the plan on the managed system (15).

Chapter 6

Evaluation

In this chapter we discuss the results of evaluating DLASeR and DLASeR+ on the two DeltaIoT instances. To that end, we first describe the evaluation setup and specify the various combination of goals that are evaluated. Then, we present the evaluation results for DLASeR, indicating whether deep learning is applicable for adaptation space reduction. Next, we discuss the evaluation results for DLASeR+, aiming to answer the different aspects of the research question. Finally, we elaborate on the threats to validity.

6.1 Evaluation setup

In this section we first describe the concrete DeltaIoT setup that is used for evaluation. Then, we present the combination of goals that will be evaluated.

6.1.1 DeltaIoT setup

We evaluate DLASeR and DLASeR+ on two instances of the DeltaIoT artifact (as described in [section 2.4](#)). For both instances we used the same type of network settings. The stochastic uncertainty profiles for the traffic load generated by the motes ranged from 0 to 10 messages per mote per cycle, while the network interference along the links varied between -40 dB and +15 dB. The MAPE-K feedback loop was designed using a network of timed automata. These models were executed by using the ActivFORMS execution engine [29, 73, 30]. We applied statistical model checking at runtime using Uppaal-SMC for the verification of adaptation options [14]. The exploration rate was set to 5%.

For both instances of DeltaIoT, we considered 275 online adaptation cycles, corresponding with a wall-clock time of 77 hours. We used a single cycle to train the network parameters. The remaining 274 cycles are evaluated as learning cycles.

The approach proposed by Quin F. et al. [53] is used as a benchmark for the threshold goals. To evaluate the quality of the adaptation decisions made by the learning approach, we benchmark the results with a *reference approach* that analyses

the whole adaptation space (without learning). This reference approach always applies the optimal adaptation action.

For the learning solutions, we used the implementations of scalers from *scikit-learn* [51] and neural networks from *Keras* and *Tensorflow* [1]. We run the simulated self-adaptive system on an i7-3770, 3.40GHz, 12GB RAM; the deep learning models are trained (and maintained) on a NVIDIA Tesla T4 GPU with 12 GB RAM.

6.1.2 Goal configurations

Table 6.1 summarizes the combinations of adaptation goals we evaluated for both instances of DeltaIoT: *TTO* (2 Threshold goals and 1 Optimization goal), *TTS* (2 Threshold goals and 1 Set-point goal), and *TSO* (1 Threshold, 1 Set-point, and 1 Optimization goal). For DLASeR we evaluated the *TTO* setting, whereas for DLASeR+ all three configurations are evaluated, i.e., *TTO*, *TTS*, and *TSO*.¹

Table 6.1: Overview of the different evaluation settings for DLASeR and DLASeR+. The considered qualities are packet loss (PL), latency (LA), and energy consumption (EC). Each configuration is evaluated on DeltaIoTv1 and DeltaIoTv2.

Reference	Goals DeltaIoTv1	Goals DeltaIoTv2
TTO	T1: $PL < 10 \%$ T2: $LA < 5 \%$ O1: minimize EC	T1: $PL < 10 \%$ T2: $LA < 5 \%$ O1: minimize EC
TTS	T1: $PL < 10 \%$ T2: $LA < 5 \%$ S1: $EC \in [12.9 \pm 0.1]$	T1: $PL < 10 \%$ T2: $LA < 5 \%$ S1: $EC \in [67 \pm 0.3]$
TSO	T1: $LA < 5 \%$ S1: $EC \in [12.9 \pm 0.1]$ O1: minimize PL	T1: $LA < 5 \%$ S1: $EC \in [67 \pm 0.3]$ O1: minimize PL

6.2 DLASeR

For DLASeR we evaluated the *TTO* setting on both instances of DeltaIoT. We first present the findings of the offline stage of the DLASeR pipeline. Then, we present the results in two steps. First, we consider only the two threshold goals for adaptation space reduction, enabling us to compare with the work of Quin F. et al. [53]. Second, we evaluate DLASeR for the complete *TTO* configuration and compare the qualities with a reference approach. Finally, we discuss the extent to which DLASeR indicates the applicability of deep learning for effective and efficient adaptation space reduction.

¹Configuration and setting will be used interchangeably in this text.

6.2.1 Offline settings

As explained in [subsection 4.4.1](#), we used grid search for selecting models for the learners. Table 6.2 shows the best parameters of the pipeline obtained by evaluating the approaches on 30 sequential cycles. Each row in the table corresponds to a single grid search process. In total, 6 grid search processes were completed; 3 (one for each of the three goals) $\times$ 2 (the two instances of DeltaIoT).

Table 6.2: Best grid search results for DLASeR on the TTO configuration.

Problem	Goal	Hyper parameters					Objective
	Threshold	Scaler	Batch size	LR	Optimizer	Hidden layers	F1 score
DeltaIoTv1	T1	Standard	128	2e-3	Adam	[20, 10, 5]	83.18%
	T2	MaxAbs	64	5e-4	Adam	[20, 10, 5]	93.96%
DeltaIoTv2	T1	Standard	2056	5e-3	Adam	[50, 25, 10, 5]	75.81%
	T2	Standard	1028	2e-3	Nadam	[50, 25, 10, 5]	96.05%
	Optimization	Scaler	Batch size	LR	Optimizer	Hidden layers	Spearman's rho
DeltaIoTv1	O1	Standard	32	5e-3	RMSprop	[20, 10, 5]	87.63%
DeltaIoTv2	O1	MaxAbs	1028	5e-4	Nadam	[50, 25, 10, 5]	26.04%

For the threshold goals, we note that the models can predict T2 (latency) significantly better than T1 (packet loss). We speculate that this is due to the latency goal being less restrictive than the packet loss goal. On average the percentage of adaptation options that meet the latency goal lies around 90% for DeltaIoTv1 and 80% for DeltaIoTv2. Whereas the packet loss goal is satisfied respectively by ca. 40% and 20% of the adaptation options. On the other hand, when scaling up to the larger instance, we observe (in absolute numbers) that the F1 score of the packet loss degrades with 10.37%, whereas the F1 score of the latency improves with 2.09%.

For the optimization goal, we notice a large decay in Spearman correlation that is reduced from 87.63% for DeltaIoTv1 to 26.04% for DeltaIoTv2. At first sight, this might seem problematic. But, further analysis will show that this difference does not lead to substantially worse results.

Based on the obtained configurations, we deploy the models in the simulator to perform adaptation space reduction.

6.2.2 Results

With DLASeR, the goal is to indicate whether deep learning is applicable for adaptation space reduction.² We first evaluate the size of the adaptation space reduction. To that end, we evaluate the TTO adaptation space reduction, but also the reduction for just the two threshold goals (T1 & T2). The latter enables us to compare with related work, where only threshold goals were considered. Once we outlined the adaptation space reduction, we focus on the quality of the adaptation

²The evaluation results for DLASeR are limited. For DLASeR+ we will answer the research goal and elaborate in detail on each aspect of the research question.

decisions. Again, we will evaluate the TTO adaptation space reduction and also the reduction for just the two threshold goals (T1 & T2).

Adaptation space reduction

Table 6.3 presents the results obtained with DLASeR. For the threshold goals (T1 & T2) we obtain a total reduction of the adaptation space of 64.42% for DeltaIoTv1 and 92.94% for DeltaIoTv2. The F1 scores are excellent, but, decrease from 85.78% to 73.06% when scaling up to the larger instance. DLASeR achieves a good reduction of the adaptation space for the small instance of DeltaIoT (64.42%) and an excellent result for the large instance (92.94%).

Table 6.3: Results for DLASeR and comparison with Quin F. et al. [53]. Table A.1 in the appendix presents the detailed results for the DLASeR’s reduction of the adaptation space on TTO configuration.

Problem	Approach	Goals	F1 score	Total reduction
DeltaIoTv1	DLASeR	T1 & T2	85.78%	64.42%
	DLASeR	T1 & T2 & O1	/	94.81%
	Quin F. et al.	T1 & T2	85.85%	50%
DeltaIoTv2	DLASeR	T1 & T2	73.06%	92.94%
	DLASeR	T1 & T2 & O1	/	98.53%
	Quin F. et al.	T1 & T2	73.01%	75.5%

We compare these results with the work of Quin F. et al. [53], where classification was used for adaptation space reduction on an identical setup with similar requirements, i.e., T1 & T2. The results show that both approaches achieve very similar results for F1 score. However, there is a significant difference for the total reduction of the adaptation space: for DeltaIoTv1 we have 64.42% for DLASeR versus 50.00% for Quin et al.; for DeltaIoTv2 we have 92.94% for DLASeR versus 75.50% for Quin et al. At first sight the discrepancy in the total reduction obtained by DLASeR versus Quin F. et al. seems weird when we consider that the F1 scores are nearly identical. But, given that we used the same simulator (with the same uncertainty profiles), it seems plausible to explain this large difference as a consequence of a more restrictive adaptation space drift.

The results for the TTO setting (threshold goals T1 & T2, and optimization goal O1) demonstrate the effectiveness of DLASeR. With a total reduction of 94.81% for DeltaIoTv1 and 98.53% for DeltaIoTv2. For DeltaIoTv2, DLASeR comes close to the optimum of what can be achieved in terms of (total) adaptation space reduction.

Figure 6.1 visualizes the predicted adaptation space reduction based on T1 & T2 for DeltaIoTv1 at one particular point in time. The red lines in the figure spans the region with relevant adaptation options according to the threshold goals.

In Figure 6.2 we show for DeltaIoTv2 how DLASeR adapts its predictions for the threshold goals over two sequential cycles. We can clearly observe the dynamic

behavior of the adaptation space, i.e., the shift in qualities of the adaptation options over sequential adaptation cycles. It is undoubtedly visible that DLASeR handles properly, by means of online learning, the dynamics of the adaptation space.

Figure 6.3 visualizes the ranked adaptation space based on O1 for DeltaIoTv1 at some particular point in time. The gray box its (vertical) surfaces indicate the region with relevant adaptation options according to the two threshold goals (T1 & T2). The color of the dots correspond to the ranking based on predicted energy consumption.

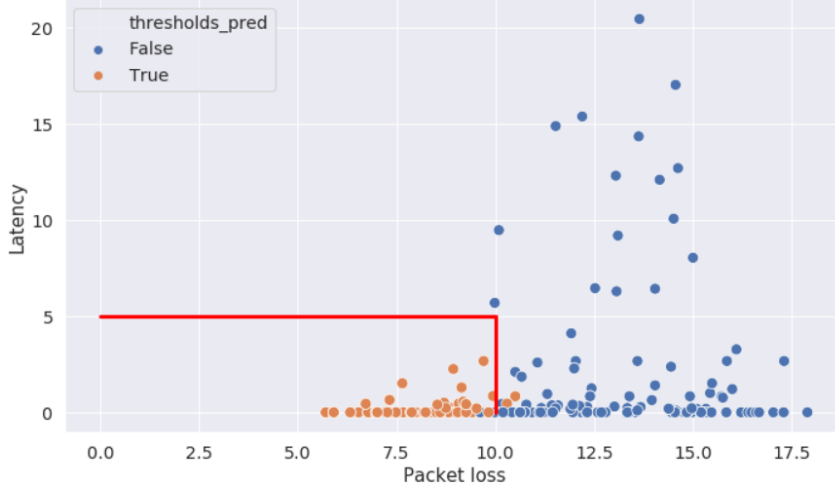


Figure 6.1: The predicted adaptation space reduction by DLASeR for some cycle in DeltaIoTv1. The orange and blue dots represent respectively the adaption options that are predicted to meet and not meet the threshold goals T1 & T2 (represented by the two red lines) of the TTO configuration.

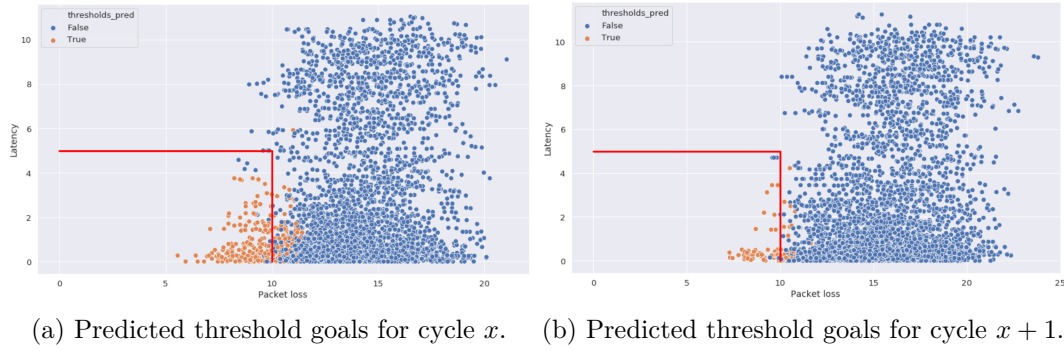


Figure 6.2: The predicted adaptation space reduction by DLASeR for two sequential adaptation cycles in DeltaIoTv2. The orange and blue dots represent respectively the adaption options that are predicted to meet and not meet the threshold goals T1 & T2 of the TTO configuration. **Left:** the threshold predictions of the adaptation space for some cycle x . **Right:** the threshold predictions of the adaptation space for the succeeding adaptation cycle, i.e., cycle $x + 1$.

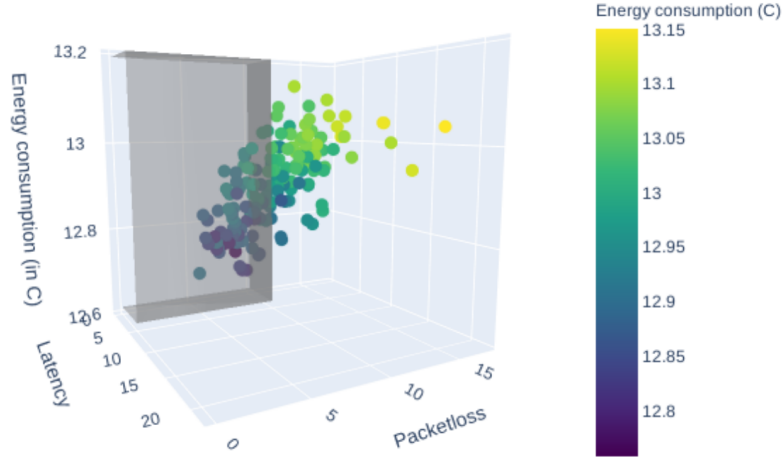


Figure 6.3: The ranking of the adaptation space by DLASeR for some cycle in DeltaIoTv1. The dots represent the adaptation options, their color corresponds to the predicted quality of the optimization goal O1 (minimize energy consumption) of the TTO configuration. Figure A.1 in the appendix is a 2D version of this figure.

Quality of the adaptation decisions

While the F1 score and total reduction give us insight into the reduction of the adaptation space, these metrics do not tell us anything about the quality of the adaptation decisions (made with reduced adaptation spaces). We measure this and benchmark DLASeR with a reference approach that applies exhaustive verification (which is the ideal case, but practically not always possible due to time constraints).³

Figure 6.4 shows the qualities of the adaptation decisions for only the threshold goals (T1 & T2). The adaptation decision is guided by the optimization goal O1 (minimize energy consumption). To that end, Quin F. et al. and DLASeR selects the adaptation option (in the reduced space) that has the best verified quality according to the optimization goal.⁴ The results show no visible difference between the qualities for DLASeR and the reference approach. For DeltaIoTv1, the p-values for the different qualities are between 0.15 and 0.93 (Wilcoxon signed-rank test). In the case of DeltaIoTv2, the tests indicate that there is a small difference for all qualities (all p-values lower than 0.014). For example, the energy consumption for DLASeR is 0.07% higher, but this is from a practical point of view negligible. The results are also in line with the results of Quin F. et al. [53].

Figure 6.5 displays the qualities of the adaptation decisions for the two threshold goals and the optimization goal (thus the complete TTO setting; T1 & T2 & O1), both for DeltaIoTv1 and DeltaIoTv2. In this case, it is DLASeR that performs the adaptation decision by cleverly selecting adaptation options for verification (based

³For DeltaIoTv2 it is impossible to exhaustively verify all the adaptation options in the available cycle time.

⁴DLASeR does not predict the optimization goal in this experiment. The adaptation space reduction is only performed based on the two threshold goals (T1 & T2).

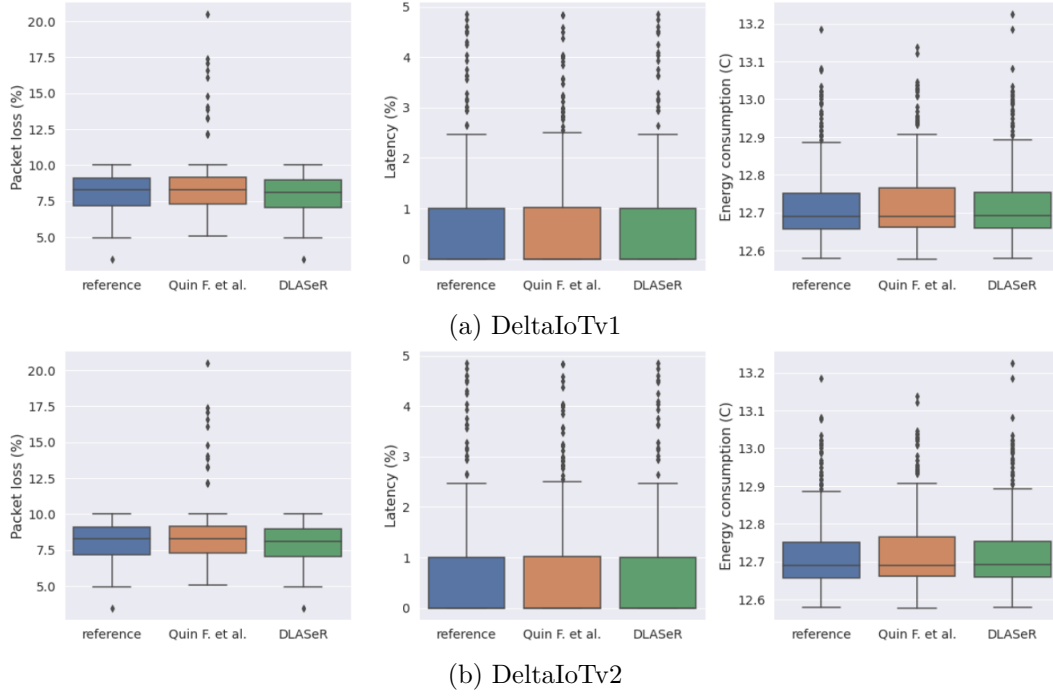


Figure 6.4: Comparison of DLASeR’s quality of the adaptation decisions for threshold goals T1 & T2 of the TTO configuration.

on the optimization goal). The results show that the threshold goals for packet loss and latency are always satisfied with DLASeR. Regarding the optimization goal, the total energy consumption of DLASeR is respectively 0.34% and 1.11% higher for DeltaIoTv1 and DeltaIoTv2, compared to the reference approach. This is a small cost for the dramatic improvement of adaptation time. The total reduction in adaptation time, i.e., the analysis time + the learning time of DLASeR compared to the reference approach, is respectively 92.88% and 94.87% for DeltaIoTv1 and DeltaIoTv2. Table A.2 in the appendix presents a detailed overview of the adaptation time of DLASeR.

6.2.3 Discussion

We observe that DLASeR achieves a very large adaptation space reduction with little overhead compared to the reference approach. The quality of DLASeR’s decision-making is high, as it always satisfies the threshold goals of the TTO configuration. We notice that the optimization goal (energy consumption) is compromised to some extent in favor of a large adaptation space reduction.

In addition to an excellent adaptation space reduction, DLASeR achieves a very large time reduction with little to no jeopardizing of the adaptation goals. Thus, we can conclude that DLASeR successfully indicates the applicability of deep learning for effective and efficient decision-making in self-adaptive systems.

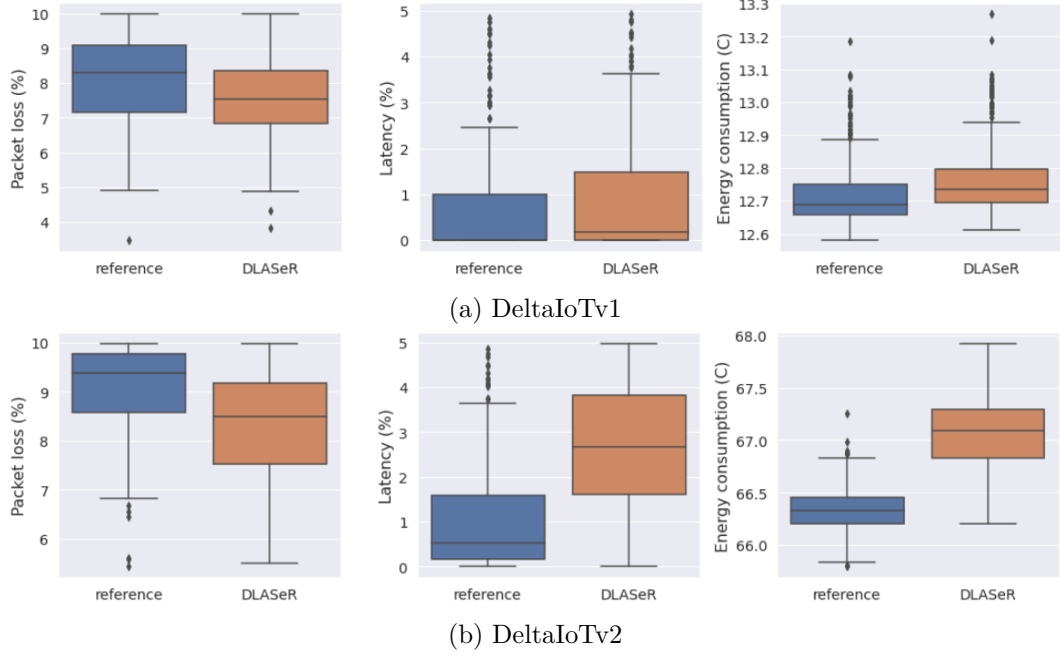


Figure 6.5: Quality of the adaptation decisions for both types of goals (T1 & T2 & O1) of the TTO configuration.

6.3 DLASeR+

For DLASeR+ we evaluate the three configurations of Table 6.1 on both instances of DeltaIoT. We first present, for each configuration, the results of the offline stage of the DLASeR+ pipeline. Then, for each configuration, we present the results. We will elaborate on DLASeR+'s results from a perspective of the research question, i.e., with a focus on effectiveness and efficiency. Where possible, we compare DLASeR+ for the TTO configuration with DLASeR's results. Finally, we discuss the extent to which DLASeR+ achieves the research goal of enabling effective and efficient decision-making in self-adaptive systems.

6.3.1 Offline settings

As explained in subsection 5.2.1, we used grid search for tuning the deep learning model. We performed grid search on 30 sequential cycles. Table 6.4 shows the best parameters of the pipeline for each of the three combinations of adaptation goals. Each row in the table corresponds to a single grid search process. In total, 6 grid search processes were completed; 3 (one for each of the three configurations) $\times$ 2 (the two instances of DeltaIoT). Note that DLASeR requires the same number of grid search processes for just the TTO configuration, as it uses a distinct model for each adaptation goal.

Given that DLASeR+ comprises a single deep neural network architecture, with multiple classification and regression heads, we use the validation loss to select the

Table 6.4: Best grid search results for DLASer+ on the TTO, TTS, and TSO configuration.

Problem	Goals	Hyper parameters						
		Scaler	Batch size	LR	Optimizer	Core layers	Class. layers	Regr. layers
DeltaIoTv1	TTO	Standard	64	5e-3	Adam	[50, 25, 15]	[20,10,5]	[40, 20, 10, 5]
DeltaIoTv2	TTO	Standard	512	5e-3	Adam	[200, 100, 50, 25]	[40, 20, 10, 5]	[30, 40, 15, 5]
DeltaIoTv1	TTS	Standard	64	5e-3	Adam	[50, 25, 15]	[30, 10, 4]	/
DeltaIoTv2	TTS	Standard	512	2e-3	Adam	[200, 100, 50, 25]	[40, 20, 10, 5]	/
DeltaIoTv1	TSO	MinMax	64	5e-3	Adam	[50, 80, 35, 15]	[20,10,5]	[20,10,5]
DeltaIoTv2	TSO	MinMax	512	2e-3	RMSprop	[200, 100, 50, 25]	[30, 10, 4]	[40, 20, 10, 5]

best hyper-parameter configuration. We consider this a valid means as the loss corresponds to the sum of the losses for each head, and thus captures the quality of the classification and regression heads.

In total, grid search evaluated 4120 configurations for DeltaIoTv1 (1728 for TTO and TSO goals and 768 for TTS goals) and 3456 configurations for DeltaIoTv2 (1296 for TTO and TSO goals and 864 for TTS goals). The median of the validation loss for DeltaIoTv1 is 7.439 for TTO goals, 2.166 for TTS goals, and 13.09 for TSO goals. For DeltaIoTv2, the median of the validation loss is 367.8 for TTO goals, 1.374 for TTS goals, and 12.69 for TSO goals.

The validation losses for the selected TTO, TTS, and TSO goal combinations are respectively 0.427, 0.974, and 1.306 for DeltaIoTv1. For DeltaIoTv2, the validation losses are respectively 0.919, 0.880, and 1.070. We observe that the loss for the configuration with TTO goals is significantly lower for DeltaIoTv1. For the other two configurations, the loss for DeltaIoTv2 is lower. Further analysis will show that these differences do not lead to significantly inferior results.

Based on the obtained configurations, we deploy the three DLASer+ instances (corresponding to the three configurations) in the simulator.

6.3.2 Results

With DLASer+ we aim to answer the research question of [section 1.2](#). To that end, we elaborate in detail on each aspect of the research question (see [Table 3.1](#)). First, we discuss the three aspects defining the effectiveness; the coverage of relevant adaptation options, the reduction of the adaptation space, and the effect on realization of the adaptation goals. Then, we continue to investigate the two aspects defining the efficiency; the adaptation time, and the scalability. While we answer the various parts of the research question, we compare DLASer+ with DLASer and other related work.

Effectiveness - Coverage of relevant adaptation options

To assess the first aspect of the effectiveness of DLASer+, we look at the F1 score for the threshold and set-point goals, and Spearman’s rho for the optimization goal. [Table 6.5](#) presents the results for each of the types of adaptation goals.

Table 6.5: Results for the coverage of relevant adaptation options for the different evaluation settings.

Problem	Setting	F1 threshold	F1 set-point	Spearman’s rho optimization
DeltaIoTv1	TTO	81.01%	/	74.42%
	TTS	79.35%	70.26%	/
	TSO	95.76%	71.22%	91.49%
DeltaIoTv2	TTO	69.50%	/	24.86%
	TTS	71.71%	67.85%	/
	TSO	96.27%	67.85%	95.97%

For DeltaIoTv1, we notice an F1 score for the threshold goals around 80% for the settings TTO and TTS, while the score for TSO is significantly higher with 95.76% (with 100% being perfect precision and recall). This discrepancy can be explained by the constraints imposed by the threshold goals. Since the setting of TSO has only one threshold goal, the F1 score is (obviously) higher than for the other settings that combine two threshold goals. The F1 score for the set-point goals of TTS and TSO are similar, around 70%. Note that despite the TTO and TTS settings having the same threshold goals, we observe for both DeltaIoT instances a difference in F1 score of around 2%. This discrepancy might be caused on the one hand by the uncertainties present in the DeltaIoT simulator. On the other hand, we need to consider that these two settings were handled by two independently (online) trained neural networks (on slightly different simulator data). The same explanation holds for the set-point goals of DeltaIoTv1. For Spearman’s rho, we observe a difference between the settings with an optimization goal, with 74.42% and 91.49% for TTO and TSO respectively. This shows that regression is more difficult for energy consumption (TTO) compared to packet loss (TSO).

For DeltaIoTv2, we observe an F1 score for the threshold goals around 70% for the settings TTO and TTS, while the threshold F1 score is significantly higher for TSO with 96.27%. The F1 score for the two set-point goals, are exactly the same, with 67.87%. These results for F1 score are similar to the results for DeltaIoTv1. For Spearman’s rho, we observe values of 24.86% and 95.97% for TTO and TTS respectively. The low score for the setting with TTO might seem problematic, but further evaluation will show that even a lower Spearman’s rho value does not imply inferior results.

Overall, we obtained acceptable to excellent results for the F1 score and Spearman’s rho. Compared to the results obtained by Quin F. et al. [53] (where classic machine learning was applied for adaptation space reduction) and DLASeR we observe similar results. Note that Quin F. et al. and DLASeR only considered the TTO setting for both versions of DeltaIoT; the other combinations of goals were not studied.

Effectiveness - Reduction adaptation space

The second aspect of effectiveness tackles the reduction of the adaptation space. To that end, we consider the three metrics presented in [section 3.1](#), i.e., AASR, AANR, and total reduction. [Table 6.6](#) displays the results for adaptation space reduction obtained with DLASer+ for the different configurations.

Table 6.6: Results for the reduction of the adaptation space for DLASer+ on the three configurations.

Problem	Configuration	AASR	AANR	Total reduction
DeltaIoTv1	TTO	66.99%	94.55%	98.20%
	TTS	87.85%	86.50%	98.36%
	TSO	46.40%	94.81%	97.22%
DeltaIoTv2	TTO	94.17%	99.00%	99.94%
	TTS	95.77%	98.49%	99.94%
	TSO	58.60%	99.86%	99.95%

We observe that the TTS setting for both IoT instances achieves the highest average adaptation space reduction (AASR). This can be explained by the fact that TTS has three classification goals, whereas the two other settings have only two classification goals. We also observe that the AASR for TTO is considerably higher than for TSO. This indicates that the two threshold goals of TTO are more restrictive than the threshold and set-point goal of TSO.

For the average analysis reduction (AANR), we notice that only a limited number of adaptation options need to be analyzed from the selected subspace before a valid one is found that complies with the classification goals. This means that the adaptation space reduction captured by AASR is of high quality. Only for the TTS setting of DeltaIoTv1 we observe a slightly smaller value for AANR, meaning that this setting required some more analysis.

For DeltaIoTv1, we measured a total reduction for the three combinations of goals of 97.93%, while the total reduction for DeltaIoTv2 was 99.94%. These excellent results show that DLASer+ is particularly effective in reducing the adaptation space, i.e., the total reduction is near to the optimum of what can be achieved.

In [Table A.3](#) in the appendix we merged the results of DLASer+ with the results of DLASer and Quin F. et al. [\[53\]](#) (see [Table 6.3](#)). We compared the results of the TTO configuration. Note that Quin F. et al. focused on only threshold goals, i.e., perform adaptation space reduction on the two threshold goals T1 & T2. We first discuss the results when only the two threshold goals are used for learning. In this case, the optimization goal is not learned and thus all the adaptation options of the reduced subspace are verified.⁵ We observe that DLASer+ achieves the highest total

⁵For DLASer+ we present the AASR instead of the total reduction (when the goals are just T1 & T2). We did this because it was impossible to randomly select adaptation options for verification from the reduced subspace, since the optimization goal O1 was used to make decision-making (and

reduction of the three approaches. For DeltaIoTv1 the total reduction is (in absolute numbers) respectively 2.57% and 16.99% higher than for DLASer and Quin F. et al. For DeltaIoTv2 the total reduction is (in absolute numbers) 1.23% higher than for DLASer and 18.67% than for Quin F. et al. When we look at the TTO setting (now the optimization is also learned), we notice that DLASer+ achieves again the highest total reduction. The total reduction is (in absolute numbers) respectively 3.39% and 1.44% higher than DLASer for DeltaIoTv1 and DeltaIoTv2.

Effectiveness - Effect on realization of adaptation goals

As final aspect of the effectiveness, we consider the effect on the realization of adaptation goals. In order to assess this aspect, we look at the differences in distribution, observed in box plots, of DLASer+ and the reference approach. These box plots are shown in Figure 6.8, Figure 6.9, and Figure 6.10, and present the results for the quality properties of the adaptation goals with DLASer+ and a reference approach. Recall that the reference approach exhaustively analyzes the whole adaptation space. This is the ideal case, but practically not always feasible due to time constraints.⁶

For the setting with TTO goals, see Figure 6.8, the results for packet loss and latency (threshold goals) are similar to the reference approach. We observe that DLASer+ always satisfies the goals (just as DLASer). For energy consumption (optimization goal), the results for DLASer+ are slightly higher compared to the reference approach, i.e., 0.42% for DeltaIoTv1 and 1.21% for DeltaIoTv2. Figure A.2 in the appendix compares the effect on the realization of adaptation goals for threshold goals T1 & T2 (of the TTO configuration). We compare DLASer+ with DLASer and Quin F. et al. [53]. For this setting, we observe that the performance of DLASer and DLASer+ is very similar. The results lie also in line with those of Quin F. et al., except for threshold goal on packet loss (T1) for which their approach had some violations. Figure A.3 in the appendix compares DLASer+'s effect on the realization of adaptation goals for the TTO setting with DLASer. Again, we observe very similar results for both approaches.

For the TTS setting, see Figure 6.9, we observe similar results for DLASer+ and the reference approach. The threshold goals for packet loss and latency are again always satisfied by DLASer+. The set-point goal of energy consumption is also satisfied for both instances of DeltaIoT. In Figure 6.6 we present the quality values of the selected adaptation options of the set-point goal (energy consumption) for this setting. We visually observe that the set-point goal is always satisfied.

Finally, for the setting with TSO goals, see Figure 6.10, the results show that the threshold goal for latency is always satisfied with DLASer+ and the same applies for the set-point goal for energy consumption. As for the optimization goal, the relative packet loss of DLASer+ is slightly higher compared to the reference approach,

this goal was not learned). Therefore, DLASer+ verified (exceptionally) the complete reduced subspace.

⁶For DeltaIoTv2 it is impossible to exhaustively verify all the adaptation options in the available cycle time. (this is the exact same note as footnote 3)

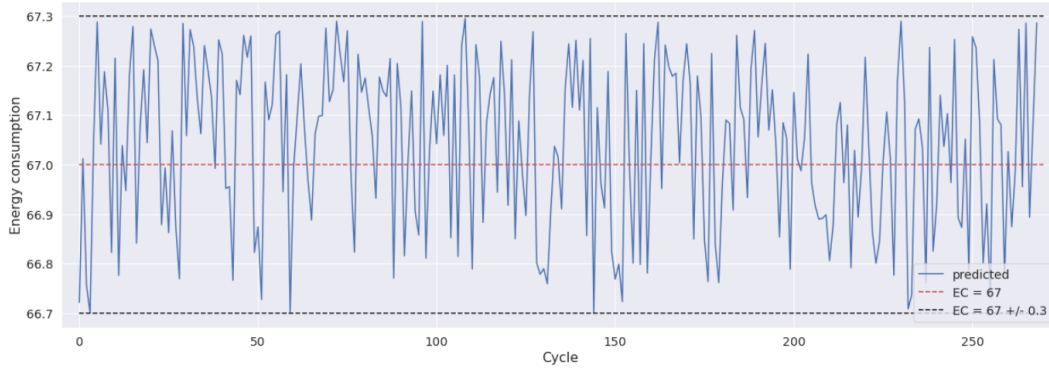


Figure 6.6: The energy consumption (S1) of the selected adaptation option over 274 sequential cycles for the TTS setting on DeltaIoTv2. The gray dotted lines indicate the bounds of the set-point goal.

respectively 10.51% and 14.64% for DeltaIoTv1 and DeltaIoTv2. Figure 6.7 displays the ranking of the adaptation options. We observe that the quality of the ranking is rather high. The range of packet loss values is very large, i.e., from 6% to 20%. Thus, an average (relative) increase of 15% corresponds to a packet loss of 6.9%, which is not too bad considering the best (possible) value is 6% and the worst is 20%.

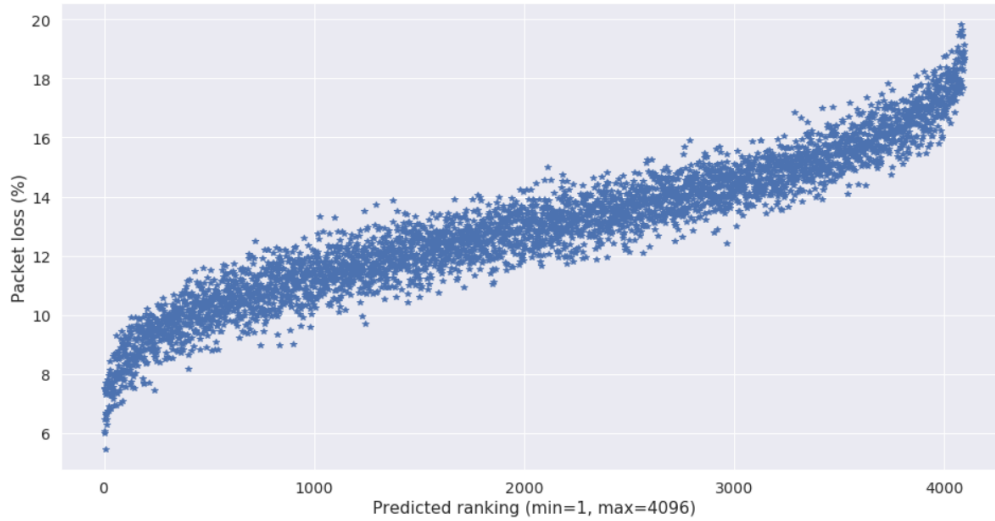


Figure 6.7: The predicted ranking of the packet loss (O1) for the TSO setting on DeltaIoTv2 for a certain adaptation cycle. On the y-axis we present the verified packet loss. On the x-axis, the adaptation options are ordered according to their predicted ranking (low = left, high = right).

In summary, the results for the quality properties using DLASer+ and the reference approach are similar. DLASer+ realizes all threshold and set-point goals, and the effect on the realization of the optimization goal with DLASer+ is small to negligible compared to the reference approach.

6. EVALUATION

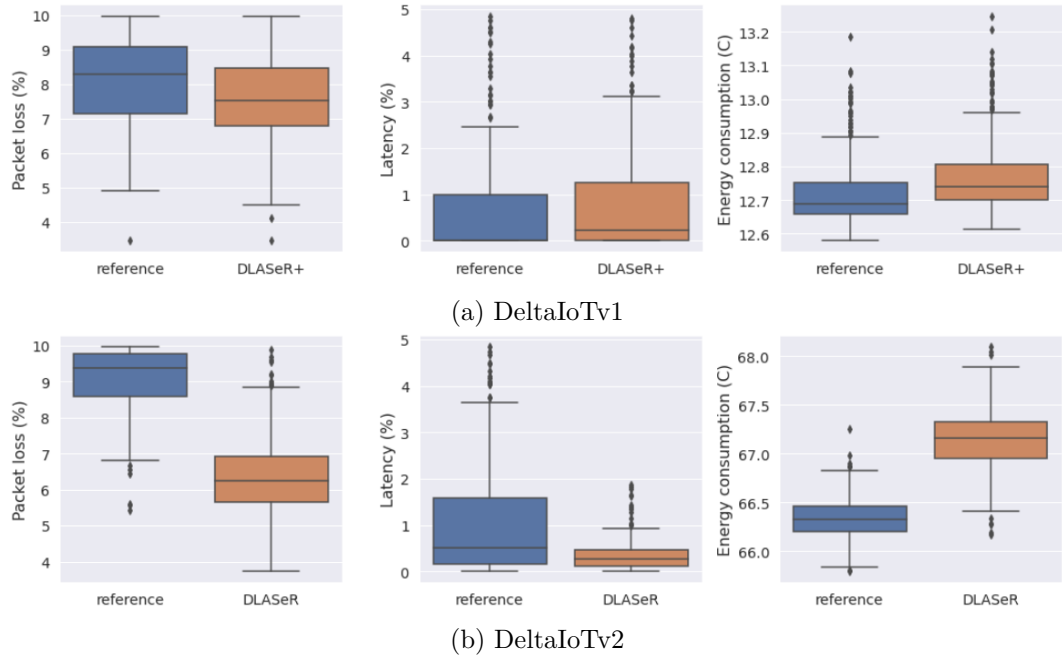


Figure 6.8: Effect on the realization of the adaptation goals for the TTO setting.

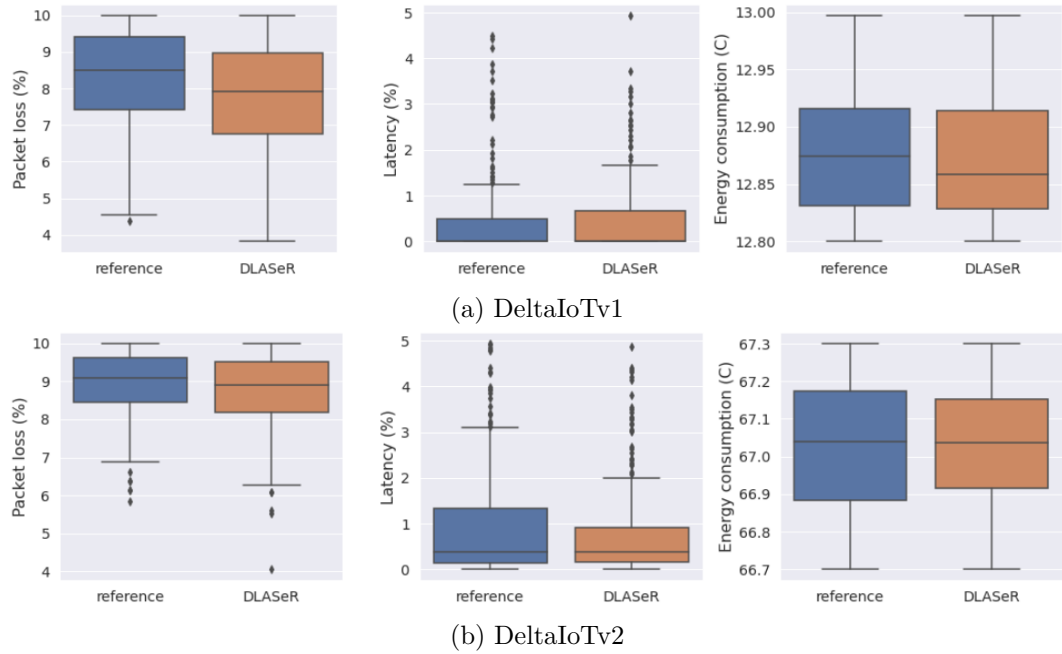


Figure 6.9: Effect on the realization of the adaptation goals for the TTS setting.

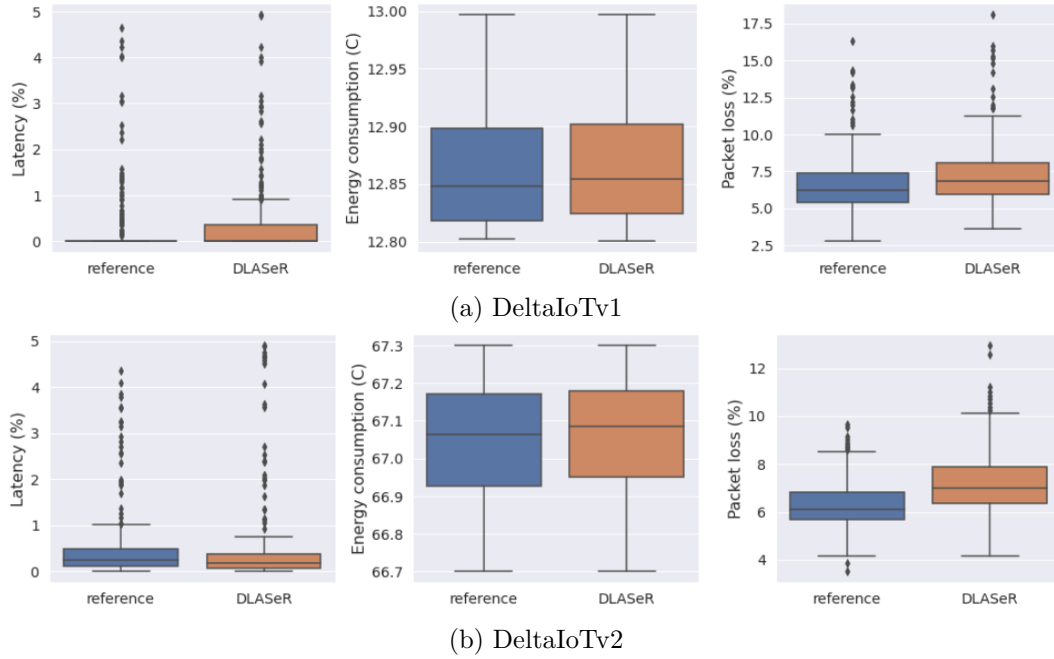


Figure 6.10: Effect on the realization of the adaptation goals for the TSO setting.

Efficiency - Adaptation time

To assess the first aspect of the efficiency of DLASer+, we look at the adaptation time. Concretely, we consider the analysis time (i.e., the verification time), the learning time, and the time reduction (i.e., the time reduction of DLASer+ compared to the reference approach). Table 6.7 presents the results for these three measures.

Table 6.7: Analysis and learning time for DLASer+. Time reduction compares the total time used by DLASer+ with the time used by the reference approach.

Problem	Setting	Analysis time	Learning time	Time reduction
DeltaIoTv1	TTO	2.09 s	0.69 s	90.66%
	TTS	3.44 s	0.71 s	86.05%
	TSO	2.20 s	0.57 s	90.69%
DeltaIoTv2	TTO	50.32 s	2.83 s	94.70%
	TTS	51.22 s	4.49 s	94.45%
	TSO	50.53 s	5.59 s	94.44%

We observe that the major part of the time is spent on analysis, i.e., the verification of adaptation options. This is in contrast with the learning time, on average 20.3% of the time is spent on learning for DeltaIoTv1 and 7.8% for DeltaIoTv2. We observe very high time reductions, on average 89.1% for DeltaIoTv1 and 94.5% for DeltaIoTv2. Note that the optimal time reduction is 95%, since we used an exploration rate of 5%,

i.e., in each adaptation cycle, 5% of the (unexplored) adaptation space is explored.⁷

When we compare with DLASeR (see Table A.2 in the appendix), we observe that the time reduction is slightly better for DLASeR. This difference is caused by the learning time being smaller for DLASeR compared to DLASeR+, i.e., respectively 0.43 s and 0.92 s learning time for the TTO configuration of DeltaIoTv1 and DeltaIoTv2. Despite this discrepancy in learning time, the absolute difference in total time reduction remains minimal, i.e., respectively 2.22% and 0.17% higher time reduction in favor of DLASeR.

In sum, the learning time of DLASeR+ is only a fraction of the total time used for analysis. DLASeR+ realizes an impressive reduction of the time required for decision-making compared to the reference approach.

Efficiency - Scalability

The final aspect of efficiency tackles scalability. To that end, we discuss the difference for the metrics (of the four aspects above) when scaling up the evaluation setting from DeltaIoTv1 with 216 adaptation options to DeltaIoTv2 with 4096 adaptation options.

For the *coverage of relevant adaptation options* (see Table 6.5), we observe (in absolute numbers) a decrease in F1 score for the threshold goals of 11.51% and 7.64% for respectively the settings with TTO and TTS goals, but a minor increase of 0.51% for TSO. Considering the set-point goals, we observe (in absolute numbers) a small decrease in F1 score of 2.41% for the TTS setting and 3.37% for TSO setting. For the optimization goal, we notice (in absolute numbers) a large decrease for Spearman's rho from 74.42% to 24.86% for the setting with TTO goals, but a small increase from 91.49% to 95.97% for the setting with TSO goals. Yet, these results indicate that (in general) the coverage of relevant adaptation options for larger adaptation spaces tends to slightly decrease.

Considering the *reduction of the adaptation space* (see Table 6.6), we look at the AASR, AANR, and total reduction when increasing the adaptation space from DeltaIoTv1 to DeltaIoTv2. We measured (in absolute numbers) an increase in AASR for each of the three configurations when scaling up to DeltaIoTv2, on average 15.6% over the three configurations. For the AANR, the results also improve for the three configurations with DeltaIoTv2, with an average of 7.5% (in absolute numbers). Summarizing for the total reduction, we observe that we achieve for DeltaIoTv2 a near optimal reduction of more than 99.9%. This total reduction is (in absolute numbers) on average 2.02% higher than for DeltaIoTv1.⁸

For the *effect on realizing the adaptation goals* (see Figure 6.8, Figure 6.9, and Figure 6.10), we observe that DLASeR+ realizes the threshold and set-point goals for

⁷Note that 5% of the unexplored adaptation space will always be less than 5% of the complete adaptation space. But, a (considerable) chunk of the adaptation space is verified before we proceeded to this random exploration. These two verification steps will always verify (in total) more than 5% of the adaptation space. Therefore, we consider the limit of 95% for the time reduction as valid.

⁸Since DeltaIoTv1 has a smaller adaptation space, it is obvious that maximum possible reduction of the adaptation space is smaller than for a larger adaptation space.

all configurations on both DeltaIoTv1 and DeltaIoTv2. Considering the optimization goal, the total energy consumption for the TTO setting slightly increases from 0.42% for DeltaIoTv1 to 1.21% for DeltaIoTv2 compared to the reference approach. For the setting with TSO goals, we observe that the packet loss also slightly increases from 10.51% for DeltaIoTv1 to 14.64% for DeltaIoTv2 compared to the reference approach.

Finally, the results for the *adaptation time* (see Table 6.7) show that the ratio of the time required for learning compared to the time required for analysis decreases from 20.8% for DeltaIoTv1 to 7.8% for DeltaIoTv2. The results show that with increasing scale, the total time reduction improves (in absolute numbers) on average with 5.40% and nears closer to the optimum for DeltaIoTv2.

In summary, while some of the indicators for the coverage of relevant adaptation options are slightly inferior for configurations with larger adaptation spaces, the other metrics of Table 3.1 show no negative effects on the effectiveness and efficiency of DLASer+ when the size of the adaptation space is increased (here with a factor of about 20).

6.3.3 Discussion

Just as for DLASer, we observe that DLASer+ achieves a very large adaptation space reduction with little overhead compared to the reference approach. The quality of DLASer+'s decision-making is high, as it always satisfies the threshold and set-point goals for each of the three configurations. We notice that the optimization goal is compromised to some extent (slightly for the TTO configuration and considerably for the TSO configuration). Yet this is a small cost for the dramatic improvement of adaptation time.

To summarize, we can state that together with an excellent reduction of the adaptation space, DLASer+ achieves a very large time reduction with little to no jeopardizing of the adaptation goals. When scaling up to a larger instance the reduction of the adaptation space remains impressive, but the effectiveness is slightly compromised for optimization goals. Thus, we can conclude that DLASer+ successfully achieves the research goal. Yet, there is still some room for improvement, albeit mainly for optimization goals.

6.4 Threats to validity

The evaluation results for DLASer+ and DLASer show the applicability of deep learning as an effective and efficient approach to reduce large adaptation spaces. However, the evaluation is subject to a number of validity threats.

External validity. Since we evaluated the approach in only one domain, we cannot generalize the conclusions. We mitigated this threat to some extent by applying DLASer+ to two IoT networks that differ in their topology and size of adaptation space. Nevertheless, more extensive evaluation is required in different domains to strengthen the validity of the results. Furthermore, the optimization stage of

DLASeR and the reduction stages of the unified architecture of DLASeR+ target only a single optimization goal. Hence, the approach is not directly applicable to systems with multi-objective optimization goals. Additional research will be required to extend our work for such settings.

Internal validity. We have evaluated the effectiveness and efficiency of DLASeR+ using different metrics (see Table 3.1). It might be possible that the specifics of the evaluation setting of the applications that we used, in particular the topology of the network, the uncertainties, and the choices for the specific goals that we considered may have an effect on the complexity of the problem of adaptation space reduction. To mitigate this threat to some extent we applied DLASeR+ to simulated settings of real IoT deployments that were developed in close collaboration with an industry partner in IoT.

Reliability. For practical reasons, we performed the evaluation of DLASeR and DLASeR+ in simulation. The uncertainties used in this simulation are based on stochastic models. This may cause a threat that the results may be different if the study would be repeated. We minimized this threat in three ways: (i) the profiles are designed based on field tests, (ii) we evaluated the two approaches over a period of several days, and (iii) the complete package we used for evaluation is available to replicate the study; <https://github.com/jvdd/DLASeR>.

Chapter 7

Related work

Over the past years, we can observe an increasing interest in using machine learning and related techniques to support self-adaptation. Examples of initial work are Richert W. et al. [55] that apply reinforcement learning to robots that learn to select tasks that balance their own benefit with the needs of other robots, and Sykes D. et al. [62] that use a probabilistic rule learning approach to update the environment models of a self-adaptive system at runtime relying on execution traces.

In this chapter, we focus on the use of machine learning and other related techniques that are used for adaptation space reduction. We have structured the related work in five groups based on their main focus: (1) learning used for the reduction of adaptation spaces, (2) learning used for the reduction of features, (3) learning used for efficient decision-making, (4) search-based techniques to efficiently explore large adaptation spaces, and finally (5) approaches for efficient verification. For each group we discuss a representative selection of related work.

7.1 Learning used for adaptation space reduction

Quin F. et al. [53] apply classical machine learning techniques, in particular classification and regression, to reduce large adaptation spaces. That work only considers threshold goals based on linear learning models. In contrast, our work considers threshold, optimization, and set-point goals based on non-linear deep learning models.

Jamshidi P. et al. [32] present an approach that learns a set of Pareto optimal configurations offline, which are then used during operation to generate adaptation plans. The approach reduces adaptation spaces, while ensuring that the system can still use PRISM [40] at runtime to quantitatively reason about adaptation decisions. Compared to that work, DLASeR and DLASeR+ are more versatile by reducing the adaptation space at runtime in a dynamic and learnable fashion.

Camara J. et al. [8] use reinforcement learning to select an adaptation pattern relying on two long-short-term memory deep learning models. Similar to our work, these authors demonstrate the benefits of integrating deep learning with runtime verification. However, the focus differs in the type of goals considered (they only con-

sider threshold goals) and the type of verification used (they use runtime quantitative verification). Their work also does not consider scalability.

7.2 Learning used for the reduction of features

Elkhodary A. et al. [18] propose the FUSION framework that learns the impact of adaptation decisions on the system's goals. In particular, FUSION utilizes M5 decision trees to learn the utility functions that are associated with the qualities of the system. The results show a significant improvement in analysis. Whereas our work targets the adaptation space, FUSION targets the feature selection space, focusing on proactive latency-aware adaptations, relying on a separate model for each utility.

Metzger A. et al. [43] apply online learning to explore the adaptation space of self-adaptive systems using feature models. The authors demonstrate a speedup in convergence of the online learning process. Their approach is centered on rule-based adaptation, whereas our work is centered on model-based self-adaptation. Furthermore, the work also looks at the evolution of the adaptation space, while DLASeR and DLASeR+ only consider dynamics

Chen T. et al. [10] study feature selection and show that various learning algorithms perform significantly different depending on the types of quality of service attributes and their fluctuations. The work is centered on an adaptive multi-learners technique that dynamically selects the best learning algorithms at runtime. Similar to our work, the authors target efficiency and effectiveness, but the focus is on the feature selection space.

7.3 Learning used for efficient decision-making

Kim D. et al. [34] present a reinforcement learning-based approach. The framework enables a software system to improve its behavior by learning the results of its actions and dynamically changing its plans based on the learning in presence of environmental changes. Compared to DLASeR and DLASeR+, the focus is on just effective adaptation decision-making. This work does not rely on verification techniques to ensure guarantees, nor do the authors study the scalability of their approach.

Anaya I. D. P. et al. [4] present a framework for proactive adaptation. The framework fuses predictive models with historical information to model the environment. The predictions are fed to a reasoning engine to improve the decision-making process. The authors show that their approach outperforms a typical reactive system by evaluating different prediction models (classifiers). Whereas they focus on leveraging proactive techniques to make decision-making more effective, our work focuses on both the efficiency and effectiveness of decision-making.

Qian W. et al. [52] study goal-driven self-adaptation centered on case-based reasoning for storing and retrieving adaptation rules. Depending on goal violations and context changes, similar cases are used and if they are not available, goal reasoning

is applied. This way, the approach realizes more precise adaptation decisions. The evaluation is done only for threshold goals and the authors only provide some hints to scale up their solutions to large-sized systems.

Ho H. N. et al. [26] rely on model-based reinforcement learning to improve system performance. By utilizing engineering knowledge, the system maintains a model of interaction with its environment and predicts the consequence of its actions. Just as DLASer and DLASer+, their work focuses on effective adaptation decision-making. The authors do not use techniques to ensure guarantees, nor do they study scalability.

Stevens C. et al. [60] present Thallium. Their approach exploits a combination of modeling techniques to significantly reduce the number of states that need to be considered with each adaptation decision. Thallium addresses the adaptation state explosion by applying utility bounds analysis. The approach (currently) targets solutions operating on a Markov decision process that represents the structure of the system itself, independent of the stochastic environment. Our work on the other hand focuses on effective and efficient reduction of adaption spaces that rely on hybrid quality models that take the system and its environment into account.

7.4 Search-based techniques to explore large adaptation spaces

Nair V. et al. [48] present FLASH that aims at quickly finding good configurations of a software system. FLASH sequentially explores the configuration space by reflecting on the configurations evaluated so far to determine the next best configuration to explore. FLASH can solve both single-objective and multi-objective optimization problems. Whereas FLASH assumes that the system is stationary, DLASer and DLASer+ use incremental learning to stay up to date at runtime.

Kinneer C. et al. [35] propose a planner based on genetic programming that reuses existing plans. Their approach uses stochastic search to deal with unexpected adaptation strategies, specifically by exploiting prior knowledge. Their genetic programming planner is able to handle very large search spaces. Similar to DLASer+, the evaluation of their work considers efficiency and effectiveness. However, their focus lies exclusively on planning, and they put particular emphasis on reuse.

Chen T. et al. [11] present FEMOSAA, a framework that leverages a feature model and a multi-objective evolutionary algorithm to optimize the decision-making of adaptation options at runtime. The authors demonstrate that FEMOSAA produces statistically better and more balanced results, with a reasonable overhead compared to other search-based techniques as trade-off. Compared to our work, the authors use a different analysis technique and rely on feature models. The latter implies that the approach relies on domain expertise to construct a feature model for the self-adaptive system.

7.5 Approaches for efficient verification

Filieri A. et al. [19] present a mathematical framework for efficient run-time probabilistic model checking. Before deployment, a set of symbolic expressions that represent satisfaction of the requirements is precomputed. At runtime, the verification step simply evaluates the formulae by replacing the variables with the real values gathered by monitoring the system. By shifting the cost of model analysis partially to design time, the approach enables more efficient verification at runtime. Whereas our approach focus on reducing the set of adaptation options, their work focuses on efficient runtime verification.

Gerasimou S. et al. [22] propose three techniques, namely caching, lookahead and nearly-optimal reconfiguration to speed up runtime quantitative verification. The authors evaluate several combinations of the techniques on various states of self-adaptive systems. The focus of this work is different from DLASeR and DLASeR+, but the proposed techniques can perfectly be integrated in our work.

Moreno G. A. et al. [46] present an approach for proactive latency-aware adaptation that relies on stochastic dynamic programming to enable more efficient decision-making. Experimental results show that this approach is nearly a magnitude faster than runtime probabilistic model checking to make adaptation decisions, while preserving the same effectiveness. Whereas our approach focus on reducing the set of adaptation options, their work focuses on fast verification. Their work can complement ours to enable even more efficient analysis.

Chapter 8

Conclusion

In this work, we focused on the research goal: *Enable effective and efficient decision-making in self-adaptive systems based on threshold, optimization, and set-point goals*. To tackle this research goal, we presented DLASeR and DLASeR+; two novel frameworks that leverage deep learning to enhance decision-making in self-adaptive systems.

The former was developed as an initial approach with as aim to indicate the applicability of deep learning to reduce large adaptation spaces. Based on the evaluation results we can conclude that DLASeR successfully indicated that deep learning can effectively reduce large adaptation spaces, without compromising the system goals. This initial work can thus be considered as a first step towards effective and efficient decision-making.

The latter can be considered as a more general and extendable version of DLASeR. In addition to the two types of adaptation goals supported by DLASeR, i.e., threshold and optimization goals, DLASeR+ also supports set-point goals. In contrast with our initial work, where a grid search process is required for every goal, DLASeR+ requires only a single grid search for all adaptation goals. This is because DLASeR+ relies on an integrated deep neural network architecture that shares knowledge among different adaptation goals, whereas DLASeR uses a separate deep neural network for each adaptation goal. Evaluating DLASeR+ showed that together with an excellent reduction of the adaptation space, it achieves a very large time reduction with little to no jeopardizing of the adaptation goals. The approach realizes the threshold and set-point goals for all the evaluation configurations we tested. Compared to the ideal approach, we observe only a small trade-off for the quality property of the optimization goal. Yet, this is a minor cost for the dramatic improvement of adaptation time. Therefore, we conclude that we achieved successfully the research goal, enabling effective and efficient decision-making in self-adaptive systems.

8.1 Future work

Given that DLASeR+ is considered as a refined version of DLASeR, we present future work mainly for this approach.

First, we want to validate DLASer⁺'s performance beyond the domain of IoT. To that end, we are currently applying the approach to service-based systems. If this evaluation indicates that the optimization goal is (again) slightly compromised, we plan to look into alternative techniques to further improve the effectiveness for optimization goals. One possible solution is to let DLASer⁺ verify the top- k ranked adaptation options, with k specified by the user. We suspect that this might boost the effectiveness of DLASer⁺ for optimization goals at the (controllable with k) cost of efficiency.

Next to this, it might be interesting to evaluate the extent to which DLASer⁺ is versatile. Concretely, we plan to evaluate the effect of adding and changing adaptation goals at runtime. In contrast with DLASer, adding or changing adaptation goals at runtime would not require to train a deep neural network from scratch. DLASer⁺ is able to leverage the representations learned by the shared layers.

Next, we have some suggestions to the architecture of DLASer⁺. First, we plan to extend DLASer⁺ for multi-objective optimization goals. This would require some significant changes in the reduction cycle. One concrete way to realize multi-objective optimization goals is by adopting user defined utility functions. Next, we want to research an architecture with only regression heads. The qualities regressed for threshold and set-point goals, will be converted to binary values in the reduction cycle. There is a very important reason that we did not explore this option first; for a complex problem, regression tends to be a lot harder than binary classification. Note that in an architecture with only regression heads changing goals would (probably) be a lot easier. Since it is the reduction cycle that handles the change and thus no new concepts have to be learned by the deep neural network. This architecture would use the same principles as FUSION [18], except that it targets the configuration space and not the feature space. We also plan to look into the loss function of the deep neural network. It might be interesting to alter the importance of the heads in the loss function at runtime. This enables to focus dynamically on heads with poor performance and omits an ambiguous distribution of importances that is determined in advance. Next, we want to increase DLASer⁺'s performance by upgrading the verifier to more efficient variants, see for example [22, 46]. As our current approach explores rather naively unselected adaptation options, we could also leverage techniques from active learning. This would mean that the deep neural network decides, based on uncertainty, which options it wants to verify [20]. We suspect that incorporating these extensions would lead to faster convergence and thus result in a more effective and efficient decision-making. Finally, we have some plans for the mid term and the long term. In the mid term, we plan to explore the application of machine learning to self-adaptation in a decentralized setting. In the long term, we aim to investigate how we can define bounds on the guarantees that can be achieved when combining (as in this work) formal analysis techniques, in particular runtime statistical model checking, with machine learning.

Bibliography

- [1] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, et al. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, pages 265–283, 2016.
- [2] G. Agha and K. Palmskog. A survey of statistical model checking. *ACM Trans. Model. Comput. Simul.*, 28(1):6:1–6:39, Jan. 2018.
- [3] G. Alain and Y. Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.
- [4] I. D. P. Anaya, V. Simko, J. Bourcier, N. Plouzeau, and J.-M. Jézéquel. A prediction-driven adaptation approach for self-adaptive sensor networks. In *Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, pages 145–154, 2014.
- [5] A. P. Athreya, B. DeBruhl, and P. Tague. Designing for self-configuration and self-adaptation in the internet of things. In *9th IEEE International Conference on Collaborative Computing: Networking, Applications and Worksharing*, pages 585–592. IEEE, 2013.
- [6] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.
- [7] R. Calinescu, L. Grunske, M. Kwiatkowska, R. Mirandola, and G. Tamburrelli. Dynamic qos management and optimization in service-based systems. *IEEE Transactions on Software Engineering*, 37(3):387–409, May 2011.
- [8] J. Cámara, H. Muccini, and K. Vaidhyanathan. Quantitative verification-aided machine learning: A tandem approach for architecting self-adaptive iot systems. In *Proceedings International Conference on Software Architecture, ICSA*, 2020.
- [9] L. Castañeda, N. M. Villegas, and H. A. Müller. Self-adaptive applications: On the development of personalized web-tasking systems. In *Proceedings of the 9th international symposium on software engineering for adaptive and self-managing systems*, pages 49–54, 2014.

- [10] T. Chen and R. Bahsoon. Self-adaptive and online qos modeling for cloud-based software services. *IEEE Transactions on Software Engineering*, 43(5):453–475, 2016.
- [11] T. Chen, K. Li, R. Bahsoon, and X. Yao. Femosaa: Feature-guided and knee-driven multi-objective optimization for self-adaptive software. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 27(2):1–50, 2018.
- [12] B. H. Cheng, R. de Lemos, H. Giese, P. Inverardi, J. Magee, J. Andersson, B. Becker, N. Bencomo, Y. Brun, B. Cukic, et al. Software engineering for self-adaptive systems: A research roadmap. In *Software engineering for self-adaptive systems*, pages 1–26. Springer, 2009.
- [13] B. H. Cheng, P. Sawyer, N. Bencomo, and J. Whittle. A goal-based modeling approach to develop requirements of an adaptive system with environmental uncertainty. In *International Conference on Model Driven Engineering Languages and Systems*, pages 468–483. Springer, 2009.
- [14] A. David, K. Larsen, A. Legay, M. Mikučionis, and D. Poulsen. Uppaal smc tutorial. *International Journal on Software Tools for Technology Transfer*, 17(4):397–415, 2015.
- [15] R. De Lemos, D. Garlan, C. Ghezzi, H. Giese, J. Andersson, M. Litoiu, B. Schmerl, D. Weyns, L. Baresi, N. Bencomo, et al. Software engineering for self-adaptive systems: Research challenges in the provision of assurances. In *Software Engineering for Self-Adaptive Systems III. Assurances*, pages 3–30. Springer, 2017.
- [16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [17] G. Edwards, J. Garcia, H. Tajalli, D. Popescu, N. Medvidovic, G. Sukhatme, and B. Petrus. Architecture-driven self-adaptation and self-management in robotics systems. In *2009 ICSE Workshop on Software Engineering for Adaptive and Self-Managing Systems*, pages 142–151. IEEE, 2009.
- [18] A. Elkhodary, N. Esfahani, and S. Malek. Fusion: a framework for engineering self-tuning self-adaptive software systems. In *Proceedings of the eighteenth ACM SIGSOFT international symposium on Foundations of software engineering*, pages 7–16, 2010.
- [19] A. Filieri, C. Ghezzi, and G. Tamburrelli. Run-time efficient probabilistic model checking. In *2011 33rd International Conference on Software Engineering (ICSE)*, pages 341–350. IEEE, 2011.
- [20] Y. Gal and Z. Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059, 2016.

- [21] D. Garlan, S. Cheng, A. Huang, B. Schmerl, and P. Steenkiste. Rainbow: Architecture-based self-adaptation with reusable infrastructure. *Computer*, 37(10):46–54, Oct. 2004.
- [22] S. Gerasimou, R. Calinescu, and A. Banks. Efficient runtime quantitative verification using caching, lookahead, and nearly-optimal reconfiguration. In *Proceedings of the 9th international symposium on software engineering for adaptive and self-managing systems*, pages 115–124, 2014.
- [23] A. Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O’Reilly Media, 2019.
- [24] S. Ghahremani, H. Giese, and T. Vogel. Improving scalability and reward of utility-driven self-healing for large dynamic architectures. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, 14(3):1–41, 2020.
- [25] I. Goodfellow, Y. Bengio, and A. Courville. *Deep learning*. MIT press, 2016.
- [26] H. N. Ho and E. Lee. Model-based reinforcement learning approach for planning in self-adaptive software system. In *Proceedings of the 9th International Conference on Ubiquitous Information Management and Communication*, pages 1–8, 2015.
- [27] C.-L. Hwang and A. S. M. Masud. *Multiple objective decision making-methods and applications: a state-of-the-art survey*, volume 164. Springer Science & Business Media, 2012.
- [28] M. U. Iftikhar, G. S. Ramachandran, P. Bollansée, D. Weyns, and D. Hughes. Deltaiot: A self-adaptive internet of things exemplar. In *2017 IEEE/ACM 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, pages 76–82. IEEE, 2017.
- [29] M. U. Iftikhar and D. Weyns. ActivFORMS: Active formal models for self-adaptation. In *International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. ACM, 2014.
- [30] U. Iftikhar, J. Lundberg, and D. Weyns. A model interpreter for timed automata. In *International Symposium on Leveraging Applications of Formal Methods*, pages 243–258. Springer, 2016.
- [31] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- [32] P. Jamshidi, J. Cámara, B. Schmerl, C. Kästner, and D. Garlan. Machine learning meets quantitative planning: Enabling self-adaptation in autonomous robots. In *2019 IEEE/ACM 14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, pages 39–50. IEEE, 2019.

- [33] J. O. Kephart and D. M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003.
- [34] D. Kim and S. Park. Reinforcement learning-based dynamic adaptation planning method for architecture-based self-managed software. In *2009 ICSE Workshop on Software Engineering for Adaptive and Self-Managing Systems*, pages 76–85. IEEE, 2009.
- [35] C. Kinneer, Z. Coker, J. Wang, D. Garlan, and C. L. Goues. Managing uncertainty in self-adaptive systems with plan reuse and stochastic search. In *Proceedings of the 13th International Conference on Software Engineering for Adaptive and Self-Managing Systems*, pages 40–50, 2018.
- [36] C. Klein, M. Maggio, K.-E. Årzén, and F. Hernández-Rodriguez. Brownout: Building more robust cloud applications. In *Proceedings of the 36th International Conference on Software Engineering*, pages 700–711, 2014.
- [37] S. Kokoska and D. Zwillinger. *CRC standard probability and statistics tables and formulae*. Crc Press, 2000.
- [38] J. Kramer and J. Magee. Self-managed systems: An architectural challenge. In *Future of Software Engineering*, 2007.
- [39] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.
- [40] M. Kwiatkowska, G. Norman, and D. Parker. Prism 4.0: Verification of probabilistic real-time systems. In *International conference on computer aided verification*, pages 585–591. Springer, 2011.
- [41] H. Lee, R. Grosse, R. Ranganath, and A. Y. Ng. Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. In *Proceedings of the 26th annual international conference on machine learning*, pages 609–616, 2009.
- [42] A. Legay, B. Delahaye, and S. Bensalem. Statistical model checking: An overview. In *International conference on runtime verification*, pages 122–135. Springer, 2010.
- [43] A. Metzger, C. Quinton, Z. Á. Mann, L. Baresi, and K. Pohl. Feature-model-guided online learning for self-adaptive systems. *arXiv preprint arXiv:1907.09158*, 2019.
- [44] D. C. Mohr, M. Zhang, and S. M. Schueller. Personal sensing: understanding mental health using ubiquitous sensors and machine learning. *Annual review of clinical psychology*, 13:23–47, 2017.

-
- [45] G. A. Moreno, J. Cámara, D. Garlan, and B. Schmerl. Proactive self-adaptation under uncertainty: A probabilistic model checking approach. In *Foundations of Software Engineering*, pages 1–12. ACM, 2015.
 - [46] G. A. Moreno, J. Cámara, D. Garlan, and B. Schmerl. Flexible and efficient decision-making for proactive latency-aware self-adaptation. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, 13(1):1–36, 2018.
 - [47] H. Muccini, M. Sharaf, and D. Weyns. Self-adaptation for cyber-physical systems: a systematic literature review. In *Proceedings of the 11th international symposium on software engineering for adaptive and self-managing systems*, pages 75–81, 2016.
 - [48] V. Nair, Z. Yu, T. Menzies, N. Siegmund, and S. Apel. Finding faster configurations using flash. *IEEE Transactions on Software Engineering*, 2018.
 - [49] OpenAI, C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. Józefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H. P. de Oliveira Pinto, J. Raiman, T. Salimans, J. Schlatter, J. Schneider, S. Sidor, I. Sutskever, J. Tang, F. Wolski, and S. Zhang. Dota 2 with large scale deep reinforcement learning. *arXiv eprint arXiv:1912.06680*, 2019.
 - [50] S. J. Pan and Q. Yang. A survey on transfer learning. *IEEE Transactions on knowledge and data engineering*, 22(10):1345–1359, 2009.
 - [51] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of machine learning research*, 12:2825–2830, 2011.
 - [52] W. Qian, X. Peng, B. Chen, J. Mylopoulos, H. Wang, and W. Zhao. Rationalism with a dose of empiricism: combining goal reasoning and case-based reasoning for self-adaptive software systems. *Requirements Engineering*, 20(3):233–252, 2015.
 - [53] F. Quin, T. Bamelis, S. B. Sarpreet, and S. Michiels. Efficient analysis of large adaptation spaces in self-adaptive systems using machine learning. In *2019 IEEE/ACM 14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, pages 1–12. IEEE, 2019.
 - [54] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.
 - [55] W. Richert and B. Kleinjohann. Adaptivity at every layer: A modular approach for evolving societies of learning autonomous systems. In *Proceedings of the 2008 international workshop on Software engineering for adaptive and self-managing systems*, pages 113–120, 2008.

- [56] S. Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.
- [57] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3):211–252, 2015.
- [58] M. Shanker, M. Hu, and M. Hung. Effect of data standardization on neural network training. *Omega*, 24(4):385–397, 1996.
- [59] S. Shevtsov, D. Weyns, and M. Maggio. Simca* a control-theoretic approach to handle uncertainty in self-adaptive systems with guarantees. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, 13(4):1–34, 2019.
- [60] C. Stevens and H. Bagheri. Reducing run-time adaptation space via analysis of possible utility bounds. In *Proceedings of the 42nd International Conference on Software Engineering. ICSE*, 2020.
- [61] E. Strubell, A. Ganesh, and A. McCallum. Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243*, 2019.
- [62] D. Sykes, D. Corapi, J. Magee, J. Kramer, A. Russo, and K. Inoue. Learning revised models for planning in adaptive systems. In *2013 35th International Conference on Software Engineering (ICSE)*, pages 63–71. IEEE, 2013.
- [63] J. Van Der Donckt, D. Weyns, M. U. Iftikhar, and S. S. Buttar. Effective decision making in self-adaptive systems using cost-benefit analysis at runtime and online learning of adaptation spaces. In *International Conference on Evaluation of Novel Approaches to Software Engineering*, pages 373–403. Springer, 2018.
- [64] J. Van Der Donckt, D. Weyns, M. U. Iftikhar, and R. K. Singh. Cost-benefit analysis at runtime for self-adaptive systems applied to an internet of things application. In *ENASE*, pages 478–490, 2018.
- [65] J. Van Der Donckt, D. Weyns, F. Quin, J. Van Der Donckt, and S. Michiels. Applying deep learning to reduce large adaptation spaces of self-adaptive systems with multiple types of goals. In *2020 IEEE/ACM 15th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. IEEE, 2020.
- [66] W. E. Walsh, G. Tesauro, J. O. Kephart, and R. Das. Utility functions in autonomic systems. In *International Conference on Autonomic Computing, 2004. Proceedings.*, pages 70–77. IEEE, 2004.
- [67] D. Weyns. Software engineering of self-adaptive systems: an organised tour and future challenges. *Chapter in Handbook of Software Engineering*, 2017.

- [68] D. Weyns and T. Ahmad. Claims and evidence for architecture-based self-adaptation: A systematic literature review. In *European Conference on Software Architecture*, pages 249–265. Springer, 2013.
- [69] D. Weyns, N. Bencomo, R. Calinescu, J. Cámara, C. Ghezzi, V. Grassi, L. Grunske, P. Inverardi, J.-M. Jezequel, S. Malek, et al. Perpetual assurances for self-adaptive systems. In *Software Engineering for Self-Adaptive Systems III. Assurances*, pages 31–63. Springer, 2017.
- [70] D. Weyns and M. U. Iftikhar. Model-based simulation at runtime for self-adaptive systems. In *2016 IEEE International Conference on Autonomic Computing (ICAC)*, pages 364–373, July 2016.
- [71] D. Weyns, M. U. Iftikhar, D. Hughes, and N. Matthys. Applying architecture-based adaptation to automate the management of internet-of-things. In *European Conference on Software Architecture*, pages 49–67. Springer, 2018.
- [72] D. Weyns and U. Iftikhar. Model-based simulation at runtime for self-adaptive systems. *Proceeding Models at Runtime, Würzburg 2016*, pages 1–9, 2016.
- [73] D. Weyns and U. Iftikhar. Activforms: A model-based approach to engineer self-adaptive systems, 2019.
- [74] D. Weyns, S. Malek, and J. Andersson. Forms: Unifying reference model for formal specification of distributed self-adaptive systems. *ACM Transactions on Autonomous and Adaptive Systems*, 7(1):8:1–8:61, 2012.
- [75] D. Weyns, G. S. Ramachandran, and R. K. Singh. Self-managing internet of things. In *International Conference on Current Trends in Theory and Practice of Informatics*, pages 67–84. Springer, 2018.
- [76] H. L. Younes and R. G. Simmons. Statistical probabilistic model checking with a focus on time-bounded properties. *Information and Computation*, 204(9):1368–1409, 2006.
- [77] J. Zhang and B. H. Cheng. Model-based development of dynamically adaptive software. In *Proceedings of the 28th international conference on Software engineering*, pages 371–380, 2006.